TWIN-64
Architecture Document

Helmut Fieres
October 29, 2026

Contents

Tables of Contents **vii**

List of Figures **ix**

List of Tables **xi**

I Introduction **1**

Introduction **2**
Twin-64 2
Design Goals 3
This book 3

II Architecture **4**

System Organization **5**
A Register–Memory Architecture 5
System 5
Processor 6
Virtual Memory 6
Security and Protection Support 6
Privilege Changes 7
Debugging Support 7
Input/Output 7
Interrupt System 8

Processing Resources **9**
General Registers 9
Control Registers 9
Program State Register 12
Data Types 13
Byte Ordering and Alignment 14
Translation Lookaside Buffer 14
Caches 16
Instruction Set 16

System Memory **18**
Virtual Address Space 18

Address Space Layout 19

Addressing and Access Control **20**
Address Translation 20
Access Control 20
Access Protection 21
Access Check Flow 22

Control Flow **23**
Unconditional Branches 23
Conditional Branches 23
Linkage 23
Cross-Region Branches 24
Privilege Level Changes 24
Traps Associated with Branches 24

Interruptions **25**
Interrupt Handling 25
Interrupt Vector Table 26
Debug and Trace Support 27

III Instruction Set **28**

Instruction Set **29**
Arithmetic Instructions 29
Bit Operations Instructions 29
Immediate Instructions 30
Memory Reference Instructions 30
Control Flow Instructions 31
System Instructions 31
Instruction Formats 32
Synthetic Instructions 33

Instruction Set Descriptions **34**
Instruction Operations 34
Instruction Options 34
Instruction Subfields 34
Constant Expressions 35

Program Instructions **36**
ABR - Add and branch 37
ADD - Integer Addition 38
ADDIL - Add immediate left 39
AND - Boolean Operation 40
B - Branch Unconditional 41
BB - Branch on Bit 42
BE - Branch External 43
BR - Branch Register 44

BV - Branch Vectored . . . 45
CBR - Compare and Branch . . . 46
CMP - Compare . . . 47
DEP - Deposit Bit Field . . . 49
DSR - Double Shift Right . . . 50
EXTR - Extract Bit Field . . . 51
LD - Load Register . . . 52
LDIL - Load Immediate Left . . . 53
LDO - Load Offset . . . 55
LDR - Load Register Reserved . . . 56
MBR - Move and Branch . . . 57
NOP - No Operation . . . 58
OR - Boolean Operation . . . 59
SHLxA - Shift Left and Add . . . 61
SHRxA - Shift Right and Add . . . 62
ST - Store Register . . . 63
STC - Store Register Conditional . . . 64
SUB - Integer Subtraction . . . 65
XOR - Boolean Operation . . . 66

System and Supervisory Instructions **67**
DIAG - Diagnose Instruction . . . 68
FxCA - flush from Instruction or Data Cache . . . 69
FxCE - flush Instruction or Data Cache Entries . . . 70
IxTLB - Insert TLB Entry . . . 71
LPA - Load Physical Address . . . 73
MFIA - Move from Instruction Address . . . 74
MFCR - Move From Control Register . . . 75
MTCR - Move To Control Register . . . 76
PxCA - Purge from Instruction or Data Cache . . . 77
PxCE - Purge Instruction or Data Cache Entries . . . 78
PRB - Probe Virtual Address . . . 79
PxTLB - Purge TLB Entry . . . 80
RFI - Return from Interrupt . . . 81
RSM - Reset System Mask . . . 82
SSM - Set System Mask . . . 83
TRAP - Trap Instruction . . . 84

IV Runtime Environment **85**

Runtime Overview **86**
System Environment . . . 86
Runtime Environment . . . 87
Register Usage . . . 88

Stack Layout **91**
Stack Frame . . . 91
Stack Frame Addressing . . . 92

Stack Frame Marker 92
Parameter Area 93
Local Data Area 93
Spill Area 93

Calling Convention **94**
Calling Sequence 94
Local Call 94
Parameter passing 96
External Calls 96
Linkage Table 98
Function Labels 99
Privilege Level Changes 100
Design Rationale 100

V I/O Subsystem **102**

Input/Output **103**
Concepts 103
I/O Address Space 104
Hard physical address range 105
Broadcast address range 106
Soft physical address range 106
Supervisor Register Set 107
Interrupt Processing 108
Design Rationale 109

VI Firmware Architecture **110**

Firmware Architecture **111**
PAL/SAL Address Space 111
PAL/SAL Calling Convention 111

Processor Abstraction Layer **112**
Concepts 112
Design Rationale 112
PAL Functions 112
PAL code area layout 112
System startup 113

System Abstraction Layer **114**
Concepts 114
Design Rationale 114
SAL Functions 114

VII Simulator Environment **115**

Twin-64 Simulator **116**

Simulator Windows **117**
Window . . . 117
Window Stacks . . . 118
Command Window . . . 119
Processor Window . . . 119
TLB Window . . . 120
Memory Window . . . 121
Text Window . . . 122

Simulator User Interface **123**
Command Line . . . 123
Data Types . . . 123
Expressions . . . 124
Predefined Variables . . . 125
User Defined Variables . . . 126
Predefined Functions . . . 126
Memory References . . . 126

Simulator Line Commands **128**
Command Summary . . . 128
Command Syntax . . . 129
ASSERT - Ensure a condition . . . 130
CHECK - Test a condition . . . 131
DM - Display Memory . . . 132
DMOD - List Module Info . . . 133
DO - perform command from history stack . . . 134
DWIN - List Window Info . . . 135
ENV - Manage Environment Variables . . . 136
EXIT - Leave Simulator . . . 137
HELP - list help info . . . 138
HIST - List Command History . . . 139
ITLB - Insert TLB Entry . . . 140
LOADELF - load ELF file . . . 141
LOG - Write a log entry to the log file . . . 142
Mx - modify memory . . . 143
MR - modify register . . . 144
NMOD - create a new T64 module . . . 145
PTLB - Purge TLB entry . . . 146
REDO - Redo command from history stack . . . 147
RMOD - remove a T64 module . . . 148
RESET - Reset T64 modules . . . 149
RUN - Run the simulation . . . 150
STEP - Step a T64 module . . . 151
W - Display an expression value . . . 153

XF - Execute commands from a file . . . 154

Simulator Window Commands **155**
Command Summary . . . 155
Command Syntax . . . 156
CWC - Clear command window . . . 157
CWL - Set command window lines . . . 158
WB - Window backward . . . 159
WC - Set current window . . . 160
WD - Window disable . . . 161
WDEF - Set window defaults . . . 162
WE - Enable Window . . . 163
WF - Window forward . . . 164
WH - Window home . . . 165
WJ - Window jump . . . 166
WK - Window kill . . . 167
WL - Set window lines . . . 168
WN - Create a window . . . 169
WOFF - Turn window display off . . . 170
WON - Turn window display on . . . 171
WR - Set window radix . . . 172
WS - Move windows to stack . . . 173
WSD - Disable Window Stack . . . 174
WSE - Enable window stack . . . 175
WT - Window toggle . . . 176
WX - Window exchange . . . 177

Simulator Breakpoints **178**
Motivation . . . 178
System-Wide Breakpoints . . . 178
Instruction Breakpoints . . . 179
Data Breakpoints . . . 179
Breakpoint State . . . 179
Breakpoint Representation . . . 180
Debugging Workflow . . . 180
Command Summary . . . 181
BD - Disable simulator breakpoint . . . 182
BE - Enable simulator breakpoint . . . 183
BK - Remove simulator breakpoint . . . 184
BL - List simulator breakpoints . . . 185
BN - Create a simulator breakpoint . . . 186

Simulator Predefined Functions **187**
ASM — Inline assembler . . . 188
DISASM — Disassemble instruction . . . 189

Simulator Variables **190**
Predefined Variables . . . 190

Environment Variables 190

Simulator Configuration **192**
Program Parameters 192
Configuration Files 192
Log Files 193
Configuration Example 193

VIII Appendix **194**

Appendix - Instruction Commentary **195**
Arithmetic Operation Instructions 195
Boolean Operation Instructions 195
Bit Operation Instructions 196
Comparison and Branch Condition Encoding 196
Control Flow Instructions 197
Immediate Instructions 197
Pseudo-OpCode Load Immediate 198
Memory Reference Instructions 198
Pseudo-OpCode Load/Store with large offset 199
Load-Reserved and Store-Conditional Instructions 200

Appendix - Instruction Notation Routines **202**

Appendix -Debug and Trace Support **204**
B - Break Enable Bit 204
T - Trace (Single-Step) Enable Bit 205
Interaction Between B and T 205
Software Breakpoint Handling 206
Single-Step Execution 206
Design Rationale 206

Appendix - Diagnostic Functions **207**

Appendix - Translation Lookaside Buffer **208**

Appendix - Processor Design **209**
Pipeline Stages 209
Single Issue Pipeline 210
Dual Issue Pipeline 211
Design Rationale 214

Appendix - Glossary **215**

Appendix - Notes **217**

List of Figures

1 General registers . . . 9
2 Control registers . . . 10
3 Program State Register . . . 12
4 Data types . . . 14
5 Big endian mode . . . 14
6 Tlb info . . . 15
7 Virtual Memory Address Range . . . 18
8 Virtual page address . . . 18
9 Memory Address Ranges . . . 19
10 Address translation . . . 20
11 Access control information field . . . 21
12 Protection Id . . . 21
13 Address translation . . . 22
14 System Environment . . . 86
15 Runtime Environment . . . 87
16 General Register Usage . . . 89
17 Stack Frame . . . 91
18 Stack Frame Marker . . . 92
19 Argument Passing . . . 96
20 Linkage Table . . . 99
21 I/O Address Space . . . 104
22 Hard physical Address Page . . . 105
23 I/O Address Space . . . 106
24 Simulator Environment . . . 116
25 Simulator Windows . . . 117
26 Simulator Window Stacks . . . 118
27 Simulator — Main Window . . . 119
28 Simulator CPU Window . . . 120
29 Simulator TLB Window . . . 121
30 Simulator Memory Window . . . 121
31 Simulator Text Window . . . 122
32 Single Instruction Pipeline . . . 210
33 Dual Instruction Pipeline . . . 213

List of Tables

1 Processor Configuration Register Fields . . . 11
2 Status Field Bits . . . 13
3 TLB Fields . . . 15
4 Access type checking . . . 21
5 Trap Table . . . 26
6 Twin-64 Exception Priority . . . 27
7 Supervisor Register Set . . . 107
8 Predefined Variables . . . 125
9 Simulator Line Commands . . . 128
10 Simulator Window Commands . . . 155
12 Predefined Simulator Functions . . . 187
13 Predefined Simulator Variables . . . 190
14 Environment Variables . . . 190
15 Simulator command-line options . . . 192
16 Load Immediate Pseudo OpCode . . . 198
17 Load Immediate Pseudo OpCode . . . 199
18 Instruction Notation Routines . . . 202
19 Behavior of the B and T Bits . . . 205
20 Diagnostic Functions . . . 207

Part I

Introduction

Introduction

Designers of CPUs in the seventies and early eighties almost all used a microcoded approach with hundreds of instructions. RISC was just beginning to emerge. Registers were typically not fully general-purpose but often had special functions or meanings, and many instructions were rather complicated, though designers believed them useful. Compiler writers, however, often used only a small subset, ignoring these complex instructions because they were too specialized. In the late eighties and nineties the shift moved to RISC-based designs, with large sets of registers, fixed instruction word lengths, large virtual memory address ranges, and instructions that were in general much more pipeline-friendly. Today, processors are highly complex, with superscalar deep pipelines and multilevel cache hierarchies—just a few of the hallmarks of modern CPU design. CPU designs have become so large and complex that no single person can completely understand them. But what if there were a simple design that one person could fully understand?

Twin-64

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a register–memory model and segmented virtual memory. The design is heavily influenced by Hewlett-Packard's PA_RISC architecture, which was initially a 32-bit RISC-style load/store machine. Many of the key architectural concepts originated there. Other processors, including DEC Alpha, IBM PowerPC, and more recently RISC-V, also influenced the design or were studied for their architectural approaches. In addition, the Blitz-64 architecture, developed at Portland State University, provided inspiration through its exclusive use of 64-bit signed arithmetic, thereby avoiding many of the problems associated with mixing signed and unsigned operations.

Three design principles guided the development of the Twin-64 architecture. First, the instruction set should be as regular and orthogonal as possible, making instructions easy to understand, decode, and combine. Second, the hardware implementation should remain simple and predictable, avoiding unnecessary complexity where practical. Third, instruction-level parallelism should be exposed primarily by software rather than discovered dynamically by hardware. These principles influenced both the instruction set architecture and the processor implementations presented later in this manual.

Where does the name *Twin-64* come from? The 64'' refers to the processor's native 64-bit architecture. The meaning of Twin'' becomes apparent in the implementation chapters, where one possible realization of the architecture employs a dual-issue pipeline capable of executing two compatible instructions in parallel. This dual-issue organization represents the architectural idea behind the name, although the instruction set itself is not tied to a particular implementation. Simpler implementations, such as a single-issue processor with one arithmetic logic unit, are equally valid realizations of the Twin-64 architecture.

Design Goals

While it is not a design goal to build an industry-strength, competitive CPU, the CPU should nevertheless be useful and largely follow established design practices. For example, instructions should not be invented just because they seem useful, but rather designed with the assembler and compilers in mind. Instruction set design is CPU design. Twin-64 instructions will be hardwired and are in general pipeline-friendly, avoiding data and control hazards and stalls where possible.

This book

This document serves as a comprehensive guide to the Twin-64 processor, its instruction set, and runtime environment. It is organized into several parts, each focusing on a key aspect of the system.

Part Two introduces the Twin-64 architecture, providing an overview of the overall system and processor's structure, core components, and design philosophy. This sets the stage for understanding how instructions are executed and how data moves through the system.

Part Three presents the instruction set in detail. It covers computational, immediate, memory reference, branch, and system control instructions. These chapters serve as the definitive reference for programmers and designers alike, offering precise information on encoding, operation, and usage.

Part Four discusses the runtime environment and software conventions. It explains program interaction with Twin-64, including calling conventions, exception handling, and runtime support for higher-level languages.

Part Five explores the I/O subsystem, detailing the mechanisms by which the processor communicates with external devices and handles input/output operations efficiently. (??? rather module as the overarching name ?)

Part Six presents the T64 firmware architecture. (??? PAL, SAL, etc. Reset sequence, etc.)

Part Seven presents a reference implementation of the architecture and instruction set processor in a simulator environment.

Finally, the **Appendix** provides supplementary material, including additional design notes, tables, and clarifications to support the main text and offer deeper insight into the Twin-64 system.

Part II

Architecture

System Organization

This chapter provides an overview of the Twin-64 system architecture and its major design elements. It begins by describing the register–memory programming model and the overall system organization built around a shared system bus. The processor's internal structure and its role in address generation and execution are then outlined.

Subsequent sections introduce the virtual memory model, including address regions, page-based translation, and protection mechanisms. The chapter also explains Twin-64's security model, privilege levels, and the gateway mechanism used for efficient privilege transitions. Support for debugging, memory-mapped input/output, direct memory access, and the interrupt system is described last, completing a high-level view of how computation, protection, and external interaction are coordinated in a Twin-64 system.

A Register–Memory Architecture

Modern RISC CPUs are typically out--of--order load / store designs and feature a large number of general-purpose registers. Operations solely take place between registers. Twin-64 follows a slightly different route. It has fewer general registers, and in addition to register–-register computation, a register–-memory model is also supported. In contrast to similar historical register–-memory designs, Twin-64 has no operations that both read from and write to memory in the same instruction. For example, there is no "increment memory" instruction, since this would require two memory operations and is generally not pipeline-friendly.

Twin-64 offers only a few addressing modes for the operand, combined with a fixed instruction word length. For most instructions one operand is a register, while the other is a flexible operand that can be an immediate, another register, or a memory address from which data is obtained or stored. The result is placed in the register, which is also the first operand. These types of machines are also called two-address machines.

System

A Twin-64 system is organized around a central system bus that interconnects the key components of the architecture. Processors, main memory modules, and I/O modules are all attached as peers to this bus, enabling uniform communication and data exchange. Larger configurations may also include I/O adapters, which bridge slower peripheral buses to the central system bus while preserving a consistent programming model.

Processors initiate memory and I/O transactions via the bus and can also receive external events such as interrupts or system resets through the same interface. This modular structure provides

flexibility in system design, allowing configurations to scale from simple processor–-memory systems to complex multiprocessor environments with diverse I/O subsystems.

Processor

The Twin-64 processor implements the instruction set architecture and provides the computational resources of the system. It contains a general register set, control and status registers, and the program state register. To support virtual memory, each processor includes a Translation Lookaside Buffer (TLB) that translates virtual addresses into physical addresses. Instruction and data caches, while optional in principle, are essential for performance and tightly integrated with the memory interface. Regardless of configuration, all processor-generated addresses are virtual and must be resolved by the TLB before memory access.

Virtual Memory

Twin-64 offers a large global address range, organized in regions. A region number and offset together form the global virtual address. regions are also associated with access rights and protection identifiers. The CPU itself can run in user or privileged mode. Twin-64 always generates virtual addresses at the CPU level. The global virtual memory range is a 52-bit space organized into a 20-bit region ID and a 32-bit byte offset. There are 2^{20} regions with 2^{32} bytes each.

Virtual addresses are translated to physical addresses by the TLB when memory is referenced. For memory management purposes, the virtual address range can also be viewed as a set of pages. A region therefore contains 2^{20} pages of 4 KB each. A virtual page number, combining the region and the page number within that region, is a 2^{40} value. Address translations are performed at the page level.

The first regions in the virtual address space are special in that they actually represent the physical address range. An address in that range bypasses the TLB and accesses physical memory directly. Access is only allowed at the highest privilege level.

A TLB can optionally support different page sizes in addition to the architected 4 KB page size. It is very common for the operating system and data to be mapped in consecutive physical pages, which can then be covered by a larger page size.

Security and Protection Support

Twin-64 implements a protection model along two dimensions. The first dimension is access type control and privilege-level checking. Each page is associated with a page type. Page types include read-only, read–write, execute, and gateway. There are two privilege levels: user mode and supervisor mode. Access type and privilege level together form the access control information, which is checked for each instruction and data memory access.

The second dimension of protection is the region ID itself. Twin-64 allows a set of region IDs to be recorded in processor control registers. For each access to a region that has the protection check bit set, one of the region ID control registers must match the region ID. If not, a protection violation trap is raised.

Privilege Changes

Instructions can only access data or execute instructions when the privilege level matches the required privilege level. Two or four levels are the most common implementations. Changing between levels is often done with a kind of trap operation to higher-privilege software, for example the operating system kernel. However, traps are expensive, as they cause the CPU pipeline to be flushed when entering a trap handler.

Twin-64 implements gateways for privilege promotion. A special option on the unconditional branch instruction raises the privilege level when the instruction is executed on a gateway page, setting the privilege level to that of the gateway page. Returning from a higher privilege level to a code page with lower privilege level automatically resets the lower privilege level. A branch to a higher-privilege page that is not a gateway page results in a privilege violation trap.

Debugging Support

A fundamental requirement is the ability to set instruction and data breakpoints. An instruction breakpoint is triggered by encountering the `TRAP` instruction. The break instruction causes an instruction break trap and passes control to the trap handler. Since breakpoint trap instructions are normally not present in the instruction stream, they must be inserted dynamically in place of the instruction normally found at that address. Upon continuation, if the breakpoint is still valid, it must be reinserted after the original instruction is executed. To facilitate this mechanism, the break and trace enable bits in the status word provide support for debug and trace tools.

The trace bit controls forward progress; the break-enable bit controls breakpoint interpretation. Both are required to ensure predictable debugger behavior in the presence of multiple breakpoints and nested execution contexts.

Input/Output

Twin-64 implements a memory-mapped I/O architecture. A portion of the physical address space is reserved for I/O modules, which respond to memory load and store instructions directed to their assigned addresses. Standard page-level protection applies during address translation, ensuring that only privileged software (typically device drivers) can safely access I/O regions. User programs may be granted direct access to I/O by mapping a user-level I/O page into virtual memory, without compromising overall system integrity.

Direct I/O is performed using the standard load and store instructions to access I/O modules. These operations are entirely controlled by software and may be executed in user mode if the virtual mapping allows it.

Direct Memory Access (DMA) modules can transfer data between memory and I/O devices independently of the processor. DMA transfers operate on physical memory, requiring that the relevant pages be locked and protected from conflicting access by the operating system. DMA transactions are initiated by issuing DMA commands, and command chaining is supported to enable complex or continuous data transfers without processor intervention.

Interrupt System

Interrupts and exceptions are events that occur asynchronously to instruction execution. Depending on the type, they occur either during the execution of an instruction or between instructions. An example of the former is an arithmetic overflow trap; an example of the latter is an external interrupt. Depending on the interrupt or exception type, the instruction is either restarted or execution continues with the next instruction.

Twin-64 implements a simple interrupt system. Each processor features an interrupt input register and an interrupt mask register. When an I/O module receives a command, it also receives information about which interrupt bit to set and which target module ID to send the interrupt to upon completion of the request. The interrupt data is compared to the interrupt mask register. If the particular bit is enabled and interrupts are globally enabled, the target processor enters the interrupt handler.

Since raising an interrupt is simply a matter of storing the interrupt request into the processor's interrupt register, I/O modules as well as processors themselves can raise interrupts on a target processor. Processors can also send interrupts to their peers, enabling low-level processor-to-processor communication.

Processing Resources

This chapter describes the processing resources and data types in a Twin-64 system. Twin-64 provides a set of general-purpose registers, a set of control registers, and the Program State Word (PSR). The general-purpose registers are used for computation and address storage, the control registers manage processor state, and the PSR holds both the current instruction address and processor status.

General Registers

Twin-64 features a set of 16 general purposes registers. They are used for all computations including address arithmetic.

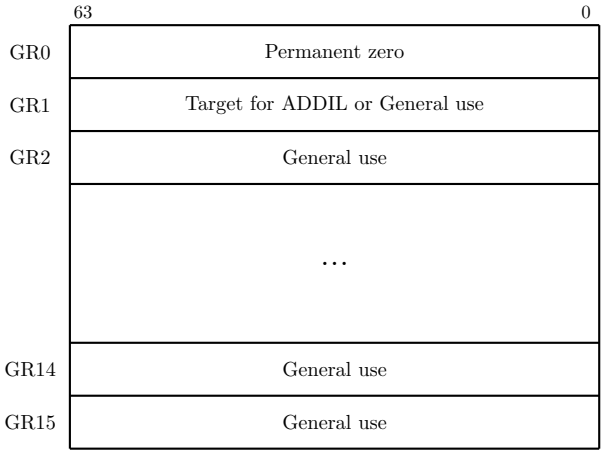


Figure 1: General registers

Some general registers have dedicated purposes. Register zero always returns a zero value when read, and writing to it has no effect. Although the CPU has only 16 general registers, implementing such a register greatly simplifies instruction design, for example by providing a target when the result is not needed. General register one is a scratch register and also serves as an implicit target for certain instructions. Other general registers may have dedicated roles as defined by the runtime architecture conventions.

Control Registers

Twin-64 has 16 defined control registers, numbered `CR0` to `CR15`. Two instructions allow moving data between a control register and a general register in either direction. A move to a control register always transfers the full 64-bit word. A move from a control register to a general register

transfers the value right-aligned to the size of the control register. The following figure depicts the control registers defined for Twin-64.

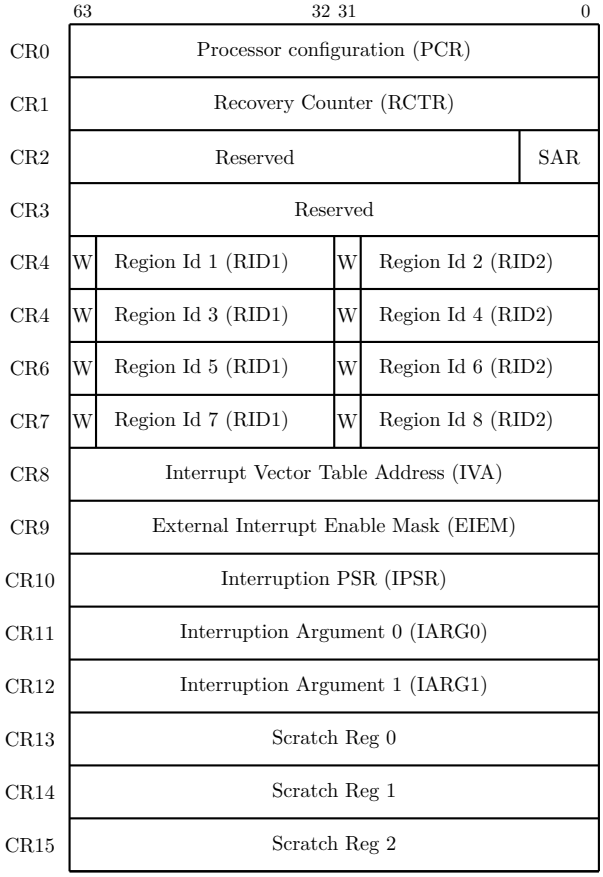
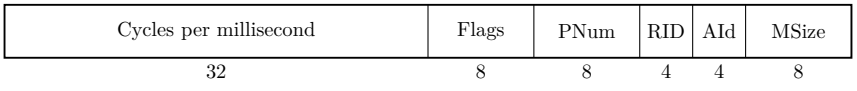


Figure 2: Control registers

Note: We may need more than 16 control registers. Under evaluation.

Processor Configuration Register

The processor configuration register contains information about the current system and processor hardware setup. Some of its fields are read-only and set directly by the hardware, while others can be modified by processor-dependent code routines.



The following table lists the processor configuration fields.

Table 1: Processor Configuration Register Fields

Field	Purpose
Memory Size (MSize)	The available physical memory in 4 Gbyte increments.
Architecture Id (AId)	...
Processor Id Id (RID)	...
Processor Core Num	in a multiprocessor configuration, the unique processor number is assigned during system startup.
Flags (tbd)	e.g default endian, etc.
Cycles per millisecond	the cycle per millisecond specifies the number of processor cycles for a millisecond.

Recovery Counter

When enabled, the recovery counter decrements on each instruction cycle. When the leftmost bit becomes a one and the R bit in the processor status word is set, a recovery trap is scheduled.

Shift Amount Register

The shift amount register (`SAR`) is a 6-bit register used by the extract, deposit and double shift instruction when a dynamic position is specified in the instruction. The valid number range is zero to 63.

Region Id Registers

Twin-64 allows to record a set of up to 8 region IDs in the processor control registers `CR4` to `CR7` which are accessible to the currently executing task. For each access to a region that has the protection check bit set, one of the protection Id control registers must match the region Id. If not, a protection violation trap is raised. The rightmost bit of each of the eight RIDs is the write disable (W) bit. When this bit is set, that RID data cannot be used to grant write access. See also the chapter on addressing and access control.

Interrupt Vector Table Address

The Interruption Vector Table Address (IVA) control register, `CR8`, holds the base address of an array of service routines assigned to the different interruption classes. The lower 12 bits of the IVA are reserved and must be set to zero, meaning any address written to it must be aligned to

a page boundary. The IVA is always located in region zero. The array of interruption service routines is indexed by interruption numbers, as described in the chapter on interrupts.

External Interrupt Enable Mask

The External Interrupt Enable Mask (EIEM) is stored in `CR9`. It is a 64-bit register, with each bit corresponding to one of the 64 external interrupts. A cleared bit in the EIEM masks the external interrupt associated with that position.

Interruption PSR

`CR10` is the interruption processor status word register. Upon an interrupt, the address of the interrupted instruction and the processor status bits are saved to this control register.

Interruption Argument Registers

`CR1` ad `CR2` are the interruption argument registers. They are used to pass an instruction and a virtual address to an interruption handler. The values of these registers for each interruption class are described in the chapter on interrupts.

Scratch Registers

Control registers `CR13`, `CR14`, and `CR15` are scratch registers used by interrupt handlers or similar routines. `CR13` and `CR14` are privileged and can only be accessed from code running at a privileged level, whereas `CR15` is accessible from any privilege level.

Program State Register

The processor state register (`PSR`) contains the virtual address of the current instruction along with the processor status flags.

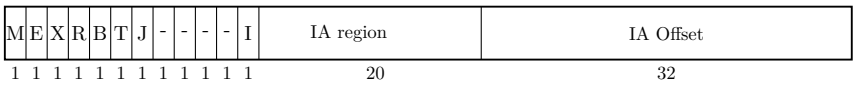


Figure 3: Program State Register

Note: Another option is to have a separate status word. Under evaluation.

Instruction Address

The instruction address portion of the PSR registers contains the region id and the byte offset into the region for the currently executing instruction.

Processor Status

Table 2: Status Field Bits

Bit	Purpose
M	High priority machine check. When set, high-priority machine checks are masked. This bit is set after an HPMC and cleared after all other interruptions.
E	Little endian access enable. When set, memory access is in little endian format, else big endian format. Endian support is an optional feature. If not implemented, all access is in big endian format.
X	When set, the processor runs in user mode.
R	Recovery counter enable. When set, the recovery counter trap is issued when the recovery counter becomes negative.
B	Break enable. When set, execution of a BREAK instruction causes a breakpoint exception.
T	Trace enable. When set, the processor generates a trace exception after the next instruction completes, regardless of the instruction type.
J	Taken branch trap enable. When set, the processor generates a trap on each taken branch.
I	When set, external interrupts are enabled.
V	Reserved.

Data Types

The fundamental data type is a 64-bit signed integer, with the most significant bit on the left and the least significant bit on the right. All computations are performed on 64-bit signed integers. Memory accesses can be performed at the granularity of a double-word, word, halfword, or byte. Data loaded as a word, half-word, or byte is normally sign-extended to 64 bits. The load instruction has however an option to zero-extend the data loaded. Store operations write the corresponding value to memory without modification.

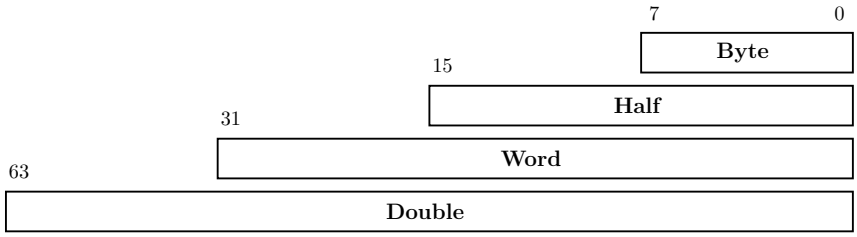


Figure 4: Data types

All memory addresses are expressed as bytes addresses. Address computation uses a 32-bit unsigned math, i.e. numbers wrap around in a 32-bit space and never cross region boundaries.

Byte Ordering and Alignment

Twin-64 is a big-endian machine. The following figure shows the memory access for load operations.

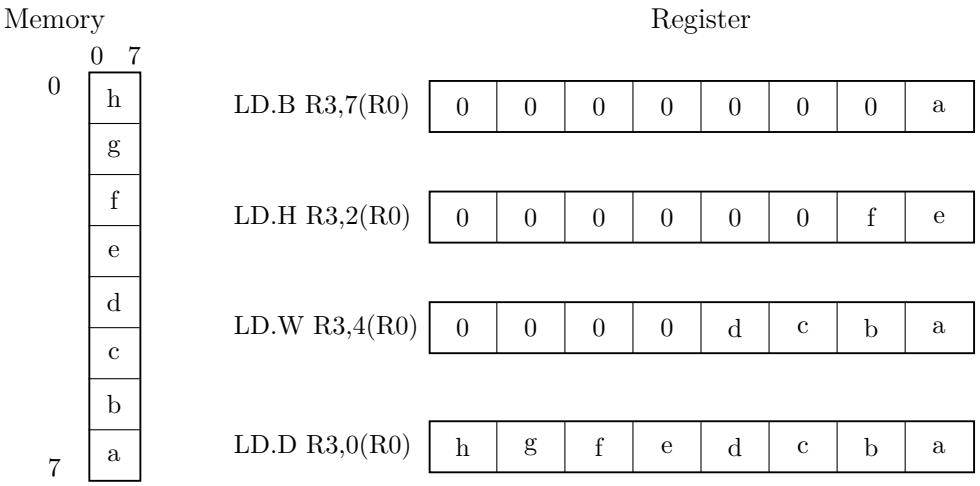


Figure 5: Big endian mode

Twin-64 enforces natural alignment for memory accesses. Byte, half-word, word, and double--word data types must be aligned to 1, 2, 4, and 8-byte boundaries respectively. If an access does not meet the required alignment, a data-alignment trap is raised.

The processor status register reserves a bit position, "E", for future support of mixed big and little endian memory access. Currently this bit is always set to zero.

Translation Lookaside Buffer

Virtual address translations are stored in a translation lookaside buffer (TLB). The TLB is essentially a cache of page table entries representing the virtual pages currently mapped to physical addresses. Depending on the hardware implementation, there may be a single combined

TLB or separate instruction and data TLBs. Each TLB entry stores the virtual page number (VPN) along with its attributes and the corresponding physical page number (PPN). The current architecture supports a 40-bit physical address space, allowing a maximum of 1 Tb of memory.

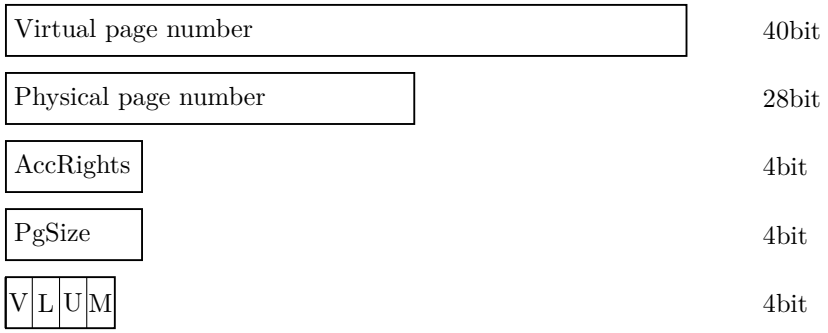


Figure 6: Tlb info

To reduce TLB processing overhead, the TLB is capable of storing virtual address ranges. There is support for a larger page sizes. The supported sizes are from 4 Kbyte to 4Gb in factors of four.

Table 3: TLB Fields

Field	**Purpose**
VPN	The virtual page number.
PPN	The physical page number.
Access Rights	Encodes the access type (read, write, and execute) and the required privilege level for the operation.
Page Size	4-bit field encoding the supported page size: `( 1U << ( pSize * 2 )) * T64_PAGE_SIZE` A page size is not allowed to cross region boundaries. The virtual address in the TLB entry must be aligned with selected the page size.
V	When set, the entry is valid.
L	When set, the TLB entry is locked and will not be selected during a replacement candidate search.
U	When set, access to the page represented by the TLB entry is not cached.
M	When set, the page has been modified.

The first 256 memory regions, region 0 to 255, are special in that they bypass the TLB entirely. A virtual address in the range 0x00_0000_0000 to 0xFF_FFFF_FFFF directly maps to the physical address of the same range. The address range represents physical memory and can only be accessed in privileged mode.

Caches

Twin-64 is a cache-visible architecture that supports both unified and separate instruction and data caches. While the hardware directly manages cache line insertion and replacement, software has explicit control over flushing and purging cache line entries. Twin-64 caches are virtually indexed and physically tagged. To accelerate memory access, the cache uses a portion of the virtual address to index into the arrays in parallel with the TLB lookup. A cache hit occurs when the physical address tag in the cache matches the physical page address from the TLB.

The architecture does not mandate specific cache line sizes, cache organization, or hierarchy models. Instead, it defines instructions for cache operations, such as deletion or flushing of cache lines. Cacher insertion is handled transparently by hardware.

For a virtual address, the TLB entry indicates whether the corresponding data should be cached. All physical memory address ranges as well as the I/O address range are not cached. Care needs therefore to be taken when referencing a data its by its virtual address and the mapped physical address. The corresponding cache entry perhaps needs to be flushed first.

Instruction Set

Twin-64 features a simple and efficient instruction set. All instructions are of fixed length, specifically a 32-bit word. The total number of instructions is relatively small, but many include execution options that make them highly versatile. Great care has been taken to ensure these options do not significantly complicate the pipeline or lengthen the data path.

Unlike CISC-style processors that support a wide variety of addressing modes, Twin-64 provides just two virtual memory addressing modes, both based on a simple base–-offset calculation. No indirection or mode requiring a memory operand to compute an address is part of the architecture.

The instruction set is organized into five groups:

- **Computational instructions:** Arithmetic, boolean, and bit-manipulation operations such as extract and deposit. Twin-64 supports only 64-bit signed arithmetic. Any numeric overflow results in a trap.
- **Immediate instructions:** Instructions for constructing immediate values up to 64 bits. Several instructions embed immediate fields within the instruction word, but a full 64-bit constant requires a sequence of two or three instructions. There are also immediate instructions that support 32-bit address arithmetic, where calculations wrap around within the 32-bit offset field and never overflow into the region identifier portion of the address.
- **Memory reference instructions:** Load and store operations for both virtual and physical memory. These transfer data between memory and general registers in sizes of double--word, word, halfword, or byte. The design resembles a load/store architecture, but unlike strict load/store machines, several computational Twin-64 instructions also allow

memory operands as one of their operands. No instruction, however, both reads from and writes to data memory in the same operation.

- **Control flow instructions:** Both unconditional and conditional branches are supported. Unconditional branches may optionally save the return address in a general-purpose register. Conditional branches compare two operands and transfer control if the condition is satisfied, eliminating the need for a separate compare instruction. Conditions are either used directly or stored as result in a register. There are no status word bits which record flags that can be tested.
- **System control instructions:** Instructions for managing processor resources such as the TLB, cache, and control registers. Control registers can be accessed directly. The TLB can be updated with insert and remove operations; and the cache can be flushed or purged. Privileged software can flush and purge cache data. Additional instructions generate traps or provide diagnostic functions for examining processor state. Most system instructions require execution in privileged mode.

The book section on the instruction set will describe each instruction group in detail, with formats, descriptions, and examples.

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range, organized into a 20-bit region identifier and a 32-bit offset within that region. The smallest unit of access is one byte. The following figure illustrates the structure of a virtual address and how the region data is selected.

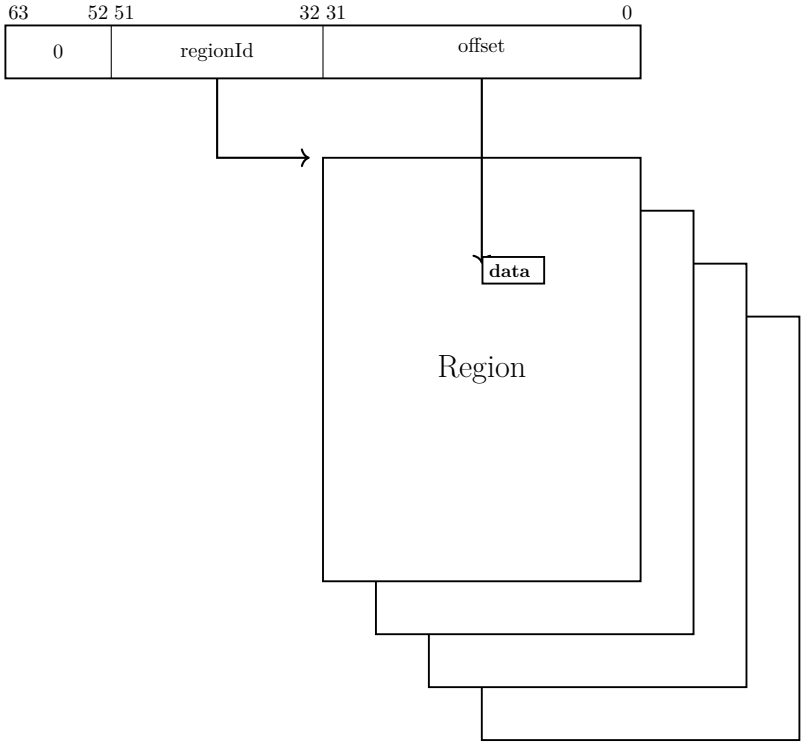


Figure 7: Virtual Memory Address Range

A virtual address is translated into a physical address. For translation purposes, the virtual address range can also be viewed as a set of virtual pages. The architected page size is 4 KBytes, with a 40-bit virtual page number and a 12-bit page offset.

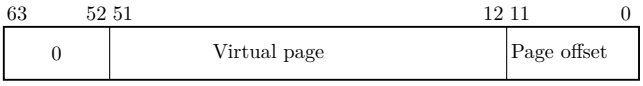


Figure 8: Virtual page address

Address Space Layout

Twin-64 architecture features a 52-bit address space divided into 2^{20} regions of 4Gbyte each. The following figure depicts the overall address space layout.

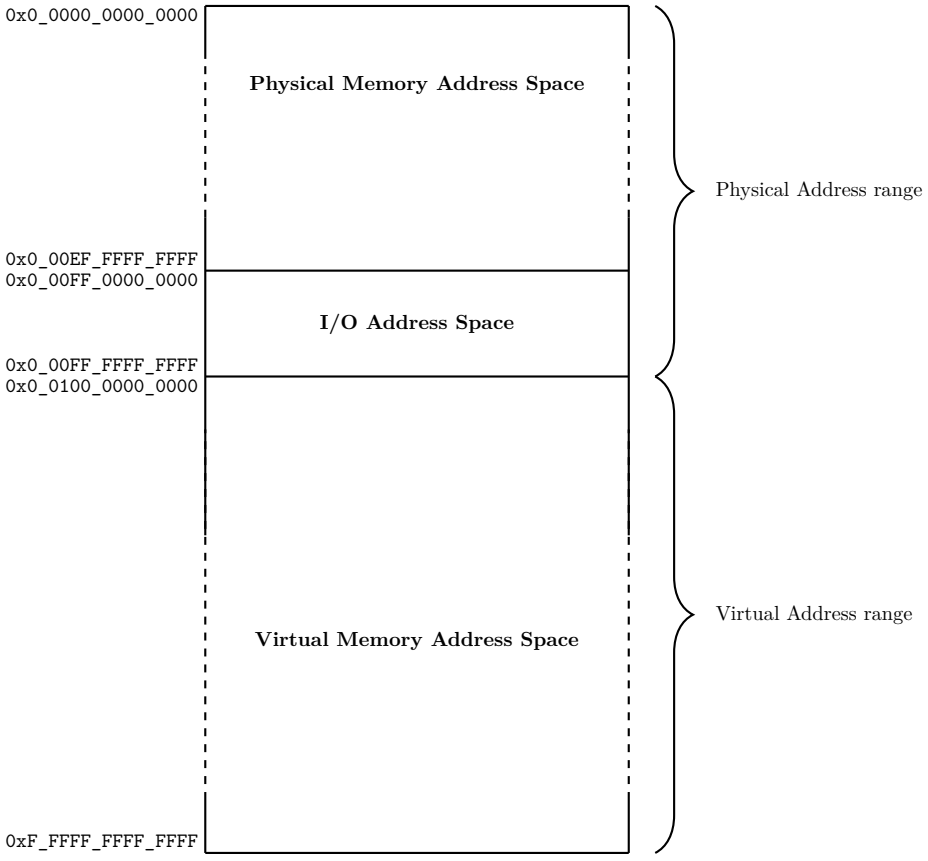


Figure 9: Memory Address Ranges

Region IDs 0 to 254 represent the physical address space, with a maximum size of 1 TB. These regions are directly mapped to physical addresses, without TLB translation or access-rights checking. Only privileged-mode code can access these regions. Control Register 0 contains a field that is set to the actual physical memory size installed in the system.

Twin-64 uses a memory-mapped I/O architecture. The I/O address space occupies the top region, region 255, of the physical address space. I/O modules allocate addresses for their registers and storage within this range, and access to this area is uncached.

The remaining address space is dedicated to virtual memory. Virtual storage is allocated dynamically, and the TLB provides the necessary translation to physical addresses while enforcing access rights.

Addressing and Access Control

The CPU generates 52-bit virtual addresses. The first 256 regions are directly mapped and bypass the virtual to physical address translation. To access this range directly, the processor must run in privileged mode. All other memory access must be translated and checked for valid access rights. This chapter describes the address translation process, as well as the access-rights and protection mechanisms.

Address Translation

Each virtual memory access generated by the processor must be translated into a physical address. The translation process maps a virtual page to a physical page. An address in the first 256 regions bypass the TLB, meaning that the virtual address is identical to the physical address.

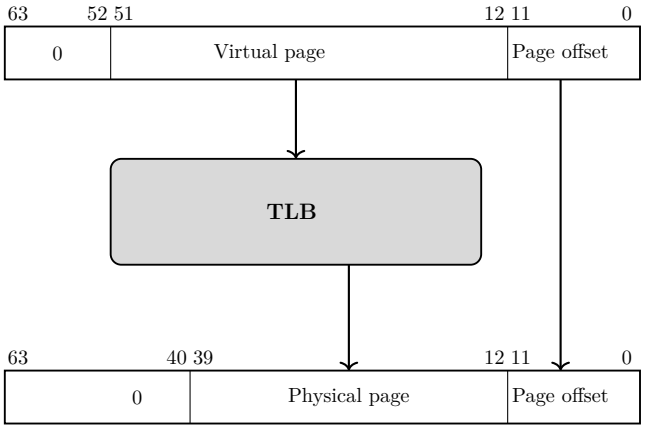


Figure 10: Address translation

Access Control

Twin-64 implements a protection model along two dimensions. The first dimension is **access control** and **privilege level**. Each virtual page is associated with a page type: read-only, read-write, execute, or gateway. The page type is encoded in the AccType field, where 0 represents read-only access, 1 represents read-write access, 2 represents execute access, and 3 represents gateway access. Each memory access is checked against the allowed access type defined in the page descriptor.

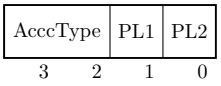


Figure 11: Access control information field

There are furthermore two privilege levels: user mode and supervisor mode. The combination of access type and privilege level forms the access control information, which is verified for every instruction and data memory access. For read access, the privilege level must be at less than or equal as the PL1 field; for write access, it must be less than or equal as the PL2 field. Exceptions are execute and gateway pages, where write access is allowed only for privileged code. For execute access, the privilege level must be less than or equal than the PL1 field.

If the instruction's privilege level falls outside these bounds, a privilege violation trap is raised. If the page type does not match the instruction's access type, an access violation trap is raised. In both cases, the instruction is aborted, and a trap occurs. A CPU running in privilege mode bypasses the access check.

Table 4: Access type checking

Page Type	**read**	**write**	**execute**
`0 - read-only`	`PL <= PL1`	`PL == 0`	`not allowed`
`1 - read-write`	`PL <= PL1`	`PL <= PL2`	`not allowed`
`2 - execute`	`PL <= PL1`	`PL == 0`	`PL <= PL1`
`3 - gateway`	`PL <= PL1`	`PL == 0`	`PL <= PL1`

Access Protection

The second dimension of access protection is **region ID checking**. Twin-64 allows the processor to store up to eight region ID values in the control registers `CR4` through `CR7`, which are accessible to the currently executing process. For each data memory access to a region at least one of the protection ID registers must contain a matching region ID. Otherwise, a protection violation trap is raised.

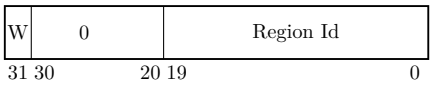


Figure 12: Protection Id

The rightmost bit of each region ID register stored in the one of the control registers `CR4` to `CR7` serves as a write-disable (W) bit. When the W bit is set, that protection ID cannot be used to grant write access.

Protection ID checking is typically used to enforce controlled access to shared or private memory regions. A common example is a stack data region accessible at user privilege level. Since the

CPU implements a global virtual memory address space, any task could potentially access the stack of another task. By enabling protection ID checking and loading the corresponding object/region ID into one of the protection ID registers for the owning process, only that user task is permitted to access its stack data region with a matching protection ID.

Protection ID checking is not performed for instruction memory access. Code is in general a shared object and only access rights are checked.

Access Check Flow

The following figures summarizes the protection and access mode checking steps during address translation.

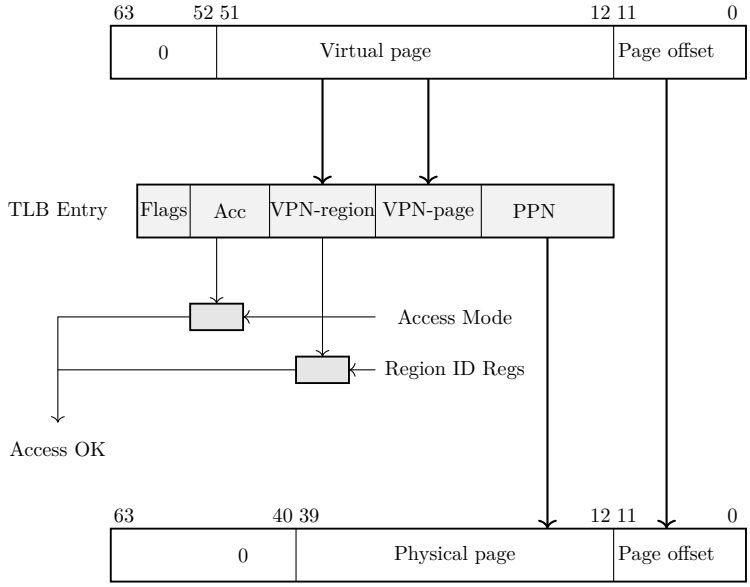


Figure 13: Address translation

For a translation found TLB, the region portion of the virtual address is compared against the set of region Id registers. If there is no match, a region Id trap is raised. The access mode of the instruction is compared against the access rights and TLB flags of the TLB entry. If the access mode does not match an access rights trap or a privilege violation trap is raised.

Control Flow

In Twin-64, control normally proceeds to the next sequential instruction in memory unless explicitly changed by a branch or interrupted by an exception. From the programmer's perspective, instructions execute in program order. Internally, however, the hardware may employ techniques such as out-of-order execution. This chapter presents the logical execution model, describing how branching and interruptions alter control flow.

Unconditional Branches

Unconditional branch instructions always redirect control flow and may optionally save the return address in a general-purpose register. The branch target address can be computed in two ways:

- **Instruction-relative branches:** The target is obtained by adding an offset to the address of the current instruction.
- **Region-relative branches:** The target is computed relative to the start of the region (address zero of the region).

Both methods use 32-bit unsigned arithmetic with wrap-around on overflow. For example, if the current instruction resides at `0xFFFF_FFFC` and the branch offset is `0x10`, the calculation is:

$$\texttt{0xFFFF_FFFC} + \texttt{0x10} = \texttt{0x1_0000_000C}$$

Since the addition is performed modulo 2^{32}, the high-order carry is discarded. The effective branch target is therefore `0x0000_000C`.

Conditional Branches

Conditional branches combine a comparison with an instruction-relative branch. If the specified condition is true, control transfers to the branch target; otherwise, execution continues sequentially. Conditional branches are always instruction address relative.

Linkage

Unconditional branch instructions support procedure calls by saving the return address (the instruction immediately following the branch) into a general-purpose register. The linkage mechanism applies to both intra-region and cross region branches. The saved return address consists of both the region identifier and the instruction offset.

Cross-Region Branches

Unconditional cross-region branches are branches to another segment. The offset is computed from the offset portion in the address specified plus the immediate value encoded in the `BE` instruction.

Privilege Level Changes

Branch instructions may alter the current privilege level. Execution is only permitted if the current privilege level satisfies the requirements of the code or data being accessed. Traditionally, privilege changes are handled through traps into the operating system kernel. While secure, this approach incurs significant overhead due to pipeline flushes and handler setup.

Twin-64 provides a more efficient gateway mechanism. An unconditional branch with the `.G` gateway option set may raise the privilege level. The processor executes the next instruction at the privilege level specified by the gateway page, with the PSR privilege bit updated accordingly. When returning to code in a lower-privilege page, the CPU automatically lowers the current privilege level. Each instruction fetch checks the privilege level of the code page against the PSR `P`-bit:

- If the page requires higher privilege than the PSR indicates, a privilege violation trap occurs.
- If the page requires lower privilege, execution privilege level is automatically demoted.
- If the G option on the branch instruction `B` is set and the page is a gateway page, the execution privilege is set to the privilege level of the gateway page where the branch instruction resides.

Traps Associated with Branches

Branch instructions may trigger traps depending on the PSR state. For example, if the PSR `J`-bit is set and a branch is taken, a *taken-branch trap* is raised. This feature is primarily intended for debugging and tracing purposes.

Interruptions

Twin-64 features a simple but powerful interrupt system. Interruptions in Twin-64 are divided into four classes - faults, traps, interrupts, and checks - that may occur during instruction execution or at instruction boundaries.

- **Fault** --- The current instruction cannot be carried out due to a system problem, such as the absence of a page from main memory. After the system problem has been corrected, the faulting instruction is restarted and should execute as intended. Faults are synchronous with respect to the instruction stream and are detected during instruction execution.
- **Trap** --- There are two kinds of traps. The first kind consists of traps raised because the function requested by the current instruction cannot or should not be carried out. A typical example is the arithmetic signed overflow trap. Such instructions are normally not re-executed. These traps are synchronous and occur during instruction execution.
  The second kind of traps are boundary traps, raised so that the system can intervene at an instruction boundary, either immediately after completion of an instruction or before execution of the next instruction. Examples include debugging breakpoint and trace traps. Boundary traps are synchronous with respect to the instruction stream.
- **Interrupt** --- An external entity such as an I/O module requires attention. Interrupts are asynchronous with respect to the instruction stream but are taken only at well-defined instruction boundaries.
- **Check** --- The processor has detected an internal fault or malfunction. Checks may be synchronous or asynchronous with respect to the instruction stream.

All four classes of interruptions are handled in a uniform manner.

Interrupt Handling

Handling of an interruption is implemented as a fast context switch. When an interruption occurs, the hardware saves the processor status register (PSR) at the time of the interruption into the IPSR control register. For instruction-caused interruptions, the saved instruction address corresponds to the faulting instruction. For boundary-caused interruptions, the saved instruction address corresponds to the next instruction to be executed. All status bits contained in the PSR are saved.

The hardware then initializes the PSR for handler execution. The M bit is set if the interruption is a high-priority machine check. All other status bits are cleared. Depending on the interruption type, additional information may be saved into the interrupt argument control registers IARG0 and IARG1.

Interrupt Vector Table

The control register `IVA` holds the virtual address of the interrupt vector table. Execution of the interrupt handler begins at the address computed using the trap number:

$$\texttt{IVA} + (32 * \texttt{trap number})$$

Each entry consists of 8 instructions that can be executed. It is the responsibility of interruption handlers to unmask external interrupts as soon as possible, so as to minimize interrupt latency. The interruptions are categorized into four groups. The following table lists the defined traps for Twin-64.

Table 5: Trap Table

Trap	Name	Instruction Adr	Arg0	Arg1
1	Machine Check	current instruction	check type	-
2	Power Failure	current instruction	-	-
3	Recovery Counter	current instruction	-	-
4	External Interrupt	next instruction	interrupt ID	-
5	Illegal Instruction	current instruction	instruction	-
6	Privileged Operation	current instruction	instruction	-
7	Privileged Register	current instruction	instruction	-
8	Integer Overflow	current instruction	instruction	-
9	Instruction TLB Miss	current instruction	-	-
10	Non-access Instruction TLB Miss	current instruction	-	-
11	Instruction Access Rights	current instruction	-	-
12	Instruction Memory Protection	current instruction	-	-
13	Instruction Alignment	current instruction	instruction	target address
14	Data TLB Miss	current instruction	instruction	target address
15	Non-access Data TLB Miss	current instruction	instruction	target address
16	Data Access Rights	current instruction	instruction	target address
17	Data Protection	current instruction	instruction	target address
18	Data Alignment	current instruction	instruction	target address
19	Page Reference Trap	current instruction	instruction	target address

Continued on next page

Trap	Name	IA	Arg0	Arg1
20	Breakpoint Trap	current instruction	instruction	trap code
21	Branch Taken Trap	current instruction	instruction	target address

Debug and Trace Support

The Twin-64 processor status register contains two bits for supporting debugging traps and traces. When the T bit is set, the processor executes exactly one instruction and then takes a trace exception at the following instruction boundary, before execution of the next instruction.

At an instruction boundary, multiple boundary exception conditions may be present. When multiple exception conditions are present, the processor resolves them according to the priority rules defined in Table 6. Instruction-caused exceptions always take priority over boundary exceptions. Among boundary exceptions, breakpoint traps have the highest priority.

Pri.	Exception Type	Notes
1 (High)	Instruction-caused exception	TLB miss, page fault, alignment fault, arithmetic overflow, illegal instruction
2	Debug breakpoint trap	`TRAP` instruction with breakpoint code, recognized only if B = 1
3	Trace trap	Raised when T = 1 at the instruction boundary after one instruction completes
4	External interrupt	Asynchronous event taken at an instruction boundary
5 (Low)	Check	Deferred or recoverable processor checks

Table 6: Twin-64 Exception Priority

The B bit controls recognition of breakpoint instructions. When B is cleared, `BREAK` instructions do not generate breakpoint exceptions and execute as no-ops. This allows trace exceptions to be taken deterministically while stepping over instructions, including instructions located at breakpoint addresses.

Twin-64 provides a general-purpose `TRAP` instruction which causes a synchronous trap and transfers control to the trap handler. The `TRAP` instruction includes a trap information code that identifies the reason for the trap.

One trap information code is reserved for use as a debugging breakpoint. When a `TRAP` instruction with the breakpoint trap code is encountered, it is treated as a boundary trap. Recognition of breakpoint traps is controlled by the B bit. If B is cleared, breakpoint `TRAP` instructions execute as no-ops and do not cause a trap.

All other `TRAP` instruction codes are unaffected by the B bit and always generate a trap. These traps are synchronous with respect to the instruction stream and are taken during instruction execution.

Part III

Instruction Set

Instruction Set

This part of the book presents the Twin-64 instruction set. Following the RISC design philosophy, it features a fixed-length format with a small number of regular instruction types. The total number of instructions is relatively small. However, many instructions encode execution options that make them highly versatile. Great care is taken to ensure that these options do not significantly complicate the pipeline or increase the overall data path length. Instead of supporting a wide variety of addressing modes, as commonly found in CISC-style CPUs, Twin-64 provides two virtual memory addressing modes based on a simple base–offset calculation. No indirection or addressing mode that requires reading a memory operand to compute an address is part of the architecture.

The instruction set is divided into five groups.

- **Memory reference instructions:** Load and store operations f.
- **Immediate instructions:** Building immediate values up to 64 bits.
- **Control flow instructions:** Conditional and unconditional branches.
- **Computational instructions:** Arithmetic, boolean, and bit operations.
- **System control instructions:** Managing hardware resources such as caches and TLBs, register movement, traps, and interrupt handling.

Arithmetic Instructions

The arithmetic instructions operate on two signed 64-bit operands and store the result in the target register. There are two instruction layouts. The immediate operand instruction format adds the sign-extended 15-bit value to a register. The register instruction format uses two registers as input operands, performs the operation, and stores the result in a third register.

The `ADD` and `SUB` instructions perform signed 64-bit integer arithmetic, with numeric overflow triggering an overflow trap. There are no operations on unsigned quantities. The `SHLxA` and `SHRxA` instructions perform an arithmetic left or right shift on the input operand and add another operand to the shifted result, storing the final value in the target register. The `CMP` instruction compares two registers for a condition and stores the result in the target register.

Bit Operations Instructions

The bit operation instructions fall into two groups. The `AND`, `OR`, and `XOR` instructions perform the logical operations indicated by their names. Like the arithmetic instructions, they have both an immediate instruction format and a register instruction format.

The second group implements bit manipulation operations. The `EXTR` instruction extracts a bit field from a register and places it in another register; optionally, the extracted field can be sign-extended. The `DEP` instruction deposits a bit field into a target register. The `DSR` instruction concatenates two general-purpose registers, shifts the resulting 128-bit quantity to the right, and stores the lower 64 bits in the target register.

Immediate Instructions

The `LDIL` instruction is used for forming large constants. With a 32-bit instruction word, it is not possible to encode even a full 32-bit constant in a single instruction. The `LDIL` instruction includes a qualifier to load a constant value at a defined position in the target register. Using a short sequence of instructions, any 64-bit value can be loaded.

The `ADDIL` instruction adds the upper portion of a 32-bit immediate value to the lower 32-bit half of a general register. This instruction is primarily used for address arithmetic, enabling access to any location within a region. The `LDO` instruction loads an offset into a general register. The address is formed by adding the signed immediate value to a base register. Address arithmetic is performed as unsigned 32-bit arithmetic, wrapping around within the 32-bit range.

Memory Reference Instructions

Twin-64 is a register-memory architecture. It supports the common load/store instructions as well as instructions that combine an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register plus offset mode, and the second is the base register plus register index mode.

The indexed instruction format builds an effective address by adding the scaled 13-bit immediate field to the base register `RegB`. The `dw` field specifies the data width: a `dw` value of 0 loads an 8-bit value, 1 loads a 16-bit value, 2 loads a 32-bit value, and 3 loads a 64-bit value. The `dw` field also scales the offset by left-shifting the immediate field according to the data width. For example, a `dw` value of 3 loads a double word from an address formed by shifting the 13-bit offset by three bits before adding it to the base register.

The register-indexed instruction format loads the content of the memory address formed by adding `RegX` to the base register `RegB`. The `dw` field specifies the data width, as in the indexed instruction format. The offset in `RegX` is interpreted as a byte offset, independent of the `dw` value.

The `LD` and `ST` instructions are the standard load and store instructions. The `LD` instruction has an option to load the content and zero extend it. The default is sign extension of data smaller than 64-bit. The `ST` stores the data as is.

The `LDR` and `STC` instructions provide the base support for a pipeline-friendly method for semaphore operations. They only support the indexed instruction mode with double-word access.

The ADD, SUB, AND, OR, XOR, and CMP instructions perform the respective operations as described above, with one operand fetched from memory using the indexed addressing modes.

Control Flow Instructions

Control flow is implemented through a set of branch instructions, which can be classified into unconditional and conditional branches.

Unconditional branches fetch the next instruction address from the computed branch target. The B and BR instructions perform an instruction address relative branch. The BV instruction performs a vectored branch. The content of a general register is added to a base register to form the branch target. The BE instruction is used to perform a cross-region branch. All unconditional branch instructions can optionally store the return address in a general register.

The BB, CBR, MBR, and ABR instructions implement conditional branching. If the specified condition is met, a branch to the target address is performed. The target address is always a local address, with the offset being instruction-address relative. Conditional branches adopt a static prediction model where forward branches are assumed not taken, while backward branches are assumed taken.

All branch address arithmetic is performed as 32-bit unsigned arithmetic with wraparound. Adding branch offsets will not overflow into the region Id portion of the address.

System Instructions

The final instruction group is the **special/system** instruction group. The system control instructions manage CPU resources such as the TLB, cache, and other hardware components. Most instructions in this group require the processor to run in privileged mode.

The MFCR and MTCR instructions are used to transfer data to and from a control register. Access to some control registers requires privileged mode. The MFIA instruction accesses the current instruction address, which is important for supporting position-independent code. The RSM and SSM instructions allow setting or clearing an individual accessible bit in the status portion of the processor state.

The LDPA, PRBR, and PRBW instructions are used to obtain information about the virtual memory system. The LDPA instruction returns the physical address of the virtual page, if present in memory. The PRBx instructions test whether the desired access mode to an address is allowed.

The translation lookaside buffers and caches are managed by the IxTLB, PxTLB, FxCA, and PxCA instructions. The IxTLB and PxTLB instructions insert or remove translations from the TLB. The FxCA and PxCA instructions manage the instruction and data caches, providing options to flush or purge a cache line. There is no instruction to insert data into a cache, as cache insertion is fully controlled by hardware.

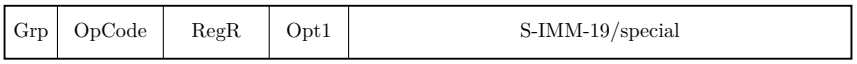
The `RFI` instruction restores the processor state from the control registers. Optionally, general registers stored in reserved control registers will also be restored.

The `DIAG` instruction issues hardware-specific implementation commands. An example is the memory barrier command, which ensures that all memory access operations complete before subsequent instructions execute.

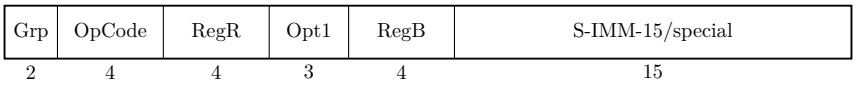
The `TRAP` instruction raises a software exception and is typically used to implement a debugging breakpoint.

Instruction Formats

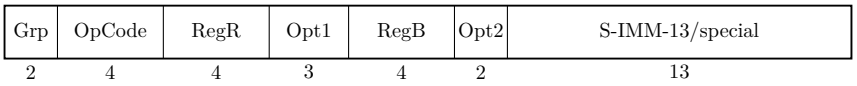
Twin-64 employs a small number of main instruction formats, which are classified according to the number of register operands and the length of the immediate value field. The **signed immediate S19 / special (S19)** format provides one register operand and a 19-bit signed immediate field. This format is typically used for branch instructions.



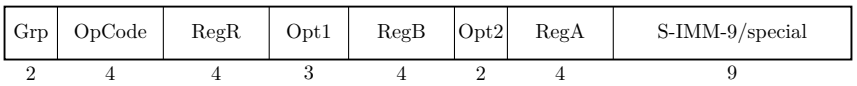
The **signed immediate S15 / special (S15)** instruction format provides two register operands and a 15-bit signed immediate field. This format is used both by conditional branch instructions and by instructions that operate on a register and an immediate value.



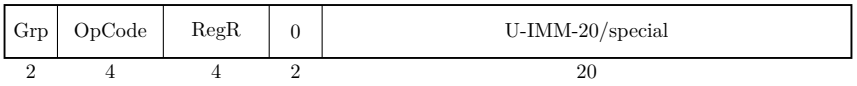
The **signed immediate S13 / special (S13)** instruction format provides two registers, an additional option field, and a 13-bit signed immediate field. Some instructions use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions, where the address is formed by a base register plus a 13-bit signed offset.



The **register / signed immediate S9 / special (S9)** instruction format provides three registers, an additional option field, and a 9-bit signed immediate field. Some instructions use the S-IMM-9 field for special encodings instead of a signed value. This instruction format is used for all computational instructions that require three registers.



The **unsigned immediate U20 (U20)** format is used by the immediate instruction group to provide a 20-bit unsigned value. This value is then placed at specific positions in the target register `Reg R`.



Synthetic Instructions

The Twin-64 instruction set provides a rich set of options for individual instructions. By setting defined option bits, an instruction can enable additional functionality with minimal impact on the overall data path. Likewise, instructions that use general register zero as an operand can take advantage of predefined behaviors.

To improve readability and convenience, an assembler may offer synthetic instructions. These are higher-level mnemonics derived from the base instruction set that simplify programming by providing more natural or concise representations of commonly used instruction patterns.

Synthetic instructions are also useful when a single machine instruction cannot fully express the desired operation. For example, loading an immediate value that exceeds the size of the instruction's immediate field requires an instruction sequence. An assembler can therefore provide a generic "load immediate" synthetic instruction that automatically emits the appropriate sequence based on the value's size. Similarly, if the offset from a base register is too small to reach a target data item, the assembler can transparently emit an additional ADDIL instruction to compute the correct address.

Where applicable, the instruction descriptions include examples and implementation guidance for synthetic instructions. A complete list of supported synthetic instructions can be found in the Twin-64 Assembler document.

Instruction Set Descriptions

This chapter provides a detailed description of the Twin-64 instruction set. Each instruction entry includes:

- The instruction word layout.
- A concise description of the instruction.
- A high-level pseudo-code representation of its operation.

Instruction Operations

Instruction operations are expressed using C-style pseudo code. The conventions and helper routines used in these descriptions are defined in the appendix. Registers are referenced by class and index:

- `GR[5]` — general register 5
- `CR[1]` — control register 1

Some registers have dedicated names; for example, the shift-amount control register is `SAR`.

Instruction Options

Instruction options are specified in assembler syntax by appending a dot to the instruction mnemonic, followed by one or more option characters. Each character represents a defined option. For example:

- `AND.N ...` — the `AND` instruction with the negate-result option.
- `LD.B ...` - the textttLD instruction with loading a byte option.

The order of option characters is insignificant. All valid options for an instruction are listed in the corresponding option set.

Instruction Subfields

Individual subfields of an instruction or register are accessed using a dot followed by the field name enclosed in square brackets. For example:

- The interrupt-enable bit in the program state register: `PSR.[I]`.

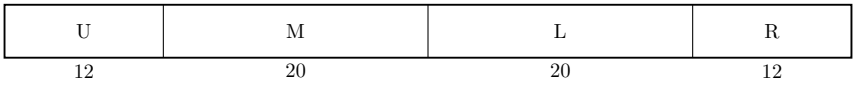
Fields marked as *reserved* are intended for future extensions and must always be set to zero. Any non-zero value in a reserved field results in undefined behavior.

Constant Expressions

Constant expressions are 64-bit values. When encoding a numeric field that cannot be represented directly, a sequence of instructions may be required to construct the value in the destination word. To support this, constant values may be preceded by a field qualifier that selects a portion of the 64-bit value.

The available field qualifiers are `U%`, `M%`, `L%`, and `R%`. Each qualifier, when applied to a 64-bit numeric value, extracts the corresponding field and right-shifts it to form the resulting expression.

The following figure illustrates the individual fields selected by each qualifier:



Examples of constant expression usage can be found in the descriptions of the `ADDIL`, `LDIL`, and `MFIA` instructions

Program Instructions

The general-purpose instructions form the core of the Twin64 instruction set. They are the instructions normally used by application and user-level programs to perform their work. They provide operations for moving data, performing arithmetic and logical calculations, comparing values, and controlling program flow.

These instructions operate primarily on the registers, memory, and execution state available to a program. Together, they provide the fundamental operations from which ordinary Twin64 programs are constructed.

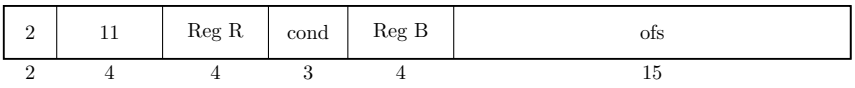
The instructions in this chapter represent the part of the instruction set that a programmer will use most frequently when writing software for the Twin64 architecture.

The division between general-purpose and system instructions is based primarily on their intended use. Consequently, a small number of instructions described in the System Instructions chapter may also be relevant to ordinary programs. For example, the shift amount register is a processor control register, but it is directly used by programs that perform variable shift operations. Such cases are kept with the system instructions because they concern processor-specific control registers, even though the associated operations can be useful in normal program code.

ABR - Add and branch

Syntax `ABR.<cond> RegR, RegB, target`

Format The ABR instruction uses the following instruction format:



Description The `ABR` instruction adds `RegB` to `RegR` and stores the result in `RegR`. If the condition specified in the `cond` field is satisfied, execution continues at the current instruction plus the offset; otherwise, execution proceeds with the next instruction.

Conditions The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == 0`
1	LT	`RegR < 0`
2	GT	`RegR > 0`
3	EV	`RegR.[0] == 0`
4	NE	`RegR != 0`
5	GE	`RegR >= 0`
6	LE	`RegR <= 0`
7	OD	`RegR.[0] != 0`

Operation

```
RegR <- RegR + RegB;

if ( cond( RegR )) {

    tmpOfs <- signExtend( ofs ) << 2;
    PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], tmpOfs );
}
else PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], 4 );
```

Exceptions Integer Overflow trap. Taken branch trap.

Notes See also the instruction notes on comparison and branch condition encoding in the appendix.

ADD - Integer Addition

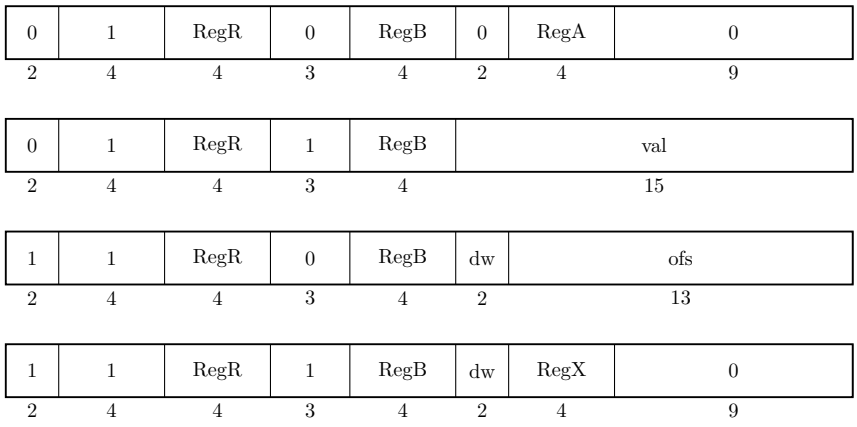
Syntax

```
ADD RegR, RegB, RegA
ADD RegR, RegB, val

ADD[.D/W/H/B] RegR, ofs ( RegB )
ADD[.D/W/H/B] RegR, RegX ( RegB )
```

Formats

The ADD instruction uses the following instruction formats:



Description

The `ADD` instruction adds two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-reference related trap.

Operation

```
RegR <- RegB + RegA;                              (S9 )
RegR <- RegB + immOperand( instr );               (S15)
RegR <- RegR + memOperandImmIndexed( instr );     (S13)
RegR <- RegR + memOperandRegIndexed( instr );     (S9 )
```

Exceptions

Integer Overflow Trap
Data Memory Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

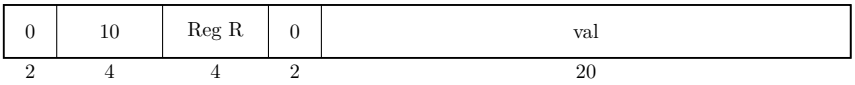
Notes

None.

ADDIL - Add immediate left

Syntax `ADDIL RegR, val`

Format The ADDIL instruction uses the uses the following instruction formats.



Description The `ADDIL` instruction adds an immediate value to a general register using 32-bit unsigned arithmetic. The 20-bit immediate value is shifted left by 12 bits (padded with zeros on the right) before being added to `RegR`. The resulting value is stored in the general register `GR1`. Any overflow that occurs during the addition is ignored.

Operation

```
R1 <- addOfs32( RegR, ( val << 12 ));
```

Exceptions None.

Notes None.

AND - Boolean Operation

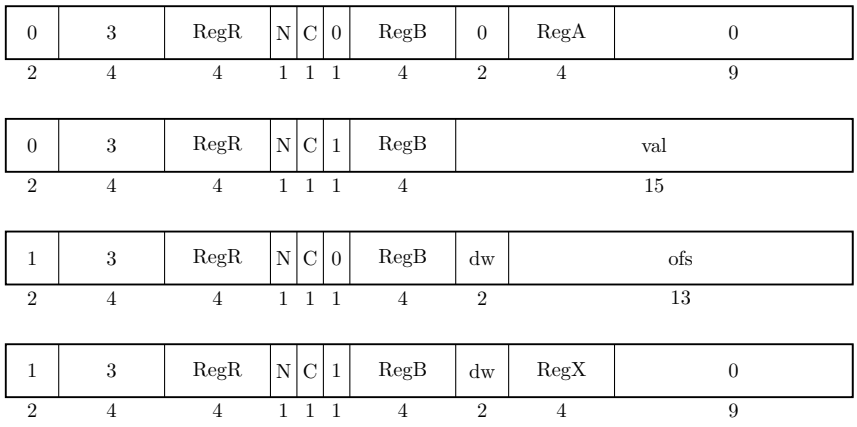
Syntax

```
AND[.C/N] RegR, RegB, RegA
AND[.C/N] RegR, RegB, val

AND[.C/N/D/W/H/B] RegR, ofs( RegB )
AND[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The AND instruction uses uses the following instruction formats.



Description

The `AND` instruction performs a bitwise AND operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegA` or the loaded content, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference related trap.

Operation

```
tmp <- RegA;                              (S9)
tmp <- immOperand( instr );               (S15)
tmp <- memOperandImmIndexed( instr );     (S13)
tmp <- memOperandRegIndexed( instr );     (S9 )

if ( instr.[C] ) tmp <- not( tmp );
RegR <- RegB & tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Memory Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

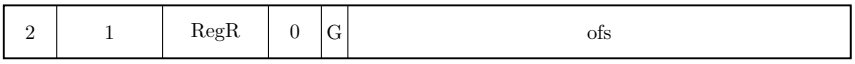
Notes

None.

B - Branch Unconditional

Syntax `B[.G] target [, RegR ]`

Format The `B` instruction uses the following instruction format:



Description The `B` instruction performs an unconditional instruction address relative branch. The target address is computed by sign-extending the immediate offset, shifting it left by two bits, and adding the result to the current instruction address. The return link is stored in `RegR`.

The `.G` flag enables the *gateway branch* option. In this case, the processor's privilege level may be raised to the privilege level stored in the TLB entry of the page from which the branch with gateway option instruction is fetched. The change is only performed if it results in a higher privilege level; otherwise, the current privilege level remains unchanged. Execution then continues at the computed target address with the (possibly new) privilege level.

Operation

```
if ( instr.[RegR] != 0 ) {

        RegR.[51:32] <- PSR.[IA-REG];
        RegR.[31:0]  <- addOfs32( PSR.[IA-OFS], 4 );
}

if ( instr.[G] ) {

    if ( TLB[PSW.[IA-OFS]].[X] > PSW.[X] )
        PSW.[X] <- TLB[PSW.[IA]].[X];
}

tmpOfs <- signExtend( ofs ) << 2;
PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], tmpOfs );
```

Exceptions Instruction Privilege Violation Trap
Instruction TLB Miss Trap
Taken Branch Trap

Notes None.

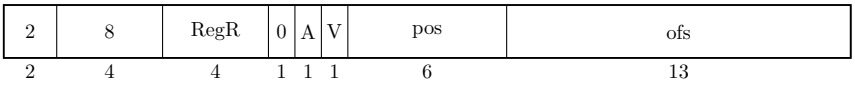
BB - Branch on Bit

Syntax

```
BB .T/F RegR, pos, ofs
BB .T/F RegR, SAR, ofs
```

Format

The `BB` instruction uses the following instruction format:



Description

The `BB` instruction tests a bit in register `RegR`. The bit position can be specified directly by the `pos` field, or indirectly through the Shift Amount Register (`SAR`) if the `A` option is set.

The `V` determines whether to test the bit for true or false. If the `V` bit is set, the branch is taken when the selected bit is `1`, otherwise the branch is taken when the selected bit is `0`.

When the condition is satisfied, execution resumes at the target address, computed by sign-extending the offset, shifting it left by two bits, and adding it to the current instruction address. If the condition is not satisfied, execution continues with the next sequential instruction.

Operation

```
if ( instr.[A] )  pos <- SAR;
else              pos <- instr.[pos];

bVal <- testBit( RegR, pos );
cond <- ( instr.[V] ? ( bVal == 1 ) : ( bVal == 0 ));

if ( cond ) {

    tmpOfs <- signExtend( ofs ) << 2;
    PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], tmpOfs );
}
else PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], 4 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

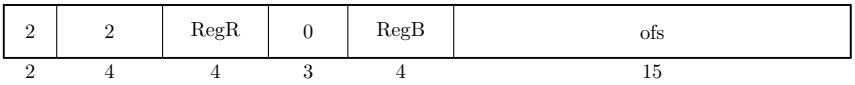
Notes

None.

BE - Branch External

Syntax `BE [ ofs ] ( RegB ) [, RegR ]`

Format The `BE` instruction uses the following instruction format:



Description The `BE` instruction performs an unconditional branch to another region. `RegB` contains a target region and offset base to which the sign extended offset shifted by two is added. The return link is stored in `RegR`.

Because `BE` is a region base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register accordingly. If no change is required, the privilege level remains unchanged.

Operation

```
if ( instr.[RegR] != 0 ) {

    RegR.[51:32] <- PSR.[IA-REG];
    RegR.[31:0]  <- addOfs32( PSR.[IA-OFS], 4 );
}

PSR.[IA-REG] <- RegB.[51:32];
PSR.[IA-OFS] <- addOfs32( RegB, RegX << 2 );
```

Exceptions Taken Branch Trap
Instruction TLB Miss Trap

Notes None.

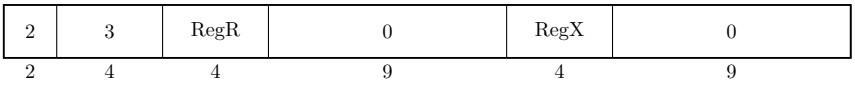
BR - Branch Register

Syntax

```
BR RegX [, RegR ]
```

Format

The `BR` instruction uses the following instruction format:



2	3	RegR	0	RegX	0
2	4	4	9	4	9

Description

The `BR` instruction performs an unconditional branch to a target address computed relative to the current instruction address (`IA`). The target address is formed by taking the contents of `RegX`, left shifting the value by two, and adding the result to the instruction address offset portion.

Optionally, a return link may be written to `RegR`. When specified, `RegR` receives the address of the next sequential instruction, allowing the `BR` instruction to be used for procedure calls or control-flow transfers that require a return address.

Operation

```
if ( instr.[RegR] != 0 ) {

    RegR.[51:32] <- PSR.[IA-REG];
    RegR.[31:0]  <- addOfs32( PSR.[IA-OFS], 4 );
}

PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], RegX << 2 ));
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

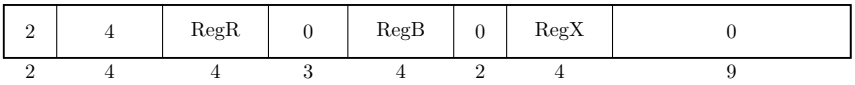
Notes

None

BV - Branch Vectored

Syntax `BV [ RegX ] ( RegB ) [ , RegR ]`

Format The `BV` instruction uses the following instruction format:



Description he `BV` instruction performs an unconditional region relative branch. The target address offset is formed by taking the contents of `RegX`, left shifting by two, and adding the result to `RegB`. The target address region is the region portion of the current instruction address.

Optionally, a return link may be written to `RegR`. When specified, `RegR` receives the address of the next sequential instruction, allowing the `BV` instruction to be used for procedure calls or control-flow transfers that require a return address.

Because `BV` is a region base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register accordingly. If no change is required, the privilege level remains unchanged.

Operation

```
if ( instr.[RegR] != 0 ) {

    RegR.[51:32] <- PSR.[IA-REG];
    RegR.[31:0]  <- addOfs32( PSR.[IA-OFS], 4 );
}

PSR.[IA-OFS] <- addOfs32( RegB, RegX << 2 );
```

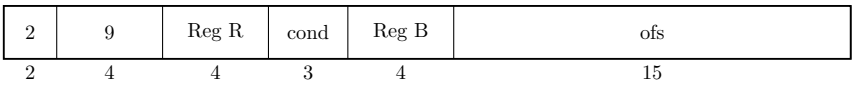
Exceptions Taken Branch Trap
Instruction TLB Miss Trap

Notes None.

CBR - Compare and Branch

Syntax `CBR.<cond> RegR, RegB, target`

Format The CBR instruction uses the following instruction format:



Description The `CBR` instruction compares the contents of `RegB` with `RegR` using the condition specified in the `cond` field. If the condition is satisfied, control is transferred to the instruction address computed by adding the sign-extended and word-aligned offset to the current instruction address. Otherwise, execution continues with the next sequential instruction. The comparison operation uses the same signed 64-bit comparator as the `CMP` instruction, and neither source register is modified.

Conditions The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	RegR == RegB
1	LT	RegR < RegB
2	GT	RegR > RegB
3	undefined	
4	NE	RegR != RegB
5	GE	RegR >= RegB
6	LE	RegR <= RegB
7	undefined	

Operation

```
if compare( cond, RegR, RegB ) {

    tmpOfs <- signExtend( ofs ) << 2;
    PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], tmpOfs );
}
else PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], 4 );
```

Exceptions Taken Branch Tap.
Instruction TLB Miss.

Notes See also the instruction notes on comparison and branch condition encoding in the appendix.

CMP - Compare

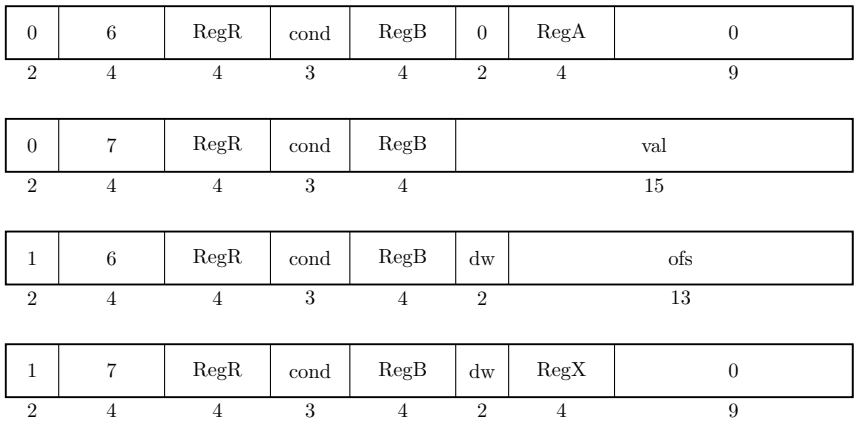
Syntax

```
CMP RegR, RegB, RegA
CMP RegR, RegB, val

CMP .cond [.D/W/H/B] RegR, ofs ( RegB )
CMP .cond [.D/W/H/B] RegR, RegX ( RegB )
```

Format

The CMP instruction uses the following instruction formats.



Description

The `CMP` instruction compares two signed 64-bit operands. In register mode, it compares `RegB` and `RegA`; in immediate mode, it compares `RegB` and the immediate value; and in memory-reference formats, it compares `RegR` with the memory operand. The operands are called `op1` and `op2`. The result of the comparison is stored in RegR. The instruction does not cause an arithmetic overflow but indexed instruction formats may cause a memory-reference–related trap.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	RegR == RegB
1	LT	RegR < RegB
2	GT	RegR > RegB
3	undefined	
4	NE	RegR != RegB
5	GE	RegR >= RegB
6	LE	RegR <= RegB
7	undefined	

Operation

```
RegR <- compare( cond, RegB, RegA );                    (S9 )
RegR <- compare( cond, RegB, immOperand());             (S15)
RegR <- compare( cond, RegR, memOperandImmIndexed()); (S13)
RegR <- compare( cond, RegR, memOperandRegIndexed()); (S9 )
```

Exceptions

Data Memory Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

See also the instruction notes on comparison and branch condition encoding in the appendix.

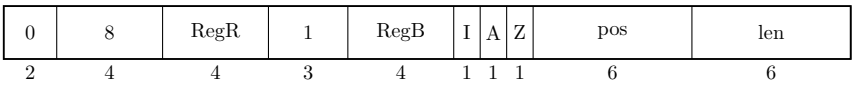
DEP - Deposit Bit Field

Syntax

```
DEP[.Z] RegR, RegB, pos, len
DEP[.Z] RegR, val, pos, len
DEP[.Z] RegR, RegB, SAR, len
```

Format

The `DEP` instruction uses the special-13 instruction format.



0	8	RegR	1	RegB	I	A	Z	pos	len
2	4	4	3	4	1	1	1	6	6

Description

The `DEP` instruction deposits a bit field in the length of len from general register `RegB`. The source bit field is right justified in `RegB`.

The rightmost bit of the target field is specified by the `pos` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set. The length of the field is specified by the `len` instruction field.

If the `Z` bit is set, the target register `RegR` is cleared before depositing the field.

If the `I` bit is set, the value to deposit is taken from the RegB field rather than the value in register `RegB`.

Operation

```
if ( instr.[Z] ) RegR <- 0;

if ( instr.[A] ) pos <- SAR;
else             pos <- instr.[pos];

if ( instr.[I] )
    RegR <- deposit( RegR, instr.[RegB], pos, instr.[len] );
else
    RegR <- deposit( RegR, RegB, pos, instr.[len] );
```

Exceptions

None

Notes

None

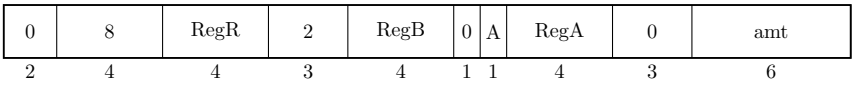
DSR - Double Shift Right

Syntax

```
DSR RegR, RegB, RegA, amt
DSR RegR, RegB, RegA, SAR
```

Format

The `DSR` instruction uses the following instruction format:



Description

The `DSR` instruction concatenates the contents of registers `RegB` (left) and `RegA` (right) and performs a right shift operation. The lower 64 bits of the shifted result are stored in `RegR`.

The shift amount is specified by the `amt` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set.

Operation

```
if ( instr.[A] ) shAmt <- SAR;
else             shAmt <- instr.[amt];

RegR <- doubleShiftR( RegB, RegA, shAmt );
```

Exceptions

None

Notes

Twin-64 does not have instruction for shifting and rotating a register value. Instead the `EXTR` and `DSR` instructions are used. A shift left operation can easily be encoded as

```
ROL Rm, Rn, amt -> DSR Rm, Rn, R0, amt, 64 - amt
```

The instruction rotates `Rn` by the amount and stores the result in `Rm`. Likewise, a rotate right operation can be encoded as:

```
ROR Rm, Rn, amt -> DSR Rm, Rn, Rn, amt
```

and a rotate left operation:

```
ROL Rm, Rn, amt -> DSR Rm, Rn, Rn, 64 - amt
```

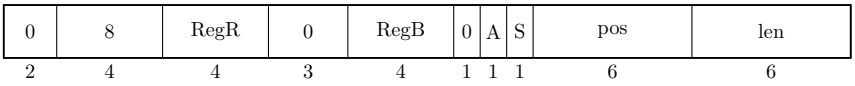
EXTR - Extract Bit Field

Syntax

```
EXTR[.S] RegR, RegB, pos, len
EXTR[.S] RegR, RegB, SAR, len
```

Format

The `EXTR` instruction uses the following instruction format:



0	8	RegR	0	RegB	0	A	S	pos	len
2	4	4	3	4	1	1	1	6	6

Description

The `EXTR` instruction extracts a bit field from general register `RegB`.

The rightmost bit of the field is specified by the `pos` instruction field. If the `A` bit is set, the position value is taken from the Shift Amount Register (`SAR`) instead of the instruction field.

The length of the field is specified by the `len` instruction field. The extracted bit field is stored right-justified in `RegR`. If the `S` bit is set, the extracted value is sign-extended.

Operation

```
if ( instr.[A] ) tmpPos <- SAR;
else             tmpPos <- instr.[pos];

RegR <- extractField( RegB, tmpPos, instr.[len] );

if ( instr.[S] ) RegR <- signExtend( RegR, instr.[len] );
```

Exceptions

None

Notes

Twin-64 does not have instruction for shifting and rotating a register value. Instead the `EXTR` and `DSR` instructions are used.

An arithmetic shift right operation can easily be encoded as

```
ASR Rm, Rn, amt -> EXTR.S Rm, Rn, R0, amt, 64 - amt
```

and an arithmetic shift left left operation:

```
ASL Rm, Rn, amt -> EXTR.S Rn, Rm, amt, 64 - amt
```

LD - Load Register

Syntax

```
LD[.D/W/H/B/U] RegR, ofs( RegB )
LD[.D/W/H/B/U] RegR, RegX( RegB )
```

Format

The `LD` instruction uses the following instruction formats:



Description

The `LD` instruction loads a data value from memory into a general-purpose register. Two formats are available: the immediate-offset format and the register-indexed format. In the immediate-offset format, the signed immediate offset is shifted left by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address.

If the data loaded is a byte, half-word or a word, the data is sign-extended. If the `U` option is set, the data is zero-extended instead.

Operation

```
RegR <- memOperandImmIndexed( instr ); (S13)
RegR <- memOperandRegIndexed( instr ); (S9)
```

Exceptions

- Data Memory Alignment Trap
- Data Memory Access Violation Trap
- Data Memory Protection Violation Trap
- Data TLB Miss Trap

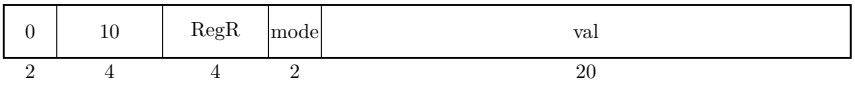
Notes

None.

LDIL - Load Immediate Left

Syntax `LDIL[.L/.M/.U] RegR, val`

Format The `LDIL` instruction uses the following instruction format:



Description The `LDIL` instruction deposits an immediate value into general register `RegR`. The optional suffix determines how the immediate value is positioned within the destination register. If no suffix is specified, the default is `.L` mode (mode value = 1).

- `.L` (mode = 1) — Deposits 20 bits of `val` into bits [31..12] of `RegR`. RegR is cleared before the operation.
- `.M` (mode = 2) — Deposits 20 bits of `val` into bits [51..32] of `RegR`. RegR is not cleared before the operation.
- `.U` (mode = 3) — Deposits 12 bits of `val` into bits [63..52] of `RegR`. RegR is not cleared before the operation.

A mode value of zero is reserved for the `ADDIL` instruction and must not be used here.

Operation

```
RegR <- ((val & 0xFFFFF) << 12 );                // .L mode (1)
RegR <- deposit( RegR, (val & 0xFFFFF), 32, 20 ); // .M mode (2)
RegR <- deposit( RegR, (val & 0xFFF), 52, 12 );   // .U mode (3)
```

Exceptions None.

Notes

The `LDIL` instruction is used to construct large values in a register. The `.L` variant clears the destination register before depositing the immediate value. It is the default option for the `LDIL` instruction. This form is intended for building larger offsets for indexed addressing modes, where an addressing instruction adds its own 12-bit offset in the low bits. Together, these operations allow the construction of any 32-bit offset.

In contrast, the `.M` and `.U` variants deposit their immediate values without clearing the register, preserving the remaining bits. Using a combination of these modes, a full 64-bit value can be assembled in two to four instructions.

For example constructing a 52-bit value, the instruction sequence

```
LDIL   R1, L%0xA_BCDE_1234_5678  ; load        0x1234_5000 into R1
LDO    R1, R%0xA_BCDE_1234_5678  ; load offset 0x678       into R1
LDIL.M R1, M%0xA_BCDE_1234_5678  ; deposit     0xA_BCDE    into R1
```

loads the constant `0xA_BCDE_1234_5678` into R1.

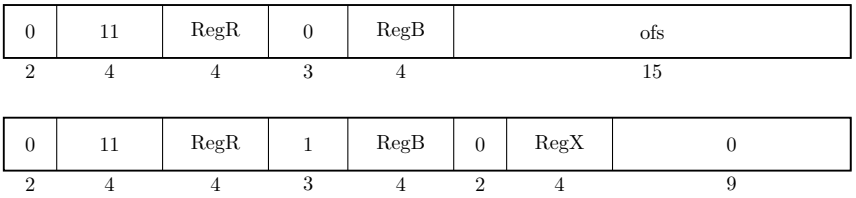
LDO - Load Offset

Syntax

```
LDO RegR, ofs(RegB)
LDO RegR, RegX(RegB)
```

Format

The `LDO` instruction uses the following instruction formats:



Description

The `LDO` instruction loads an offset into a general register. Two formats are available: the immediate-offset format and the register-indexed format. In the immediate-offset format, the signed immediate offset is added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address. The computed value is then stored in `RegR`.

The addition of base register and offset will wrap around and leave the upper bits of the base register unaffected.

Operation

```
RegR <- addOfs32( RegB, signExtend( ofs, 13 ));
RegR <- addOfs32( RegB, ( RegX ));
```

Exceptions

None.

Notes

The `LDO` instruction is often used to load immediate values.

```
LDI Rm, val -> LDO Rm, val( R0 )
```

If value to load is larger than the 13-bit offset, a sequence of instruction is however necessary.

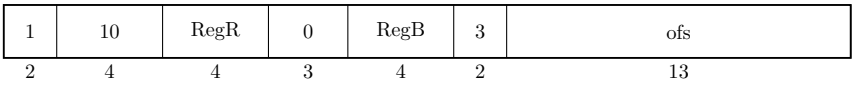
LDR - Load Register Reserved

Syntax

```
LDR RegR, ofs(RegB)
```

Format

The `LDR` instruction uses the following instruction format:



1	10	RegR	0	RegB	3	ofs
2	4	4	3	4	2	13

Description

The `LDR` instruction loads a value from memory into a general-purpose register and marks the effective address as "reserved" for a subsequent conditional store. The instruction is only the first step of an atomic read-modify-write pair. The subsequent `STC` (store conditional) instruction will succeed only if the reservation is still valid.

The signed immediate offset is shifted right by 3 and added to `RegB` to form the effective address.

Operation

```
RegR <- memLoad( memOperandImmIndexed( instr ));
markAddressReserved( memOperandImmIndexed( instr ));
```

Exceptions

- Data Memory Access Violation Trap
- Data Memory Protection Violation Trap
- Data TLB Miss Trap

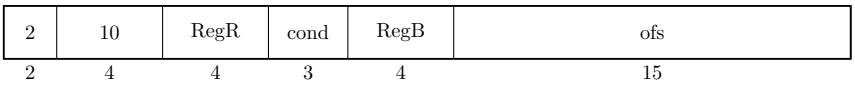
Notes

This instruction is part of a "load-reserved / store-conditional" pair for atomic memory updates and often used for implementing locks or atomic counters. Reservation may be cleared by other stores to the same address by another processor. See the appendix for more information.

MBR - Move and Branch

Syntax `MBR.<cond> RegR, RegB, target`

Format The MBR instruction uses the following instruction format:



Description The `MBR` instruction copies the contents of `RegB` into `RegR`. If the condition specified in the `cond` field evaluates true for the new value in `RegR`, execution continues at the current instruction address plus the sign-extended and shifted left by 2 bits offset. Otherwise, execution resumes with the next instruction.

Conditions The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	RegR == 0
1	LT	RegR < 0
2	GT	RegR > 0
3	EV	RegR.[0] == 0
4	NE	RegR != 0
5	GE	RegR >= 0
6	LE	RegR <= 0
7	OD	RegR.[0] != 0

Operation

```
RegR <- RegB;

if cond( RegR ) {

    tmpOfs <- signExtend( ofs ) << 2;
    PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], tmpOfs );
}
else PSR.[IA-OFS] <- addOfs32( PSR.[IA-OFS], 4 );
```

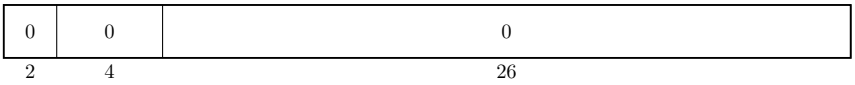
Exceptions None.

Notes See also the instruction notes on comparison and branch condition encoding in the appendix.

NOP - No Operation

Syntax `NOP`

Format The `NOP` instruction uses the following format:



Description The `NOP` (No Operation) instruction does nothing. It is typically used for timing, alignment, or to occupy a pipeline slot.

Operation

```
noOperation( );
```

Exceptions None.

Notes None.

OR - Boolean Operation

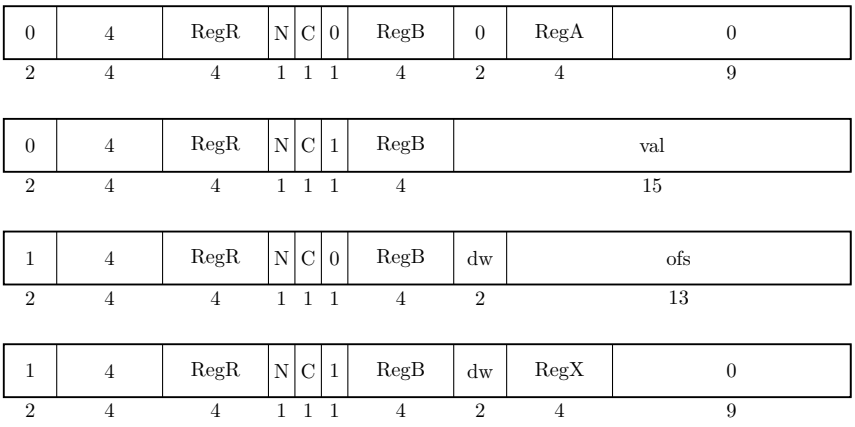
Syntax

```
OR[.C/N] RegR, RegB, RegA
OR[.C/N] RegR, RegB, val

OR[.C/N/D/W/H/B] RegR, ofs( RegB )
OR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The OR instruction uses the following instruction formats:



Description

The `OR` instruction performs a bitwise OR operation on two 64-bit operands and stores the result in the target register `RegR`. The `OR` instruction performs a bitwise OR operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegA` or the loaded content, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference related trap.

Operation

```
tmp <- RegA;                              (S9)
tmp <- immOperand( instr );               (S15)
tmp <- memOperandImmIndexed( instr );     (S13)
tmp <- memOpwerandRegIndexed( instr );    (S9 )

if ( instr.[C] ) tmp <- not( tmp );
RegR <- RegB | tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Data Memory Access Violation Trap
Data Memory Protection Violation Trap
Data TLB Miss Trap

Notes None.

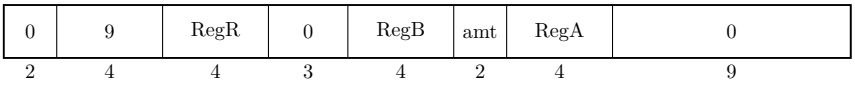
SHLxA - Shift Left and Add

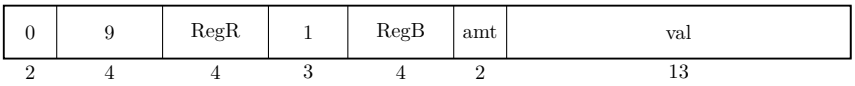
Syntax

```
SHLxA RegR, RegB, RegA
SHLxA RegR, RegB, val
```

Format

The `SHLxA` instruction uses the following instruction formats:





Description

The `SHLxA` instruction shifts the contents of `RegB` left by the amount specified in the `amt` field. In register format, `RegA` is added to the shifted value. In immediate format the sign-extended `val` field is added instead. The resulting value is stored in `RegR`.

Operation

```
RegR <- ( RegB << instr.[amt] ) + RegA;               (R)
RegR <- ( RegB << instr.[amt] ) + signExtend( val );  (I)
```

Exceptions

Integer Overflow Trap

Notes

None

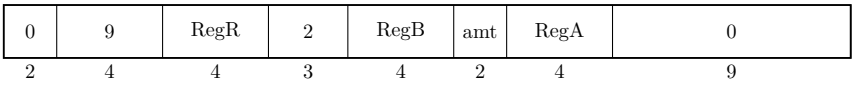
SHRxA - Shift Right and Add

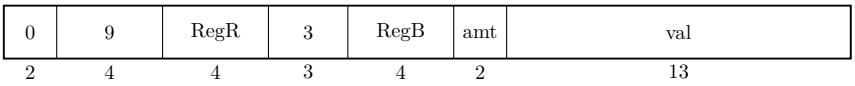
Syntax

```
SHRxA RegR, RegB, RegA
SHRxA RegR, RegB, val
```

Format

The `SHRxA` instruction uses the following instruction formats:





Description

The `SHRxA` instruction shifts the contents of `RegB` right by the amount specified in the `amt` field. In register format, `RegA` is added to the shifted value. In immediate format the sign-extended `val` field is added instead. The resulting value is stored in `RegR`.

Operation

```
RegR <- ( RegB >> instr.[amt] ) + RegA;               (R)
RegR <- ( RegB >> instr.[amt] ) + signExtend( val );  (I)
```

Exceptions

Integer Overflow Trap

Notes

None

ST - Store Register

Syntax

```
ST[.D/W/H/B] RegR, ofs( RegB )
ST[.D/W/H/B] RegR, RegX( RegB )
```

Format

The `ST` instruction uses the indexed and register-indexed instruction formats.



1	9	RegR	0	RegB	dw	ofs
2	4	4	3	4	2	13

1	9	RegR	1	RegB	dw	RegX	0
2	4	4	3	4	2	4	9

Description

The `ST` instruction stores a value from a general-purpose register into memory.

In the immediate-offset format, the signed immediate offset is shifted left by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address.

The store address must be aligned to the data width specified. The value stored to memory is stored as is in the data width specified.

Operation

```
memStoreImmIndexed( instr, RegR );
memStoreRegIndexed( instr, RegR );
```

Exceptions

- Data Memory Alignment Trap
- Data Memory Access Violation Trap
- Data Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

None.

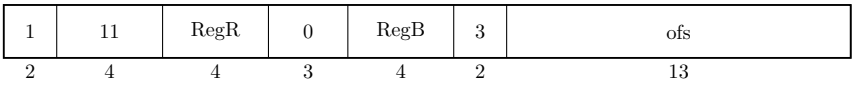
STC - Store Register Conditional

Syntax

```
STC RegR, ofs(RegB)
```

Format

The `STC` instruction uses the following instruction format:



1	11	RegR	0	RegB	3	ofs
2	4	4	3	4	2	13

Description

The `STC` instruction stores a value from a general-purpose register to memory only if the memory address is still reserved by a prior `LDR` instruction. If the reservation is valid, the store succeeds and clears the reservation. If the reservation has been lost (e.g., another processor wrote to the address), the store fails and the memory is unchanged.

offset must be 8-byte aligned.

Operation

```
if ( isAddressReserved( memOperandImmIndexed( instr ))) {

    memStore( memOperandImmIndexed( instr ), RegR );
    clearReservation( memOperandImmIndexed( instr ));
    RegR <- 1;
}
else RegR <- 0;
```

Exceptions

- Data Memory Alignment Trap
- Data Memory Access Violation Trap
- Data Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

This instruction is the second part of a "load-reserved / store-conditional" pair for atomic memory updates and commonly used for implementing spin-locks, mutexes, and atomic counters in multiprocessor systems.

SUB - Integer Subtraction

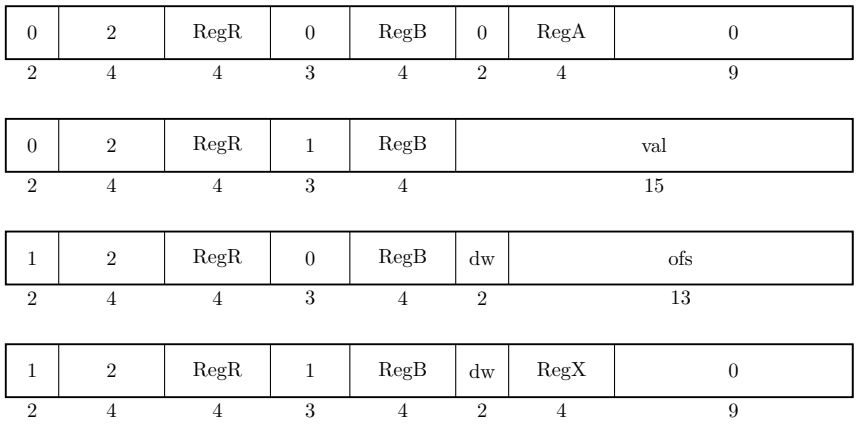
Syntax

```
SUB RegR, RegB, RegA
SUB RegR, RegB, val

SUB[.D/W/H/B] RegR, ofs ( RegB )
SUB[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The SUB instruction uses the following instruction formats:



Description

The `SUB` instruction subtracts two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-reference–related trap.

Operation

```
RegR <- RegB - RegA;                              (S9 )
RegR <- RegB - immOperand( instr );               (S15)
RegR <- RegR - memOperandImmIndexed( instr );     (S13)
RegR <- RegR - memOpwerandRegIndexed( instr );    (S9 )
```

Exceptions

Integer Overflow Trap
Data Memory Alignment Trap
Data Memory Access Violation Trap
Data Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

XOR - Boolean Operation

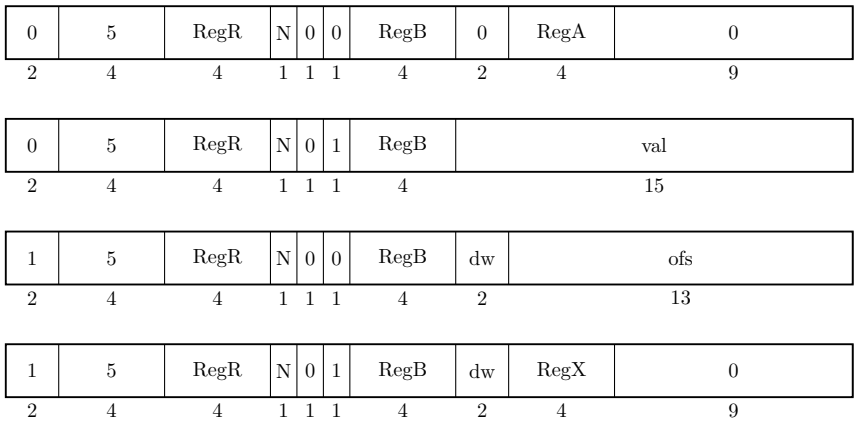
Syntax

```
XOR[.N] RegR, RegB, RegA
XOR[.N] RegR, RegB, val

XOR[.N/D/W/H/B] RegR, ofs( RegB )
XOR[.N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The XOR instruction uses the register, the following instruction formats.



Description

The `XOR` instruction performs a bitwise XOR operation on two 64-bit operands and stores the result in the target register `RegR`. The result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference–related trap.

Operation

```
tmp <- RegA;                              (S9)
tmp <- immOperand( instr );               (S15)
tmp <- memOperandImmIndexed( instr );     (S13)
tmp <- memOpwerandRegIndexed( instr );    (S9 )

RegR <- RegR ^ tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Memory Alignment Trap
Data Memory Access Violation Trap
Data Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

System and Supervisory Instructions

The system instructions are part of the Twin64 processor architecture but are intended primarily for system-level software. They provide operations that interact directly with processor control and execution state and support functions that are normally outside the scope of an application program.

A typical user-level programmer will have little need to use these instructions directly. They become relevant when writing operating-system code, low-level runtime software, firmware, or other software that needs to interact closely with the processor itself.

The system instructions are presented separately from the general-purpose instructions to make this distinction clear. They are nevertheless part of the complete Twin64 instruction set and are described here for completeness and for programmers working at the system level.

Some system instructions have a direct role in normal program execution. In particular, certain processor control registers provide facilities that are useful to application programmers. Their presence in this chapter reflects their system-level nature rather than a restriction on their use.

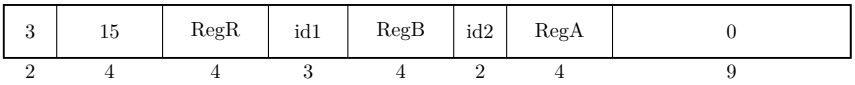
DIAG - Diagnose Instruction

Syntax

```
DIAG id, RegB, RegA [ , RegR ]
```

Format

The `DIAG` instruction uses the following instruction format:



Description

The `DIAG` instruction provides a mechanism for diagnostic and system level operations.

The concatenated `id1` and `id2` fields specify the diagnostic function. The `RegB` and `RegA` fields may provide operands or data required by the diagnostic function. An optional result is returned in `RegR`.

Operation

```
RegR <- diagOperation( concat( id1, id2 ), RegB, RegA );
```

Exceptions

Exceptions depend on diagnostic function and operands.

Notes

See the appendix for the available diagnostic operation codes.

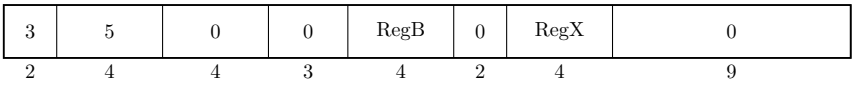
FxCA - flush from Instruction or Data Cache

Syntax

```
FICA [ RegX ] ( RegB )
FDCA [ RegX ] ( RegB )
```

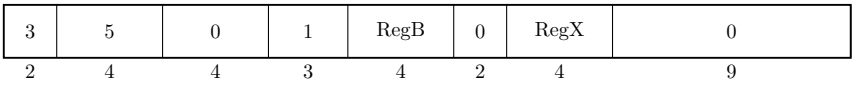
Format

The `FICA` instruction uses the following instruction format.



3	5	0	0	RegB	0	RegX	0
2	4	4	3	4	2	4	9

The `FDCA` instruction uses the following instruction format.



3	5	0	1	RegB	0	RegX	0
2	4	4	3	4	2	4	9

Description

The `FICA` instruction invalidates a cache line, selected by the effective address formed by adding `RegX` to `RegB`.

The `FDCA` instruction flushes and invalidates a cache line, selected by the effective address formed by adding `RegX` to `RegB`. The cache line addressed is written back to memory if and only if it is dirty, and then invalidated.

Both instructions are privileged instructions.

Operation

```
flushFromCache( addOfs32( RegB, RegX ));
```

Exceptions

Privilege violation trap.

Notes

None.

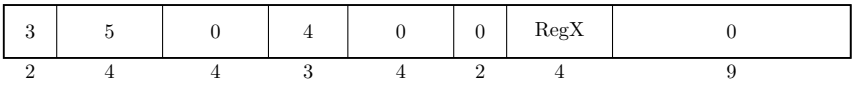
FxCE - flush Instruction or Data Cache Entries

Syntax

```
FICE [ RegX ] ( RegB )
FDCE [ RegX ] ( RegB )
```

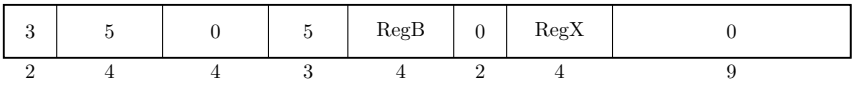
Format

The `FICE` instruction uses the following instruction format.



3	5	0	4	0	0	RegX	0
2	4	4	3	4	2	4	9

The `FDCE` instruction uses the following instruction format.



3	5	0	5	RegB	0	RegX	0
2	4	4	3	4	2	4	9

Description

The `FICE` instruction invalidates a cache line entry or a set of entries, selected by the effective address formed by adding `RegX` to `RegB`.

The `FDCE` instruction flushes and invalidates a cache line, selected by the effective address formed by adding `RegX` to `RegB`. The cache line addressed is written back to memory if and only if it is dirty, and then invalidated.

Both instructions are privileged instructions. The implementation is processor dependent.

Operation

```
flushEntriesFromCache( addOfs32( RegB, RegX ));
```

Exceptions

Privilege violation trap.

Notes

None.

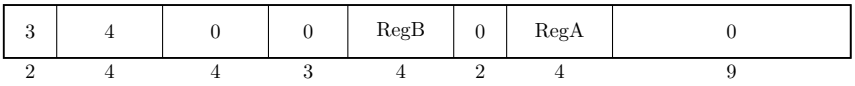
IxTLB - Insert TLB Entry

Syntax

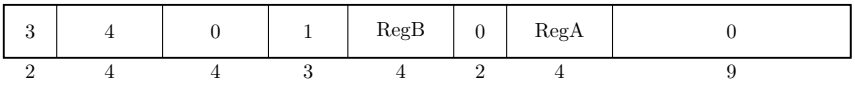
```
IITLB RegB, RegA
IDTLB RegB, RegA
```

Format

The `IITLB` instruction uses the following instruction format.



The `IDTLB` instruction uses the following instruction format.

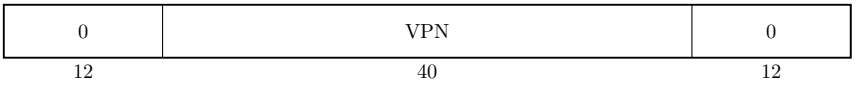


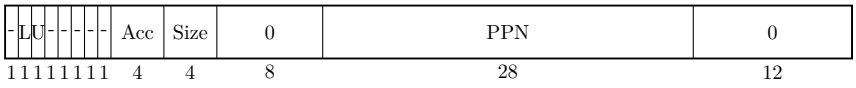
Description

The `IxTLB` instruction inserts a new entry into the translation lookaside buffer (xTLB). The virtual address is stored in `RegB` and additional control information is provided by `RegA`. This is a privileged instruction.

TLB Arguments

`RegB` and `RegA` contain the arguments passed to the TLB subsystem. `RegB` contains the virtual address. `RegA` contains the flags, access rights, page size and the physical address of the address range.





The L bit will lock the entry in the TLB. When looking for a free entry, these entries are skipped. If all entries are locked entry 0, TLB entry zero is selected for replacement. The U bit will mark the address range described by the TLB entry as uncached.

The access rights field is a four bit field that contains the page type and the privilege levels as shown in the access rights chapter.

The size field encodes the page size. It is 4-bit field encoding the supported page size:

```
( 1U << ( pSize * 2 )) * T64_PAGE_SIZE
```

A page size is not allowed to cross region boundaries and is aligned with selected the page size. Depending on the page size specified, the VPN and PPN fields are shorter and the offset is larger. The sum of both fields is however always 52 bits for virtual pages, and 40 bits for physical pages.

Operation

```
insertIntoTlb( RegB, RegA );
```

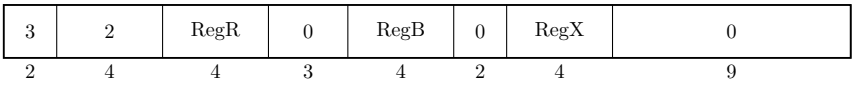
Exceptions Privilege Violation Trap.

Notes None.

LPA - Load Physical Address

Syntax `LPA RegR, [ RegX ] ( RegB )`

Format The `LD` instruction uses the following instruction formats:



Description The `LPA` (Load Physical Address) instruction computes the physical address corresponding to the given virtual address and loads it into `RegR`. The virtual address is formed by adding `RegX` to `RegB`. If there is no translation, a value of zero is returned.

The physical address, if found, is returned from information in the TLB. No memory access is performed. If there is a separate instruction and data TLB, the data TLB is used. For direct mapped regions the virtual address is the physical address.

Operation

```
RegR <- tanslateAdr( addOfs32( RegB, RegX ));
```

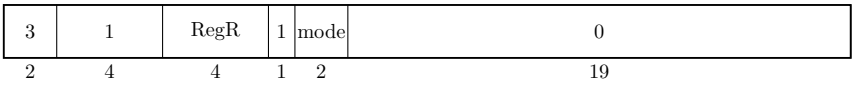
Exceptions
- Data Memory Alignment Trap
- Privileged instruction trap.
- Non access data TLB trap.

Notes A return value zero is ambiguous in that it means a non-existent translation or a mapping to page zero.

MFIA - Move from Instruction Address

Syntax `MFIA [.R/L/A]  RegR`

Format The `MFIA` instruction uses the register instruction format.



Description The `MFIA` instruction copies the current instruction address to register `RegR`.The mode field allows to select a portion of the instruction address.

- `.A` (mode = 0) — copies IA to `RegR`. This is the default operation.
- `.L` (mode = 1) — Deposits 20 bits of `IA.[31..12]` into bits [31..12] of `RegR`. `RegR` is cleared before.
- `.R` (mode = 2) — Deposits 20 bits of `IA.[51..32]` into bits [51..32] of `RegR`. `RegR` is cleared before.

Operation

```
RegR <- PSR.[IA]                          // .A mode (0)
RegR <- (( PSR.[IA] >> 12 ) & 0xFFFFF )   // .L mode (1)
RegR <- (( PSR.[IA] >> 32 ) & 0xFFFFF )   // .R mode (2)
```

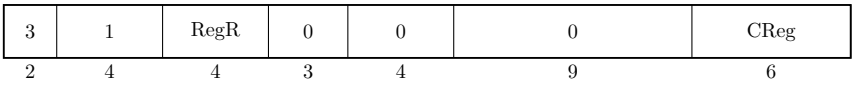
Exceptions None.

Notes None.

MFCR - Move From Control Register

Syntax `MFCR RegR, CReg`

Format The `MFCR` instruction uses following instruction format.



3	1	RegR	0	0	0	CReg
2	4	4	3	4	9	6

Description The `MFCR` instruction copies the contents of control register `CReg` to general register `RegR`. Some control registers are privileged. Attempting to read them in user mode results in a privilege exception.

Operation

```
RegR <- CReg;
```

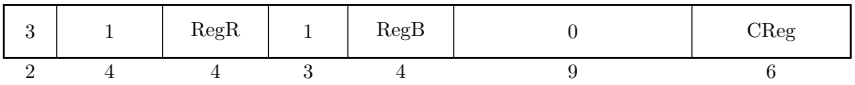
Exceptions - Privileged Operation Trap for some control registers.

Notes None

MTCR - Move To Control Register

Syntax `MTCR RegB, CReg [ , RegR ]`

Format The `MTCR` instruction uses the following instruction format.



3	1	RegR	1	RegB	0	CReg
2	4	4	3	4	9	6

Description The `MTCR` instruction copies the contents of the control register `CReg` to `RegR` and then the general register `RegB` to control register `CReg`. Some control registers are privileged. Attempting to write them in user mode results in a privilege exception.

Operation

```
RegR <- CReg;
CReg <- RegB;
```

Exceptions Privilege operation trap for some control registers.

Notes None

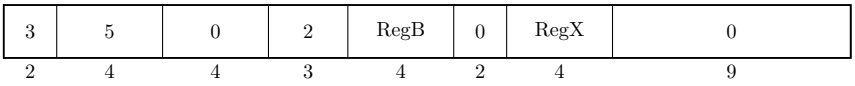
PxCA - Purge from Instruction or Data Cache

Syntax

```
PICA [ RegX ] ( RegB )
PDCA [ RegX ] ( RegB )
```

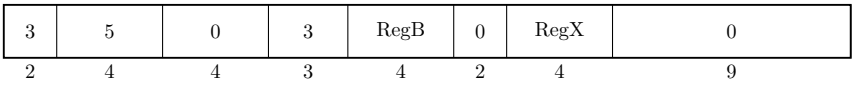
Format

The `PICA` instruction uses the following instruction format:



3	5	0	2	RegB	0	RegX	0
2	4	4	3	4	2	4	9

The `PDCA` instruction uses the following instruction format.



3	5	0	3	RegB	0	RegX	0
2	4	4	3	4	2	4	9

Description

The `PxCA` instructions purge a cache line, selected by the effective address in `RegB`. This can be used by privileged code for cache management. Unlike `FxCA`, a purge operation may also invalidate dirty lines without writing them back.

Operation

```
purgeFromCache( addOfs32( RegB, RegX ));
```

Exceptions

None.

Notes

Typically only allowed in supervisor mode.

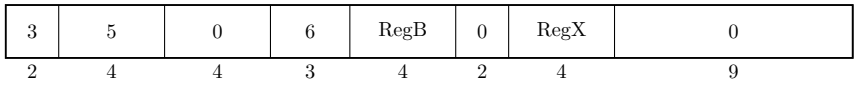
PxCE - Purge Instruction or Data Cache Entries

Syntax

```
PICE [ RegX] ( RegB )
PDCE [ RegX] ( RegB )
```

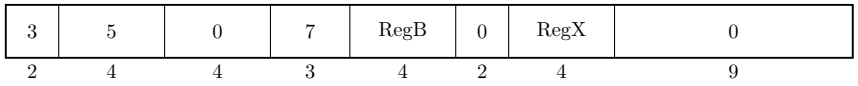
Format

The `PICE` instruction uses the following instruction format:



3	5	0	6	RegB	0	RegX	0
2	4	4	3	4	2	4	9

The `PDCA` instruction uses the following instruction format.



3	5	0	7	RegB	0	RegX	0
2	4	4	3	4	2	4	9

Description

The `PxCA` instructions purge a cache entry or a set of cache entries selected by the effective address. in `RegB`. This can be used by privileged code for cache management. Unlike `FxCA`, a purge operation may also invalidate dirty lines without writing them back.

Operation

```
purgeEntriesFromCache( addOfs32( RegB, RegX ));
```

Exceptions

None.

Notes

Typically only allowed in supervisor mode.

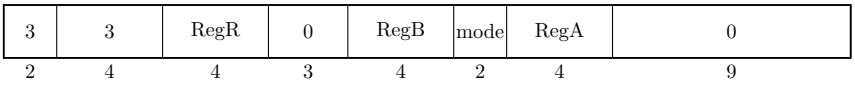
PRB - Probe Virtual Address

Syntax

```
PRB.A RegR, RegB, RegA
PRB .R/W/X RegR, RegB
```

Format

The `PRB` instruction uses both the following instruction format:



3	3	RegR	0	RegB	mode	RegA	0
2	4	4	3	4	2	4	9

Description

The `PRB` instruction probes the Translation Lookaside Buffer (TLB) to determine whether the effective address in `RegB` has a valid mapping. The result of the probe is returned in `RegR`. A value of `0` indicates that no mapping exists, while a `1` indicates a valid mapping.

The test option is encoded in the mode field. An instruction option of `R` encodes as zero, a `W` encodes as one and `X` encodes as two. A mode field value of three indicates that the instruction option is encoded in right most two bits of `RegA`. Here a value of 3 is undefined.

Operation

```
RegR <- probeVirtualAddress( RegB, RegA.[1..0]);    // Reg mode
RegR <- probeVirtualAddress( RegB, instr.[mode] );  // Imm mode
```

Exceptions

- Non-access TLB Trap
- Privileged Operation Trap

Notes

Typically used by the operating system for virtual memory management.

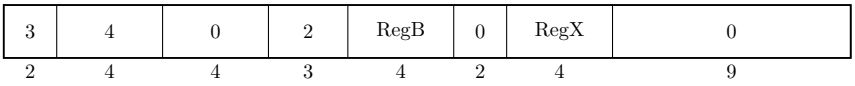
PxTLB - Purge TLB Entry

Syntax

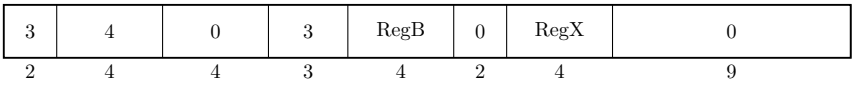
```
PITLB [ RegX ] ( RegB )
PDTLB [ RegX ] ( RegB )
```

Format

The `PITLB` instruction uses the following instruction format:



The `PDTLB` instruction uses the following instruction format.



Description

The `PxTLB` instructions purge (remove) an entry from the translation lookaside buffer (TLB). The entry to remove is identified by the effective address formed by adding `RegX` to `RegB`. This is privileged instruction.

On multi-processor system, the purge TLB instruction affects the TLB of all processors.

Operation

```
purgeFromTlb( addOfs32( RegB, RegX ));
```

Exceptions

- Privileged operation trap.

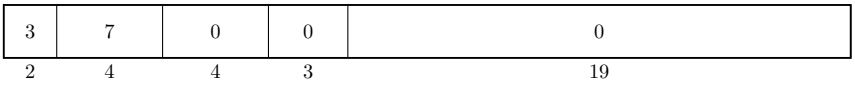
Notes

None.

RFI - Return from Interrupt

Syntax `RFI [ RegR ]`

Format The `RFI` instruction uses the branch instruction format.



3	7	0	0	0
2	4	4	3	19

Description The `RFI` instruction restores processor state after an interrupt or exception handler has completed. The content is loaded from the `IPSW` control register.

Operation

```
PSW <- IPSW;
```

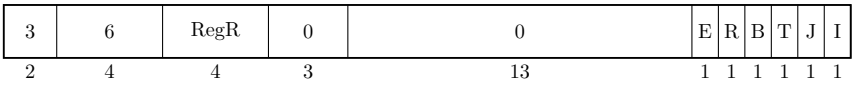
Exceptions Privileged operation trap.

Notes None.

RSM - Reset System Mask

Syntax `RSM RegR, mask`

Format The `RSM` instruction uses the following instruction format:



Description The `RSM` instruction clears bits in the system mask register that are encoded in the instruction. The previous mask values are returned in `RegR`.

Operation

```
RegR <- 0;
RegR.[58] <- PSW.[E];
RegR.[59] <- PSW.[R];
RegR.[60] <- PSW.[B];
RegR.[61] <- PSW.[T];
RegR.[62] <- PSW.[J];
RegR.[63] <- PSW.[I];
PSW.[E] <- PSW.[E] & ( ~ instr.[E] );
PSW.[R] <- PSW.[R] & ( ~ instr.[R] );
PSW.[B] <- PSW.[B] & ( ~ instr.[B] );
PSW.[T] <- PSW.[T] & ( ~ instr.[T] );
PSW.[J] <- PSW.[J] & ( ~ instr.[J] );
PSW.[I] <- PSW.[I] & ( ~ instr.[I] );
```

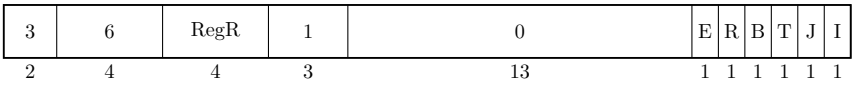
Exceptions - Privileged instruction trap.

Notes None.

SSM - Set System Mask

Syntax `SSM RegR, mask`

Format The `SSM` instruction uses the following instruction format.



Description The `SSM` instruction updates the system mask register from the bits encoded in the instruction. The previous mask is returned in `RegR`.

Operation

```
RegR <- 0;
RegR.[58] <- PSW.[E];
RegR.[59] <- PSW.[R];
RegR.[60] <- PSW.[B];
RegR.[61] <- PSW.[T];
RegR.[62] <- PSW.[J];
RegR.[63] <- PSW.[I];
PSW.[E] <- PSW.[E] | instr.[E];
PSW.[R] <- PSW.[R] | instr.[R];
PSW.[B] <- PSW.[B] | instr.[B];
PSW.[T] <- PSW.[T] | instr.[T];
PSW.[J] <- PSW.[J] | instr.[J];
PSW.[I] <- PSW.[I] | instr.[I];
```

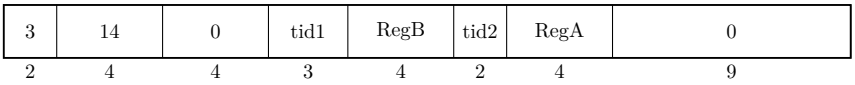
Exceptions - Privileged instruction trap.

Notes None.

TRAP - Trap Instruction

Syntax `TRAP tid, RegB, RegA`

Format The `TRAP` instruction uses the following instruction format.



3	14	0	tid1	RegB	tid2	RegA	0
2	4	4	3	4	2	4	9

Description The `TRAP` instruction generates a synchronous trap to handle exceptional conditions. The `tid1` and `tid2` are concatenated to form the trap Id. The instruction may be used to invoke operating system routines or exception handlers.

Operation

```
raiseTrap( concat( tid1, tid2 ), RegB, RegA );
```

Exceptions The trap raised. See the appendix for valid trap codes.

Notes None

Part IV

Runtime Environment

Runtime Overview

No CPU architecture with an idea of a runtime environment. Although the instruction set can be used to implement many models of runtime environments, there are common principles that exploit the CPU architecture. In fact, several CPU concepts were selected with a certain runtime already in mind. This chapter will present the Twin-64 runtime environment and describe the high level system model, the register usage convention, the execution model and calling convention.

System Environment

The following figure depicts a high-level overview of the overall software environment for the Twin-64 system. At the center of the execution model is the execution thread, referred to as a **task**. A task consists of the currently running execution object together with its private task-local data area, including its own stack and runtime context. All tasks of a job share a common job data area. At the highest level is the **system**, which contains the global data area for the system.

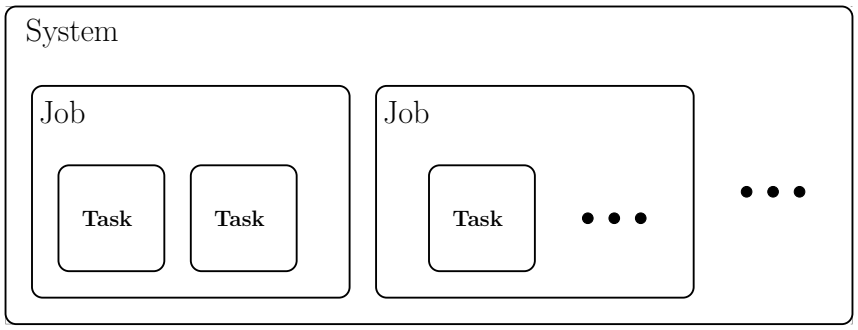


Figure 14: System Environment

All tasks belonging to the same job share a common job data area, which is used to store information and resources that must persist for the lifetime of the job and be visible across all of its tasks. A job may represent an interactive session, such as a terminal session, or it may be a non-interactive batch operation.

The system level contains the global data area, system libraries, and all services provided by what is traditionally called the operating system. The system is responsible for creating jobs, managing their lifetime, and scheduling the tasks within them.

The number of active tasks at any moment typically corresponds to the number of available processors in the system, allowing the system to take full advantage of available parallelism. While each task maintains its own private data and execution state, tasks within a single job

share access to the job data area. All tasks, regardless of job, also have access to system-level services and global resources.

Runtime Environment

The processor is a general-purpose CPU. The runtime model presented here represents one possible implementation strategy, chosen to support the calling conventions and execution model that will be described in later sections. The following figure depicts a high-level overview of the runtime environment.

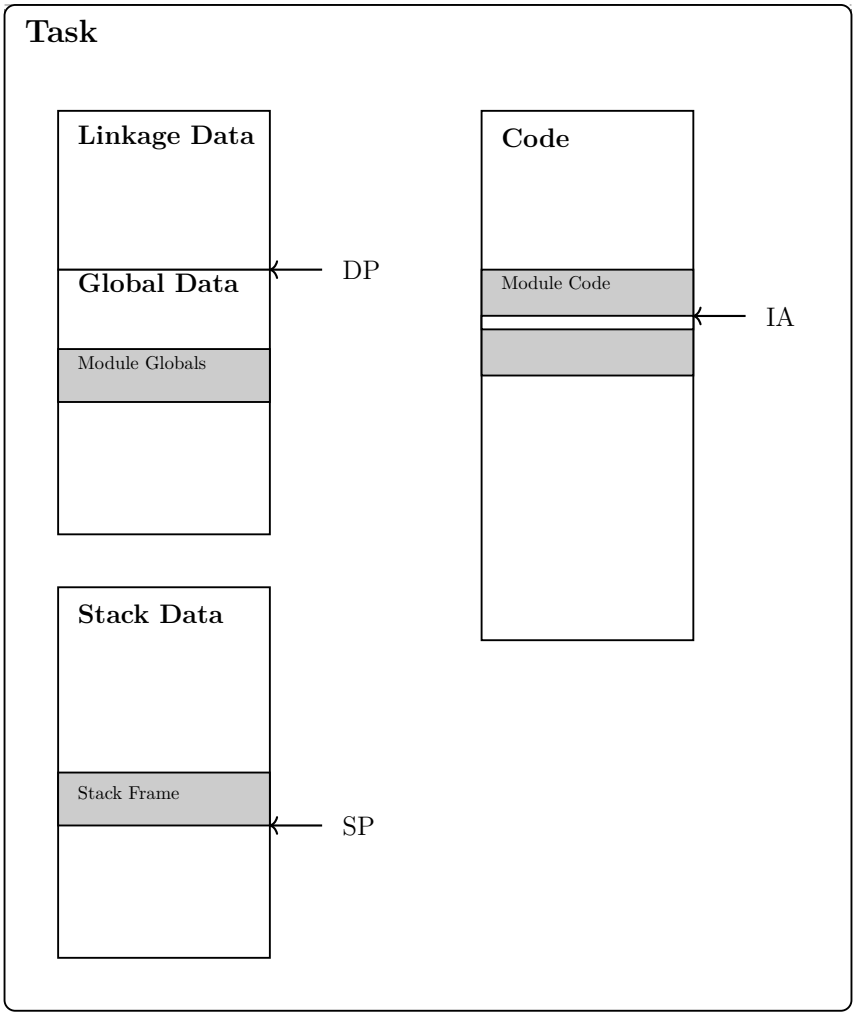


Figure 15: Runtime Environment

When a compiler or assembler generates code, the fundamental unit of work is the compilation unit. A compilation unit corresponds to a single invocation of the compiler or assembler. Although syntax varies by language, the source text is organized into modules. A module is an entity that contains constant declarations, type definitions, module-level global variables, and functions.

The task's code region contains the program text of all modules used by the task. Once loaded, this region is immutable. The currently executing instruction is identified by the instruction address (IA).

Each task also has a global data region. This task-level global data is formed by aggregating the global data sections of all modules referenced by the task. It is addressed using the data pointer (DP) plus an offset. The data region includes the task's stack, which holds stack frames created by function calls. The active frame is located via the stack pointer (SP), and items within the stack are accessed relative to SP.

The data region further contains the linkage table, a structure that provides the information needed to access functions in external libraries. Details of this mechanism are described in the section on linkage tables.

In this runtime design, the stack grows upward: new frames occupy increasing addresses, and older frame contents are typically referenced using expressions of the form `SP - offset`. Each stack frame includes a frame marker containing the previous SP and the return link (RL), enabling structured returns and proper unwinding of frames.

Register Usage

The CPU features general registers and control registers. All general registers can do computation as well as address computation. The runtime architecture builds on the general register file and assigns specific meaning to each register.

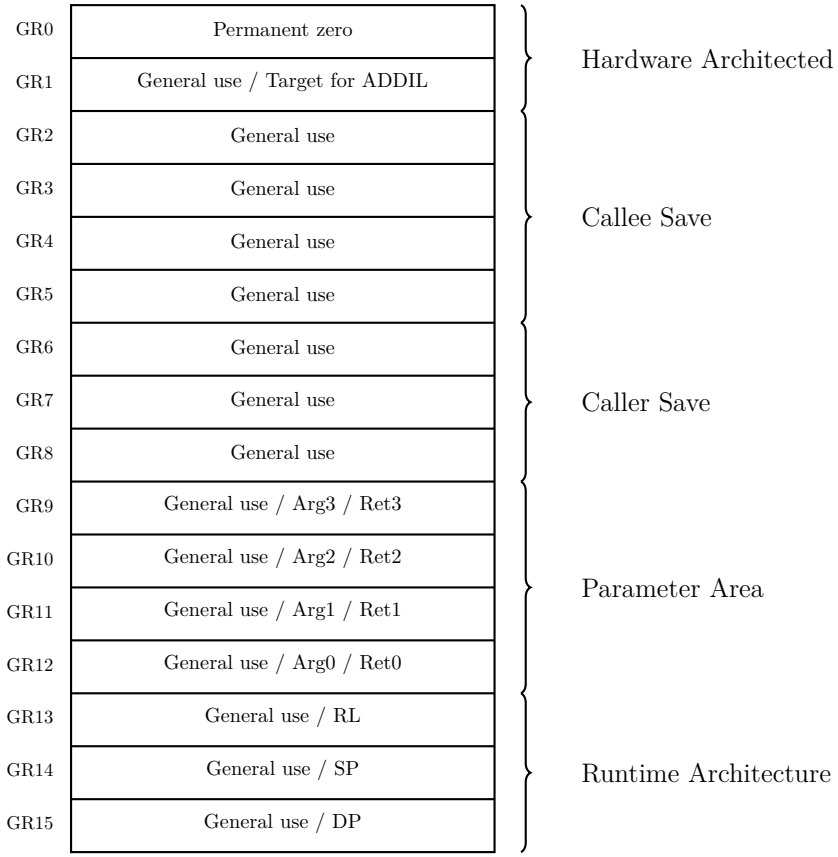


Figure 16: General Register Usage

Registers `R0` and `R1` are hardware-architected and have fixed, predefined roles within the processor. The remaining registers, `R2` through `R15`, form the general-purpose register file, whose usage is defined by the runtime environment.

Registers `R2` through `R5` form the callee-save register set. A callee-save register is one whose value must be preserved by the called function: if the function uses such a register, it must save its original contents upon entry and restore them before returning. This ensures that long-lived values in the caller remain intact across function calls.

Registers `R6` and `R8` are designated as caller-save registers. A caller-save register may be overwritten by the called function; therefore, the caller is responsible for saving the register's value before making a call if it must be preserved.

Argument passing is handled through registers `R9` to `R12`, known as the parameter registers. When a function is invoked, its first four arguments are placed into these registers. Registers `R11` and `R12` also serve as the return-value registers, allowing a function to return up to two values without using memory.

Control flow is supported by register `R13`, the return-link register (RL). During a function call, the branch instructions store the return address in `R13`, allowing the function to return control to its caller.

Register `R14` serves as the stack-frame pointer (SP). It marks the base of the current stack frame, which contains local variables, spilled registers, and temporaries, and supports predictable stack-based data management within the task's runtime environment.

Finally, register `R15` is assigned as the data pointer (DP). It serves as the base reference for accessing the task's global data region.

Stack Layout

Most modern programming languages rely on a **stack** to manage function calls, local variables, and return information. This stack is organized into **stack frames**, each representing the state of an active function invocation. When a function is called, a new frame is pushed onto the stack; when it returns, that frame is popped. This structured approach allows programs to keep track of nested calls efficiently and ensures that each function has its own isolated workspace during execution.

Stack Frame

Stack frames are aligned on a 16-byte boundary. A stack frame consists of four main areas, depicted in the following figure.

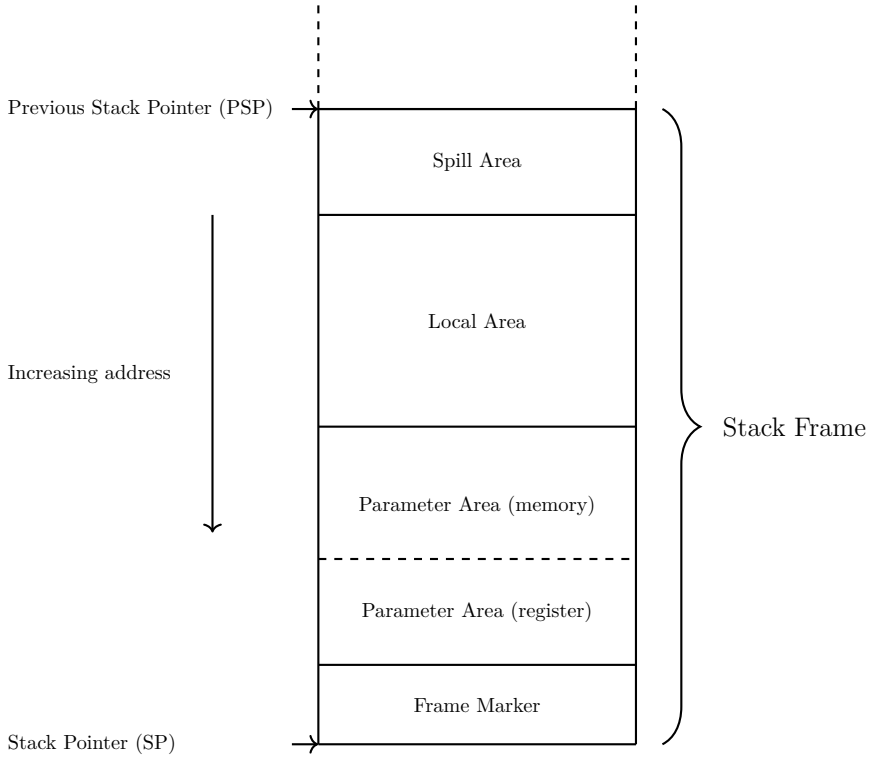


Figure 17: Stack Frame

The **stack frame marker** holds a pointer to the previous stack frame along with the return link address, allowing the call chain to be restored during function returns.

The **parameter area** is used for passing arguments to the called function and for returning data. Although by convention the first four parameters are passed in registers, space for them

must still be reserved in memory. Any parameters beyond the first four are placed directly in this area.

The **local area** stores the function's local variables. This is the region the function uses for data that must persist throughout its execution.

Finally, the spill area is used to save register values that must be preserved for the caller and to spill intermediate values when register allocation cannot keep all live data in registers.

Stack Frame Addressing

All data items within a stack frame are accessed **relative to the current stack pointer (SP)**. The architecture uses an `SP - offset` addressing scheme, where each region of the stack frame, i.e. the frame marker, parameter area, local data area, and spill area are located at a fixed negative offset from the SP at function entry. This convention allows the compiler and debugger to compute the location of every stack-resident object from the SP alone, without requiring a dedicated frame pointer. As long as the SP remains stable during the execution of the function, all stack frame items can be reliably referenced through their predefined offsets.

Stack Frame Marker

A stack frame marker consists of two double words. The frame marker is pointed by the current stack pointer register (SP).

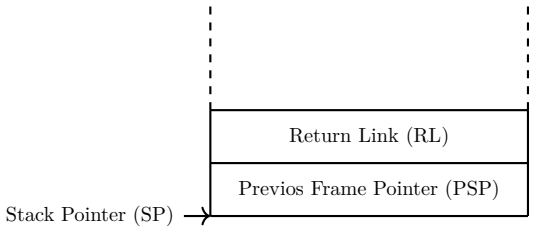


Figure 18: Stack Frame Marker

The first word holds the **address of the previous stack frame**. This field does not necessarily store the previous frame marker address, as the frame size can often be inferred from the instructions that created the current frame together with symbolic debugging information in the program image.

The second word contains the **return link**, which is the address immediately following the branch instruction in the calling function. If the called function is a leaf function, there is no need to save the return link in memory.

Parameter Area

A stack frame includes a **parameter area** large enough to hold the largest argument list passed from the calling function to the callee. Although the first four parameters are by convention passed in registers, corresponding space is still reserved in the frame to provide a uniform layout and to support cases where the callee needs to spill register arguments to memory. Any additional parameters beyond the first four are passed directly in the memory locations immediately below this register-parameter area. If a function neither receives parameters nor returns data, the parameter area is omitted from its stack frame, avoiding unnecessary stack usage.

Local Data Area

Immediately below the parameter area lies the **local data area**, which holds all automatic (local) variables of the function. These variables exist only for the duration of the function call and are allocated anew for each invocation. The compiler determines the size and placement of each local variable within this area, taking into account alignment requirements and optimization opportunities. If a function declares no local variables, this region has size zero and is omitted from the final stack frame layout.

Spill Area

The **spill area** contains any registers that the function must temporarily save in order to preserve the calling convention. Registers that must remain unchanged across function calls are saved (``spilled'') into this area upon entry and restored before returning to the caller. The compiler allocates spill slots only when necessary. If for example a function does not need to save any registers, the spill area is not present in its stack frame. This selective allocation helps keep stack usage minimal while maintaining correct program behavior.

Calling Convention

The previous chapters introduced the register-usage conventions and the overall stack layout defined by the runtime architecture. Building on that foundation, this chapter describes the mechanisms used to invoke routines and the conventions that govern control transfer, parameter passing, register preservation, and stack-frame management.

The discussion begins with the simplest form of call: the *local call*, which transfers control between routines within the same module. Local calls illustrate the fundamental steps of the calling convention without the additional considerations required for inter-module calls.

Calling Sequence

A routine call proceeds through several well-defined stages. First, the caller evaluates all argument expressions and places the resulting values into the parameter area, either in registers or on the stack as required. After preparing the parameters, the caller saves any registers whose values must remain intact across the call. These caller-saved registers must be preserved by the caller before branching to another routine.

The callee then begins execution at its entry point. It accesses the incoming parameters in the established layout of the parameter area, allocates a stack frame, records the return link, and saves any callee-saved registers it intends to overwrite. This setup ensures that caller and callee operate independently while maintaining a consistent runtime state.

When the callee completes its work, it restores all callee-saved registers, retrieves the return link from the frame marker, deallocates its stack frame, and transfers control back to the caller. This division of responsibility allows efficient cooperation between routines and avoids unnecessary state preservation.

As described earlier, the general-purpose register set is partitioned into caller-save and callee-save regions. A compiler selects registers from these regions as needed, balancing the work of saving and restoring state between the caller and the callee. When possible, redundant saves and restores are eliminated. The runtime architecture distinguishes between two basic categories of calls: *local calls*, which remain within a single module, and *external calls*, which cross module boundaries and require additional handling. The following sections describe these types in detail.

Local Call

A local call is the most common form of procedure call. The following code snippet illustrates such a call. The caller loads the arguments, saves any registers the callee might modify, and

branches to the callee. After the callee returns, the caller restores its saved registers and continues execution.

```
# calling procedure

            ...                 # load arguments
            ...                 # save caller saves
            BL target, RL       # branch to "target", save return
            ...                 # restore caller saves
```

The callee begins by saving the return link, allocating its stack frame, and saving any registers that must remain unchanged across the call. When the routine completes, it restores the return address, restores callee-saved registers, deallocates the stack frame, and branches back to the caller.

```
# called procedure

target:     ST RL, -16(SP)      # save RL
            ST SP, -8 (SP)      # save PSP
            LDO SP, size(SP)    # allocate stack frame
            ...                 # save callee save

            ...                 # do the work


            ...                 # restore callee saves
            LDO SP, -size (SP)  # deallocate stack frame
            LD RL, -16(SP)      # restore RL
            BV (RL)             # return to caller
```

This sequence shows all steps that may be required for a local call. Depending on the specific call and the nature of the callee, some steps may be omitted. For example, if the caller does not use any caller-saved registers, no saving is needed before the call. Likewise, a routine with no arguments does not need to evaluate arguments. Also, saving the previous stack pointer is an optional operation.

The callee can also omit steps. A *leaf procedure*, i.e. one that calls no other routines, does not need to save the return address. If it does not use any callee-saved registers, those need not be saved or restored. Finally, if a leaf procedure uses no caller-saved registers, has no local variables, and does not need to preserve temporary registers, it may require no stack frame at all.

The instructions `B`, `BR`, and `BV` are used for local branches. The address computation, whether IA-relative or region-relative, does not modify the region portion of the instruction address. It is always the region of the currently executing code.

Parameter passing

Procedures may pass arguments to a callee and receive return values. The parameter area is located in the caller's stack frame, below the frame marker.

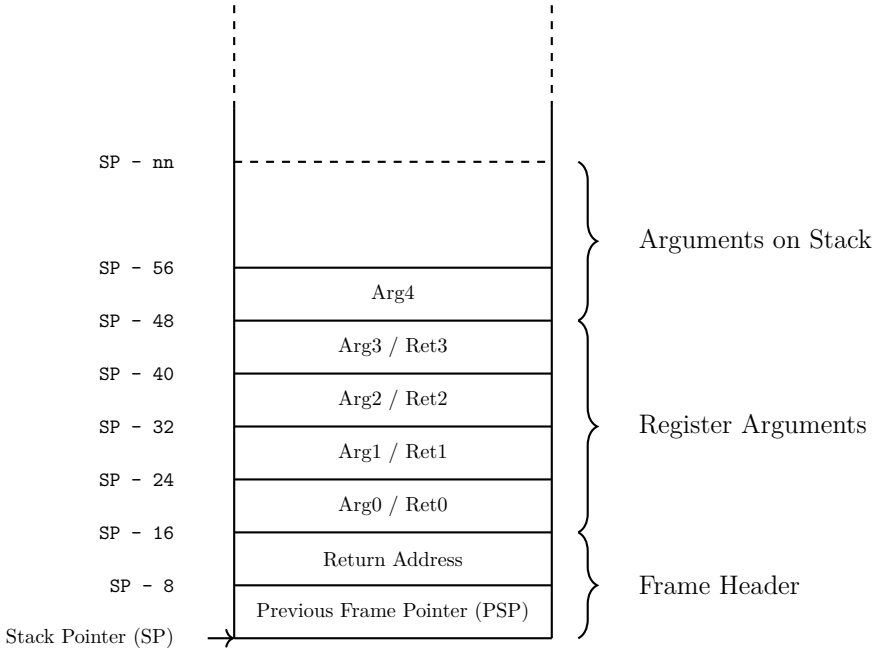


Figure 19: Argument Passing

The first four arguments are passed in registers `ARG0` through `ARG3`. Nevertheless, the caller's stack frame reserves space for them in the parameter area. Any additional parameters are stored on the stack. Upon return, the callee may supply up to two return values in registers `RET0` and `RET1`.

Arguments and return values are addressed relative to the caller's frame pointer. Because the callee may allocate its own stack frame, the size of the callee's frame must be added to the fixed offsets of the arguments in the caller's frame.

External Calls

An external call is one that may cross a region boundary. The `BE` instruction computes the effective address from a general register and an immediate offset encoded in the instruction. Like local branches, this computation produces a 32-bit unsigned address offset. However, unlike local branches, the base register contains the full virtual address, region and offset, not just an offset. The `BE` instruction is used both to call a target in another region and to return from it. The following code snippets shows a basic external call.

```
1
2  # calling code
3
4              ...                 # load arguments
5              ...                 # save caller saves
6              LD R1, <target>)    # obtain the target address
7              BE R1, RL           # branch to "target", save return
8              ...                 # restore caller saves
```

This snippet illustrates only the external branch. The full runtime convention is more elaborate. When calling an external function, the target address and its associated DP value must be obtained. This information is stored in the **linkage table**, which is populated with actual values at load time. This indirection allows compiled code to remain unchanged regardless of where the external function is ultimately located.

Because there is no practical way for the callee to determine its correct DP value in a shared--code model, the caller is responsible for setting DP before branching outside its module. The necessary steps of loading the target address, loading the target DP, and performing the external branch are placed into a small piece of code called a *stub*. A stub is generated at compile time and inserted into the module's code. The branch instruction is patched during the linking process.

A generic external call sequence is shown below:

```
1
2  # calling code
3
4              ...
5              ...                      # load arguments
6              ...                      # save caller saves
7              ...                      # save DP
8              B stubF1                 # branch to the stub for "F1"
9              ...                      # restore DP
10             ...                      # restore caller saves
11             ...
12
13 # stubF1
14
15 stubF1:     LD R1, -ofs(DP)          # load target address of F1
16             LD DP, -ofs + 8(DP)      # load target DP value
17             BE (R1)                  # branch to target
```

Before the stub loads the new DP value, the caller must save the current DP. This step is always required, even if the called function does not use DP. The Twin-64 runtime model performs this work in the caller's code before branching to the stub. This division has several benefits. First, the caller can optimize DP saving. For example, by dedicating a register to hold it so that only restoring DP is necessary after external calls. Second, the stub's sequence of loading the target

address and DP value is shared for all calls to the same external function. Finally, the target function is unaware of whether it was reached by a local or external call:

```
1
2  # called code
3
4  target:     ST RL, -16(SP)          # save RL
5              ST SP,  -8(SP)          # save PSP
6              LDO SP, size(SP)        # allocate stack frame
7              ...                     # save callee save regs
8
9              ...                     # do the function work
10
11             ...                     # restore callee saves
12             LDO SP, -size(SP)       # deallocate stack frame
13             LD RL, -16(SP)          # restore RL
14             BE (RL)                 # branch to return location
```

Local calls may use `BV` to return, since the region portion of the address does not change. Exported functions, however, may return either locally or externally, and therefore must use `BE`. It is recommended to simply always use the `BE` instruction.

Linkage Table

The assembler or compiler generates the skeleton of the linkage table. The linkage table resides below the global data area. Entries are addressed using negative DP offsets.

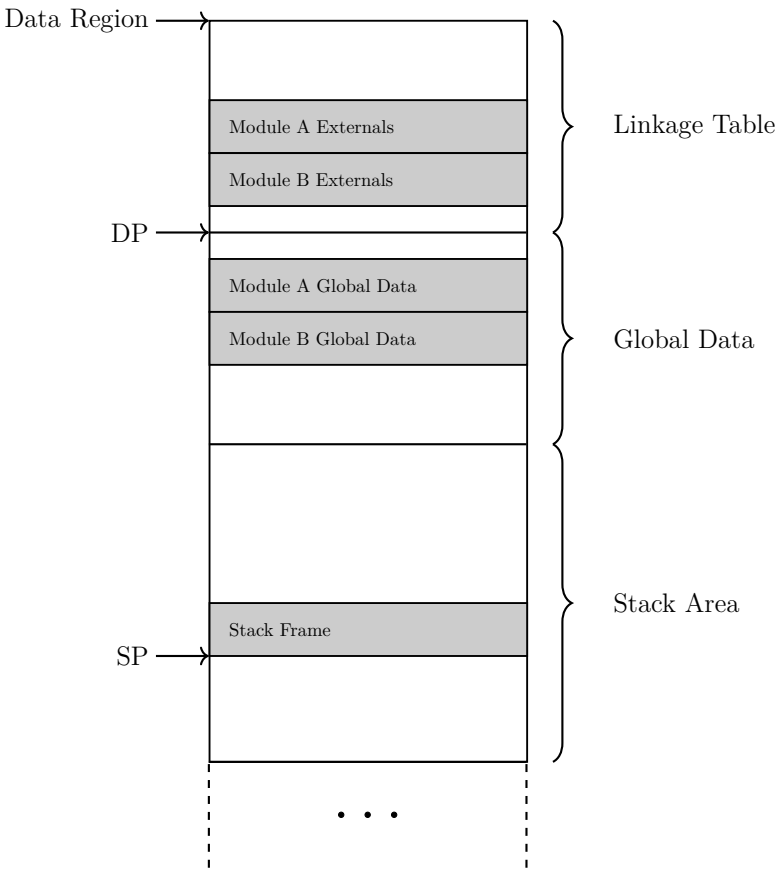
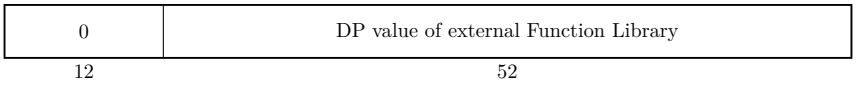


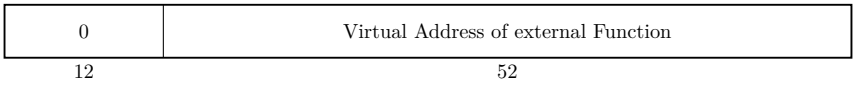
Figure 20: Linkage Table

The figure illustrates two modules within a task's global data region. All module global data is addressed as DP plus an offset, while linkage-table entries are addressed as DP minus an offset. The compiler constructs the linkage table during compilation. The linkage table contains per external library several entries.

The DP value entry starts the linkage subtable for a particular library.



The function target entry contains the target address of the function.



Function Labels

A function label is a reference to a function which is resolved at calling time. Therefore each function that is accessed via a function label has an entry in the linkage table. The function label itself is then the index into the linkage table.

??? the entry is two pieces: DP index and function index. To be defined...




```
1
2 # example
3             ...                 #
4             ...                 #
```

Privilege Level Changes

```
1
2 # example
3             ...                 #
4             ...                 #
```

Design Rationale

There are many possible ways to organize modules and their external linkages. Classic CISC architectures provided simple call instructions that implicitly performed all the required steps such as address computation, access-rights checking, and memory references needed to resolve external linkages.

RISC designs, in contrast, expose only simple branch and jump instructions and rely on explicit caller–-callee code sequences to implement calling conventions. Because compilers require a unified and predictable calling sequence, the additional work needed for external calls is typically done via import and export stubs.

Twin-64 follows a similar model. External calls use an import stub, which is responsible for locating the target function and obtaining the external library's DP value before issuing the external branch instruction. The caller saves its own DP value and restores it upon return.

This approach adds two instructions at each external call site, but provides a key benefit: it avoids two additional branches, one from the import stub to the callee and one back from the import stub to the caller. It also centralizes DP handling. Because the stub does not manipulate the caller's DP value, the caller has full flexibility. It may save DP to memory, move it to another register, or even save it once and reload it after each external call, depending on optimization goals.

Thus an external call requires only one additional branch in the calling sequence, and the code expansion for saving and restoring the caller's DP is modest. The callee is completely unaware of the caller's region, so no export stub is required.

An additional advantage is that the compiler does not need to generate any special code for functions that may be called externally. This reduces complexity and places less burden on the compiler designer.

Part V

I/O Subsystem

Input/Output

This chapter presents the Twin-64 I/O architecture and introduces its core concepts related to the processor and system architecture. It does not describe any specific I/O module, nor does it cover operating-system initialization or runtime I/O procedures.

Concepts

Twin-64 uses a memory-mapped I/O architecture. A dedicated portion of the physical address space in address region zero is reserved exclusively for the I/O subsystem. This clean partitioning ensures predictable system behavior during early initialization and simplifies both hardware implementation and software design.

The I/O subsystem is organized around functional entities known as **modules**. Typical modules include processors, memory controllers, and various I/O devices. All modules connect to the **system bus**, which supports up to 64 modules in total. Although real systems often provide fewer physical slots, the architecture itself imposes no such restriction. At system startup, each module is assigned a dedicated 4-KB physical address page. This page defines the base address range to which the module responds and serves as a fixed anchor for its control and status information.

Within each module, functionality is exposed through a collection of **registers**. These registers are accessed using standard load and store instructions, although they are not required to be 64 bits wide. A core set of registers is common across all modules, establishing a uniform interface for essential operations. Each module may define additional device-specific registers to implement its specialized capabilities.

Many modules also require additional storage to manage their internal **I/O elements**. An I/O element—also called a **device** is the smallest addressable functional unit within a module. For example, a console terminal module may expose two I/O elements: one for input and one for output.

The system bus spans a 256-KB address space, aligned to a 256-KB boundary, with one 4-KB page reserved for every possible module. In practice, physical constraints often limit the number of installed modules, but the architecture supports growth by allowing multiple modules to reside on a single I/O card, with each card occupying one physical slot on the bus.

Overall, the Twin-64 I/O subsystem emphasizes clarity, modularity, and extensibility. Its hierarchical structure—from modules to devices, and from hard physical pages to optional soft and broadcast regions—supports both straightforward configurations and more complex, densely populated systems.

I/O Address Space

As shown in the overall physical memory layout, the physical memory address range dedicated to I/O modules is located from `0xFF_0000_0000` to `0xFF_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges.

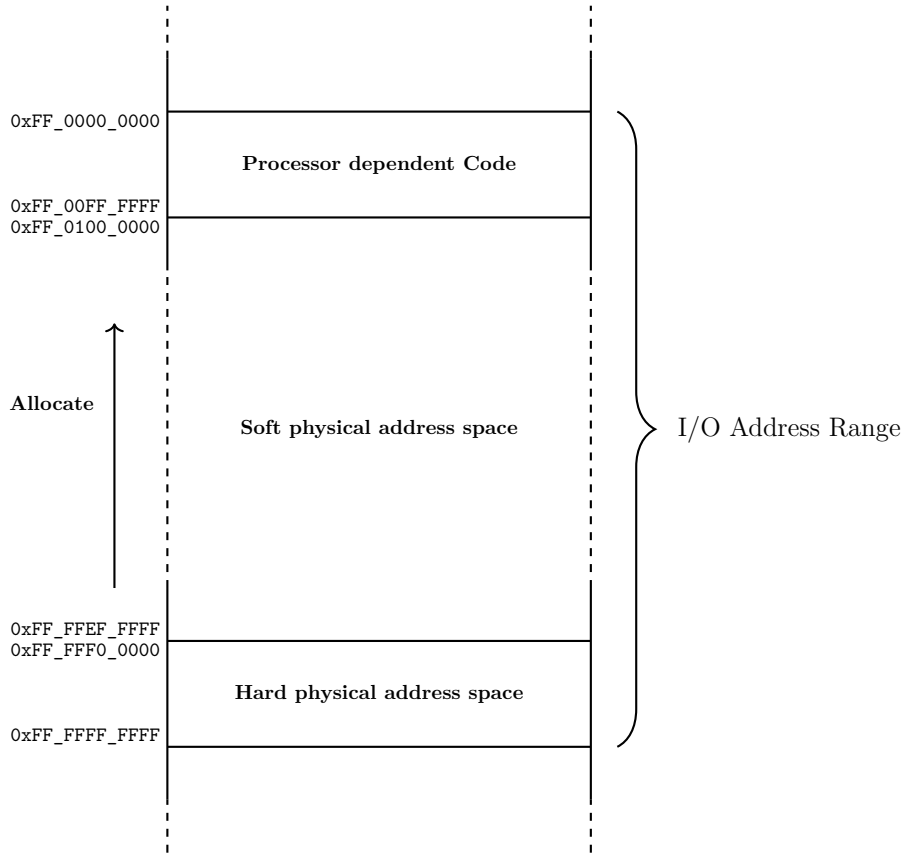


Figure 21: I/O Address Space

The I/O address range is a 4 Gbyte area at the end of the 40-bit physical address space. The area is divided into several ranges. The first range contains the **processor dependent code**. There is a set of library functions to provide low-level primitives for the boot process and the operating system.

At the upper level of the I/O address range there is the **hard physical address** range. It contains a page for each module connected at the system bus level. Up to 256 modules can be allocated. The last module slot is the broadcast HPA. At system reset or reboot, there is only the broadcast HPA. The processor will try to locate all other modules in the physical address range. Finally, each I/O module will then allocate its required soft physical address range. Each module therefore listens to three distinct address ranges:

- **Hard physical address range:** This is the module's assigned physical page at system initialization. Its position is determined by the card's physical slot and the module's index on the I/O card. Access to the hard physical page is privileged and reserved for supervisor-level operations.

- **Soft physical address range:** Certain modules require more than a single page of address space to support their full operational state. For such cases, additional memory pages may be dynamically allocated in the soft physical region, providing the necessary flexibility without compromising layout consistency.

- **Broadcast address range:** A write issued through the broadcast HPA address updates the corresponding register in *every* module on the bus. Broadcast writes are useful for system-wide resets, synchronization operations, or events that must be delivered simultaneously to all modules.

Hard physical address range

The **hard physical address** range contains an I/O page for each I/O module installed. There is room for 64 I/O modules in the system bus. Each module has a hard physical address page, which consists of up to 512 64-bit registers each. The first registers are the supervisory registers. This set is architected and must be implemented by each module. It contains among other items the I/O module description. The remainder of the registers in the HPA page and their organization is specific to the I/O module.

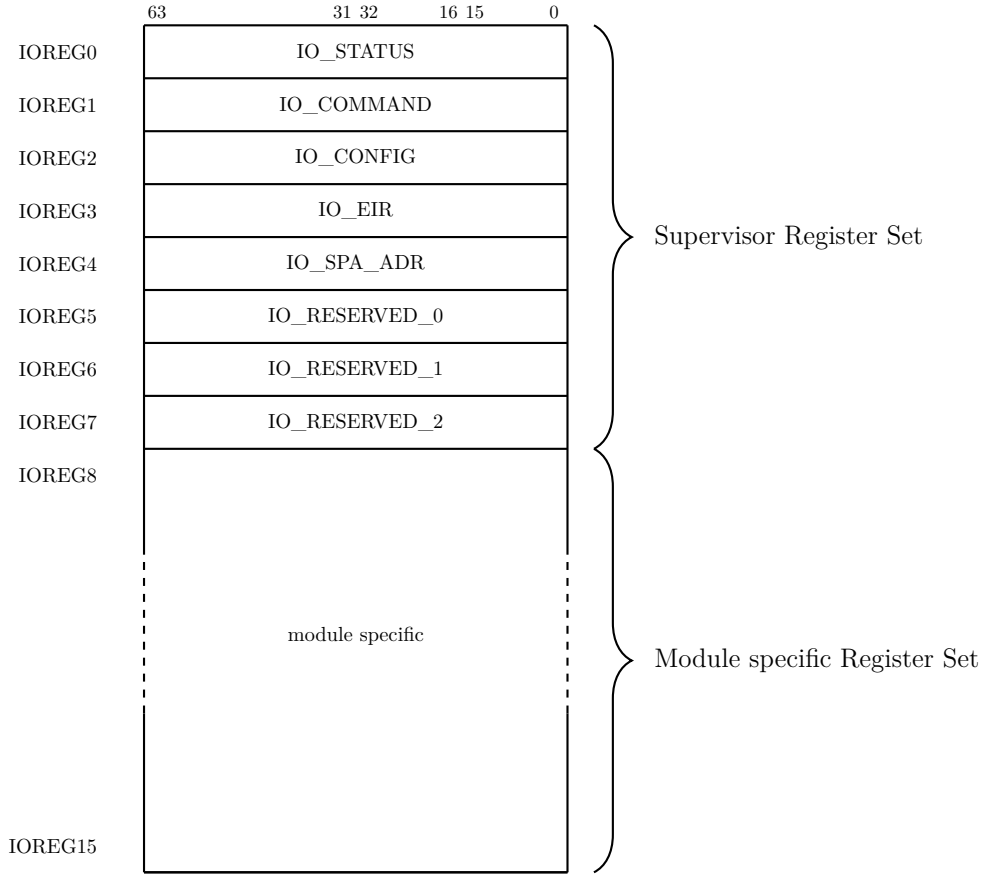


Figure 22: Hard physical Address Page

The supervisor registers are described in detail later in this chapter.

Broadcast address range

I/O module 255, i.e. the last page in the HPA address range, is the broadcast address range. It is actually not a module and contains no actual functions. Writes issued to broadcastable registers using this module number are not directed to a single module. Instead, the write operation is applied simultaneously to all I/O modules on the system bus. This mechanism is typically used for system-wide commands such as resets, synchronization events, or configuration updates that must reach every module at once.

At reset time, all modules only listen to their broadcastable registers. Only after system bus setup and module discovery, is the HPA address range available.

Soft physical address range

In many cases, a module requires more addressable registers or data structures than can be accommodated within its hard physical address page. The **soft physical address** range provides a flexible region where additional space may be allocated for an I/O module's extended functionality. From a software perspective, the allocated area is termed **I/O Element**. However, the meaning of register sets in this range is entirely up to the module. A module responds to memory accesses within its configured soft physical address space, which may reside either in the I/O address space or the general memory address space.

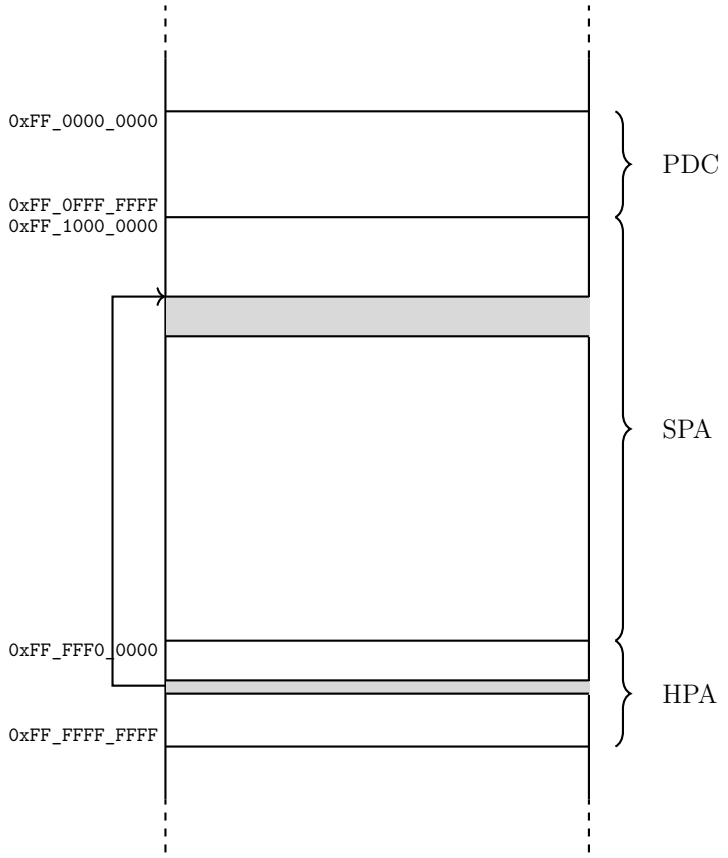


Figure 23: I/O Address Space

For example, a terminal I/O controller has a hard physical page in the I/O address space and a SPA address allocated for its terminal ports. As a another example, a memory controller has a hard physical page in the I/O address space, yet its soft physical region typically encompasses all or part of the system's physical memory. Similarly, a processor may allocate a soft physical address range within the I/O space to store performance counters or diagnostic information that needs to be accessible without interfering with normal memory traffic.

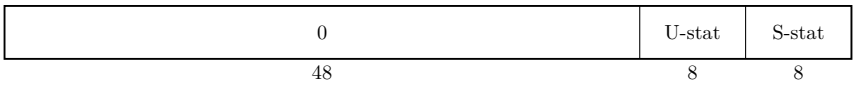
Supervisor Register Set

The supervisor registers implements the architected set of registers for configuring and managing the I/O module. The `REG 0` to `REG 7` are implemented by each module. The following figure depicts the overall layout of the HPA page. The following table lists the architected registers for an I/O module. Access to these registers is always on a double word boundary.

Table 7: Supervisor Register Set

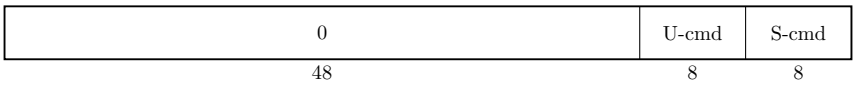
Reg	Purpose
IO_STATUS	I/O module status
IO_COMMAND	I/O module command
IO_CONFIG	I/O module configuration data
IO_EIRR	I/O module external interrupt request data
IO_SPA_ADR	I/O module soft physical address range
IO_SESERVED_0	reserved register for future use
IO_RESERVED_1	reserved register for future use
IO_RESERVED_2	reserved register for future use

Status Register



IO status: OFF, ACTIVE, READY, HW_ERR, SW_ERR, module specific bits.

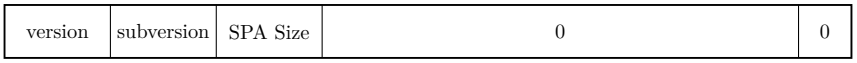
Command Register



Architected command vs. module specific commands.

Room for specifying the HPA address of the target module to interrupt.

Configuration Register

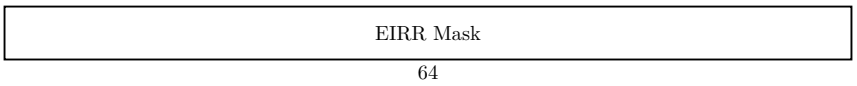


A read only register. SPA size is encoded to cover ranges up to 4GB.

(4k, 16K, 64K, 256K, 1M, 4M, 16M, 64M, 256M, 1G, 4G)

SPA is aligned to size.

External Interrupt Register



A bit for each module. When signaling an interrupt, this ...

Soft Physical Address Registers

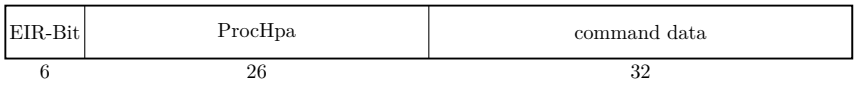
A module is capable to specify two soft physical address ranges.



Interrupt Processing

A processor has two control registers. The external interrupt register and the external interrupt mask register. Both are ANDed and checked after each instruction execution. An I/O module can set a bit in the processor external interrupt request register. There are 64 interrupt groups in the EIRR control register. When not masked by the EIMR register, the interrupt is handled.

When an I/O operation is started, the command contains in addition to the command a 32-bit field, which provides the target processor HPA page address and the bit position to set in the EIRR register of the target processor. This part of the command register is normally set up at system initialization time, but can also be set for each I/O request.



Design Rationale

??? simple but flexible design

??? a rather large address range, forward looking

??? closely modeled after PA-RISC, broadcast is simplified

??? key requirement, is that all modules have a description on board in a ROM.

??? PDC lives at the system level, literally as a ROM on the board ?

Part VI

Firmware Architecture

Firmware Architecture

This chapter presents the Twin-64 firmware architecture overview and concepts

PAL/SAL Address Space

0xFF_F000_0000 - start of PAL code. Reset vector.

0xFF_F000_0004 - PAL function entry point. Jumps to PAL entry function.

0xFF_F000_0008 - reserved.

0xFF_F000_000C - reserved.

PAL/SAL Calling Convention

ARG0 - entry point number.

ARG1, ARg2, ARG3 - arguments.

All also function as RET0 to RET3.

REGn - REGm - caller saves are used locally.

PDC has perhaps a memory area in page zero to use as working memory.

PDC also needs a stack to run on -> in lower memory.

PDC is called privileged, interrupts disabled, uncached.

Processor Abstraction Layer

This chapter presents the Twin-64 processor abstraction layer....

Concepts

Design Rationale

PAL Functions

PAL_TOC - programatic reset

PAL_CHECK - entry point to HPMC

PAL_RESET - hardware entry point, start of PAL code area.

PAL_CHASSIS - writes a value to system chassis display.

PAL_GETC - reads a character from console

PAL_PUTC - writes a character to console

PAL_GET_LINE - reads a line from console

PAL_PUT_LINE - writes a line to console

PAL_CONSOLE - general entry point to console functions. (getc, putc, flus, write line, read line, etc.)

PAL code area layout

first locations are RESET vector, PAL COMMAND vector, two reserved vectors.

RESET vector is where a processor starts after reset.

COMMAND vector is the entry point to PAL functions.

System startup

PAL plays a key role in startup.

- Test processor
- Select monarch. All other processors wait for monarch. The monarch is the processor with the lowest HPA address.
- Monarch issues RESET to all I/O modules.
- Monarch finds memory module with largest size. If there are more than one, the lowest HPA wins.
- Monarch places this module at physical memory address zero.
- Monarch checks source of RESET. (power fail ?)
- Monarch tests memory large enough for ISL image
- Monarch finds CONSOLE module
- Monarch finds boot device
- Monarch loads and launches IPL

Page Zero plays a key role, lots of low level data, such as boot paths, etc. Page Zero also has initial stack ?

Destructive vs. non-destructive memory test ?

System Abstraction Layer

This chapter presents the Twin-64 system dependent code....

Concepts

Design Rationale

SAL Functions

Part VII

Simulator Environment

Twin-64 Simulator

The Twin-64 simulator is an interactive program that runs a configured Twin-64 system. Running the simulator simply means launching the simulator application in a terminal window. During the execution of the configured system, data can be displayed and modified. The simulator features a processor model, system memory and a simulated I/O system. The simulator terminal screen displays system data and allows the monitoring, modifying and tracing of the system execution. The simulator environment is a set of configured modules. A module is a processor, memory or I/O module. The following figure shows a high level view of a configured system.

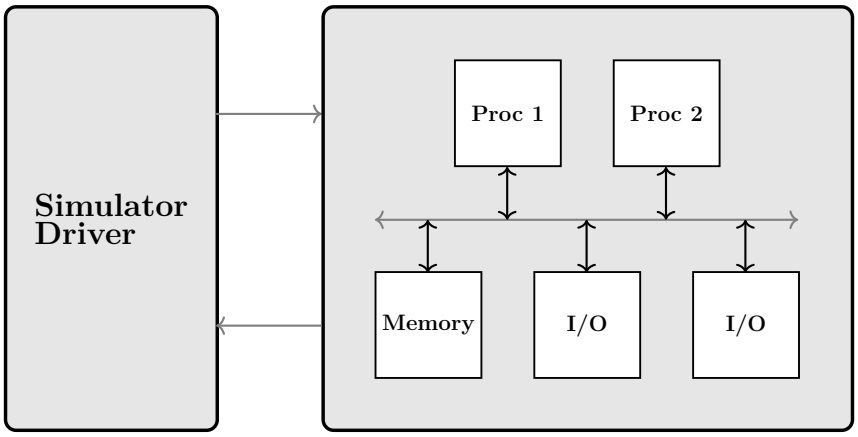


Figure 24: Simulator Environment

The figure above shows a two processor system with a memory module and two I/O modules an a shared **system bus**. The simulator driver issues commands to the simulation environment. Examples are modifying a memory cell content, or single stepping the processors. The simulated environment provides execution data to the driver.

The data is generally shown in **simulator windows**. The main window is the command window. The simulator displays them arranged within a single terminal screen. Each window begins with a banner line shown in inverse video. The banner identifies the window and indicates its current state. The last window on the terminal screen is always the command window.

The simulator normally operates with multiple windows visible, but it can also be placed into **line mode**. In this mode, only the command window is displayed. Commands are entered at the prompt line, and all output is shown sequentially, one line at a time.

Simulator Windows

At the heart of the simulator is the concept of a window to show and modify the simulation environment.

Window

A window consists of a **window banner line** and a **window body**. The banner line begins with the window identification, which includes the window stack number and the window number. The window number is a unique identifier that can be used to refer to a specific window. Beneath the banner line is the window body, which consists of several lines of ASCII text. Both the banner and the body are specific to the window type and are used to display status information, messages, and simulator output.

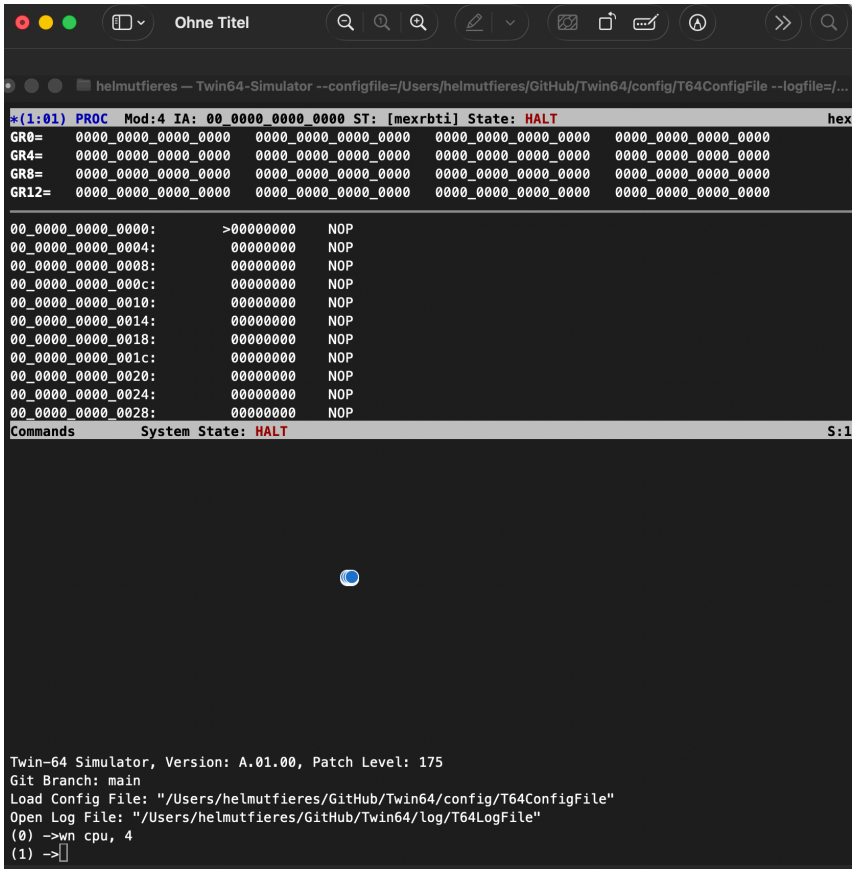


Figure 25: Simulator Windows

The banner line displays general information about the window as well as data specific to its window type. In the example above, the asterisk in the command line indicates that this is the

current window. Commands that accept a window number as an optional parameter operate on the current window when no window is specified.

The window number and window stack are displayed next. The remainder of the banner line is specific to the window type. In the example, the window is a processor window for module 4. The instruction address is 0 and the processor state is `HALT`.

Windows can be organized into stacks. This allows related windows—for example, all windows associated with a single CPU—to be grouped together. A complete stack can be displayed or hidden as a unit.

At all times, exactly one window is designated as the current window. The current window is marked with an asterisk (*) in the banner line. If a window command does not explicitly specify a window identifier, it operates on the current window.

The remainder of the banner line is specific to the window type and is described in the following sections of this manual. At the right edge of the banner line, each window displays the radix format used to present its data. The command window is an exception: its banner line displays the currently defined window stacks at the right edge.

Window Stacks

Simulator windows can be organized into **window stacks**. All visible stacks are displayed horizontally with the terminal screen size adjusted accordingly. The following figure shows a two stack example.

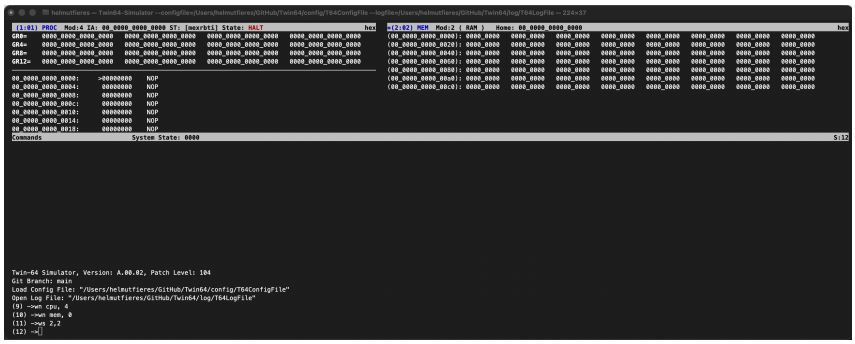


Figure 26: Simulator Window Stacks

Window stack are very useful when handling several processors and memory content. For example, all windows related to a processor can be grouped in a stack. Windows in a stack can be sorted and individual windows can be enabled or disabled. Disabled windows, like disabled stacks, do not disappear for the overall window list. They are just not displayed.

Naturally, a monitor screen cannot display more than two or three stacks next to each other. Stacks, like windows, can therefore be disabled. A disabled stack is not shown. The command window has a little status filed in its banner line which shows what stacks have been created.

Command Window

The **command window** is the primary interface where commands are entered and line-mode information is displayed. This window is always visible, regardless of whether the simulator is operating in windowed mode or line mode. It is always the last window displayed, that is, it appears at the bottom of the terminal screen.

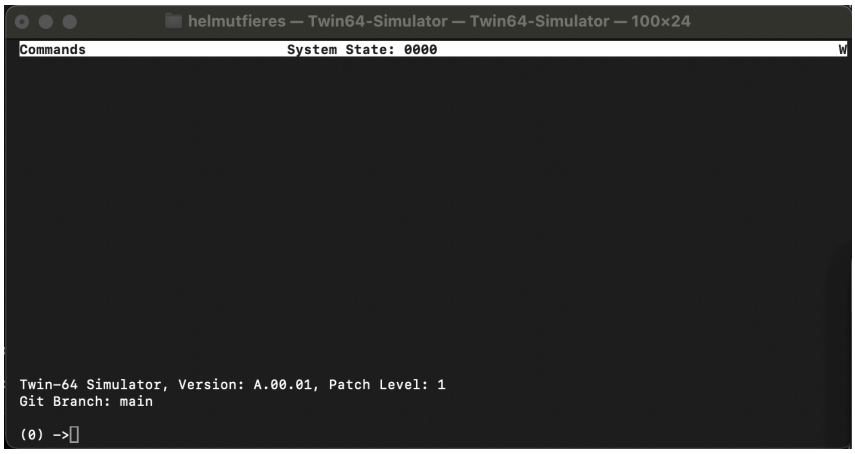


Figure 27: Simulator — Main Window

The command window prompts the user to enter a command. The command number is an incrementing value. Each command entered is assigned a unique number, which can be used to re-execute or edit previously entered commands. Several commands display the command history, showing recent commands along with their associated command numbers. The `DO` command re-executes the specified command, while the `REDO` command retrieves a command from the history and presents it for editing before execution.

Command lines are case insensitive. Internally all input, except those in quotation marks is upshifted before interpretation. Numeric data entered and displayed is shown in decimal or hexadecimal. A decimal number is a numeric string of the number digits 0 to 9. A hexadecimal number has the prefix "0x" and accepts the digits 0 to 9, A to F. Where the context is clear, a hexadecimal number may drop the "0x" prefix on output. A string is a sequence of characters enclosed in quotation marks.

Processor Window

The processor window displays the register set of a processor and the current instruction address code position. Several view modes are available, between which the user can toggle. The default view shows the general register set in the window body. The banner line displays the window identifier, the processor module number, the current instruction address, and the CPU status register. The alternate toggle display the control registers.

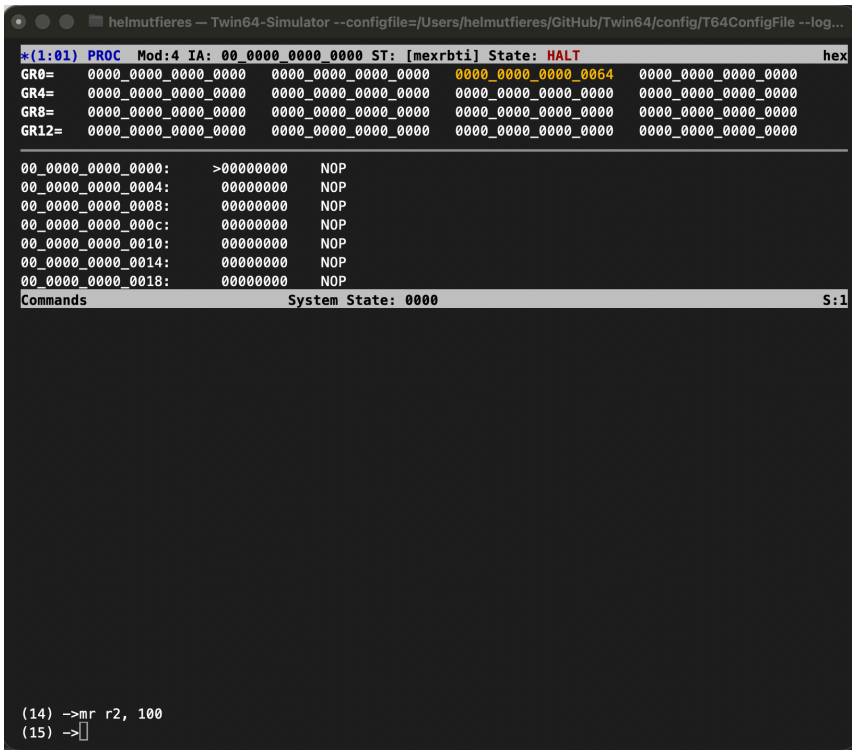


Figure 28: Simulator CPU Window

The window consist of two main parts. The first part shows the processor register set. This part can be toggled to show the general registers, (default), the control registers and a user visible subset of the control registers. The second pat depicts the actual code where the instruction address register points to. This area can be modified to show more than the default lines of 6 lines. A mark shows where exactly the current instruction address is located. Values modified are highlighted until the next command.

As with all simulator windows, multiple instances of the same window type may be created. In the case of the CPU window, all three view modes can be displayed simultaneously by creating multiple windows and toggling each one to the desired view.

TLB Window

The TLB (Translation Lookaside Buffer) window displays the contents of the simulator global TLB module. The TLB accelerates virtual-to-physical address translation by caching recently used page table entries. The global TLB module is the TLB which all processors consult when in need for a translation. The translation result is then kept in local TLB buffers on the processor module. Featuring a global TLB allows all memory commands and windows to use physical and virtual addresses without being dependent on which processor actually has a valid translation.

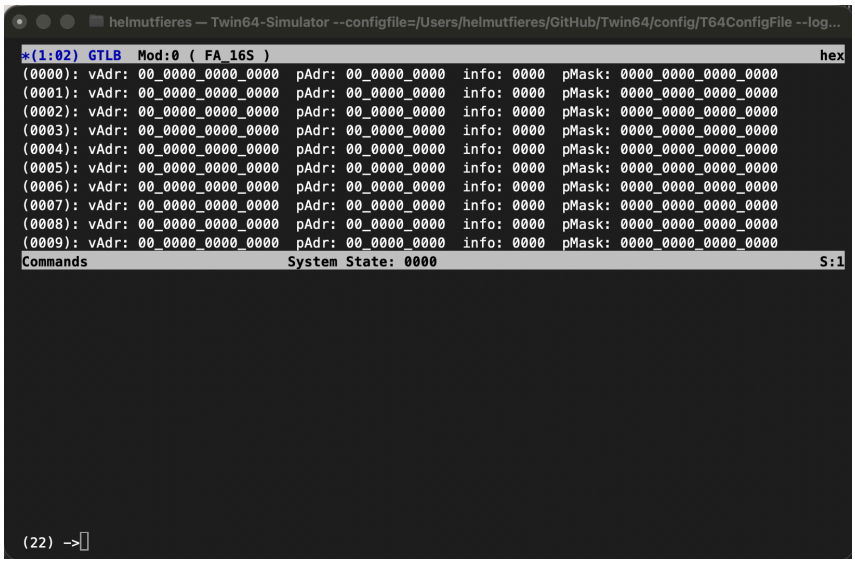


Figure 29: Simulator TLB Window

The global TLB window displays the TLB entries line by line. Each entry includes the virtual page tag, the corresponding physical page number, and the associated tlbInfo. The final field is the tlb page mask when the TLB supports a variable page size.

Memory Window

The memory window displays the contents of physical or virtual memory. The banner line contains the window identifier, the memory module number, and the window's home address. This information allows the user to quickly identify the memory region being examined.

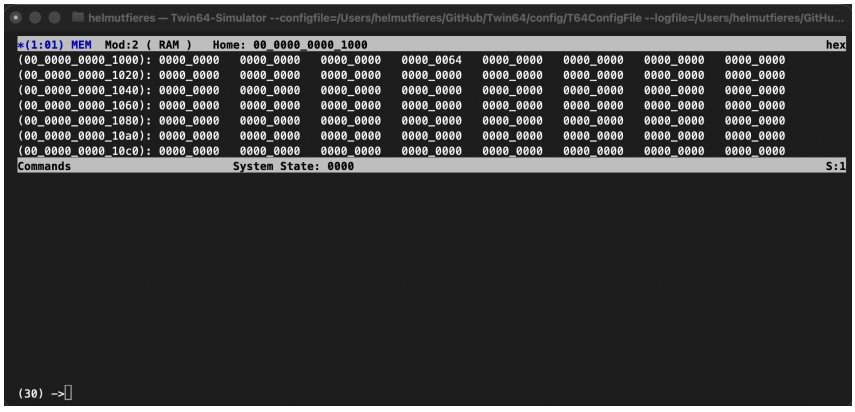


Figure 30: Simulator Memory Window

The memory window provides several display modes that allow memory data to be viewed in different formats, depending on the debugging task at hand. The window can be toggled between these modes while maintaining the same base address and line alignment.

In the 32-bit hexadecimal display mode, memory is shown as a sequence of 32-bit words formatted in hexadecimal notation. This view is useful for inspecting instruction streams and word-oriented data structures.

The 64-bit hexadecimal display mode presents memory as 64-bit quantities. This format is convenient for examining double-word data values and pointer-sized objects. In decimal mode, memory contents are displayed as signed decimal values. This view is intended primarily for examining numeric data produced or consumed by applications.

The ASCII display mode interprets memory bytes as printable characters. Non-printable bytes are shown using a placeholder representation. This mode is useful for inspecting strings, text buffers, and structured data with embedded character fields.

The code window toggle of a memory window displays the contents of memory in a disassembled form, allowing the user to examine instructions as human-readable assembly code. This window helps in debugging by showing the relationship between machine code in memory and its corresponding assembly instructions.

Text Window

The text window displays the contents of a text file. The banner line contains the window identifier, the memory or file module number, and the window's home address, allowing the user to quickly identify the source and location of the displayed text.

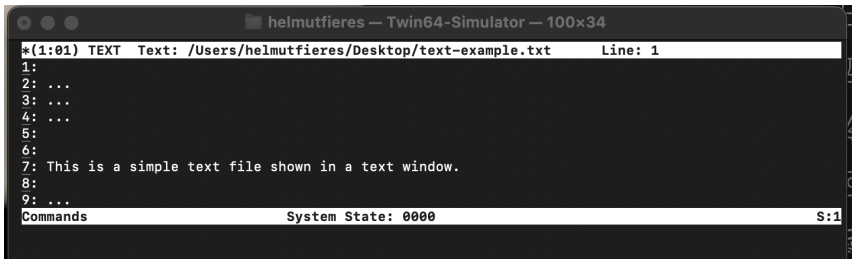


Figure 31: Simulator Text Window

The text window displays the content line by line. It preserves formatting such as line breaks and spaces, making it suitable for inspecting textual data exactly as it would appear in a file or console output.

Simulator User Interface

The simulator is controlled through an interactive command-line interface. Each command is entered at the command prompt and is executed immediately. A command consists of a command name followed by zero or more arguments. Arguments are not limited to literal values; they may be arbitrary expressions containing constants, variables, register values, memory references, and function calls. This chapter introduces the syntax of the command language. The individual commands are described in detail in the following chapter.

Command Line

The simulator displays a command prompt before accepting each command. The prompt contains a command counter which uniquely identifies every command entered during the session.

```
(100) ->
```

A command consists of a command name followed by an optional list of comma-separated arguments.

```
command arg1, arg2, arg3
```

Each argument may be a literal value or an arbitrary expression.

Long commands may be split over several input lines by terminating each continued line with a backslash (\). The simulator concatenates the lines before parsing the command.

```
WRITE R1, \
      R2 + 4, \
      "Hello"
```

The simulator maintains a history of previously entered commands. The `HIST` command displays the command history, while the `DO` and `REDO` commands allow previously entered commands to be executed again or edited before execution.

Data Types

The command language supports three basic data types:

- Numbers. A number is signed 64-bit quantity.
- Booleans. A boolean has the value "true" or "false".
- Strings. A string is character sequence enclosed in apostrophes.

Numbers

All numbers are represented as signed 64-bit integers. Numbers may be entered in decimal, hexadecimal, or binary notation. Hexadecimal numbers use the prefix `0x`, while binary numbers use `0b`.

```
12345
-17

0xFFFF1000
0xFFFF_1000_0010

0b01101001
0b1010_1111_0000
```

Underscore characters may be inserted between digits to improve readability. They do not affect the numerical value.

Booleans

Boolean values represent logical truth and may assume one of the two values

```
true
false
```

Boolean values are typically produced by comparison operators but may also be entered directly.

Strings

Strings are enclosed in double quotation marks.

```
"Hello World"
```

Adjacent string literals are automatically concatenated, allowing long strings to span multiple input lines.

```
"This is a very long "
"string which is "
"assembled automatically."
```

Expressions

Most command arguments are expressions. Expressions are evaluated before the command itself is executed.

Operator	Operand Type	Description
`+`	Number	Addition
`-`	Number	Subtraction
`*`	Number	Multiplication
`/`	Number	Signed division
`==`	Any	Equal comparison
`!=`	Any	Not equal comparison
`<`	Number	Less than
`<=`	Number	Less than or equal
`>`	Number	Greater than
`>=`	Number	Greater than or equal
`&`	Number	Bitwise AND
`|`	Number	Bitwise OR
`^`	Number	Bitwise exclusive OR
`&&`	Boolean	Logical AND
`||`	Boolean	Logical OR
`!`	Boolean	Logical NOT

Parentheses may be used to explicitly control the order of evaluation. Typical expressions are

```
R1 + 4
(R2 + R3) * 8
(R5 & 0xff) == 0x80
(R1 + [0x1000]) / 4
```

Predefined Variables

The simulator provides a collection of predefined variables describing the current machine state. These include the general-purpose registers, control registers, processor status, and other architectural state.

Table 8: Predefined Variables

Name	Type	Purpose
R0 .. R15	numeric	General register.
C0 .. C15	numeric	Control register.
IA	numeric	Instruction address register.

Additional predefined variables may be listed using the `ENV` command.

User Defined Variables

User-defined variables may also be created. Once defined, they may be used wherever expressions are accepted.

```
(8) -> ENV a_var, 200
```

Predefined Functions

The command language provides a number of predefined functions. Functions may appear anywhere within an expression and return values which may be used by the enclosing command.

For example,

```
(9) ->w ASM ("Add r1, r2, r3" )
 0x4410600

(10) ->w ADD_OFS( 0x2000_0000, 0x1000 )
 0x20001000
```

The complete list of predefined functions is given in the command reference.

Memory References

Memory contents may be accessed directly within expressions. The general syntax is

```
"[" [type] address "]"
```

where *address* is any expression evaluating to a memory address and *type* specifies the access size and sign-extension mode.

The supported access types are

BYTE	sign extended 8-bit access
UBYTE	unsigned 8-bit access
HALF	sign extended 16-bit access
UHALF	unsigned 16-bit access
WORD	sign extended 32-bit access
UWORD	unsigned 32-bit access
DOUBLE	signed 64-bit access

If no type is specified, a double-word access is assumed.

Examples (assume the value at 0x1000 is 0xaaaa_bbbb_cccc_dddd:

```
[0x1000] -> 0xaaaa_bbbb_cccc_dddd
[BYTE 0x1000] -> 0xffff_ffff_ffff_ffdd
[UBYTE 0x1000] -> 0xdd
[WORD 0x1000] -> 0xffff_ffff_cccc_dddd
```

The memory address itself can also be an expression:

```
[WORD R1 + 8]
[DOUBLE StackPointer]
```

Memory references may appear wherever expressions are accepted.

Simulator Line Commands

This chapter presents the line mode commands. Although they are called line mode commands, these commands are also available when windows are enabled. The following table is summary of the available commands. Like all commands, they are entered in the command window.

Command Summary

Table 9: Simulator Line Commands

Command	Purpose
ASSERT	ensure condition
CHECK	test condition
DM	display memory
DMOD	display configured Twin-64 modules
DWIN	display window list
DO	execute a command from the command history stack
DW	display windows
ENV	display, add, modify environment variables
EXIT	leave the Twin-64 simulator
HELP	display help information
HIST	display command history stack
ITLB	insert into the unified TLB of a CPU
LOADELF	load ELF file into memory
LOG	write a log entry to the log file
MB	modify memory byte data item (8 bit)
MS	modify memory short data item (16 bit)
MW	modify memory word data item (32 bit)
MD	modify memory double data item (64 bit)
MR	modify a processor register
NMOD	add a module to the simulated environment
PTLB	purge from the unified TLB of a CPU

Continued on next page

Command	Purpose
REDO	edit and do a command from the command history stack
RMOD	remove a module from the simulated environment
RESET	reset a module
RUN	run a configured simulation
STEP	step through a simulation
W	evaluate and display an expression
XF	execute commands from a file

Command Syntax

Command syntax is described using the following conventions. Reserved words are written in uppercase letters. Parameters are written in lowercase letters.

Optional elements are enclosed in square brackets `[ ]`. Elements that may be repeated are enclosed in curly braces . If a square bracket or curly brace is to be entered literally, it is shown enclosed in quotation marks, for example `"["` or `"]"`.

Numeric values are written in decimal notation by default. When expressed in hexadecimal notation, numeric values are prefixed with `0x`. For improved readability, underscore characters may be used as digit separators within hexadecimal values.

String literals are enclosed in double quotation marks (`" "`). Longer strings may be constructed by placing multiple string literals adjacent to each other.

A command line may extend across multiple lines. To continue a command on the next line, place a backslash (\) at the end of the line before the carriage return.

The following sections describe each command in detail, including its purpose, syntax, parameters, and usage examples.

ASSERT - Ensure a condition

Syntax

```
assert <expr> [ , <msgStr> ]
```

Description

The `ASSERT` command evaluates the specified expression and checks whether the result is true or false. If the expression evaluates to true, execution continues normally and no output is generated. If the expression evaluates to false, the assertion fails and an error message is displayed.

An optional message string may be provided to describe the purpose of the assertion or to identify the condition that failed. If the message text is omitted, the default text is taken from the environment variable `ENV_ASSERT_DEF_MSG`.

If a log file is available, the `ASSERT` command also writes its result to the log file, including the evaluation status and any associated message.

Assertions are intended to verify assumptions and detect unexpected program states during execution. If an assertion fails, the program is aborted.

Example

The following examples demonstrate the `ASSERT` command. If the expression evaluates to false, the assertion fails and the specified message string is displayed.

```
(4) -> assert ( 2 < 3 )
(5) ->
(6) -> assert ( 3 < 2 ), "Assert Test 25"
"Assert Test 25" : FAIL
(7) ->
```

Notes

Assertions are primarily intended for debugging and testing. If no message string is specified, a default failure message may be used. The expression is considered successful if it evaluates to a non-zero or true value.

CHECK - Test a condition

Syntax

```
check <expr> [ , <msgStr> ]
```

Description

The `CHECK` command evaluates the specified expression and reports whether the condition passes or fails. Unlike the `ASSERT` command, a failed check does not generate an error or interrupt execution. Instead, the result is reported and execution continues normally.

If the expression evaluates to true, the command reports a `PASS` condition. If the expression evaluates to false, it reports a `FAIL` condition. An optional message string may be provided to describe the purpose of the check or to identify the condition being tested. If the message text is omitted, the default text is taken from the environment variable `ENV_CHECK_DEF_MSG`.

If a log file is available, the `CHECK` command also writes its result to the log file, including the evaluation status and any associated message.

The `CHECK` command is intended for lightweight testing, validation, and interactive diagnostics where failures should be reported without stopping execution.

Example

The following examples demonstrate the `CHECK` command. The first condition passes, while the second condition fails and displays the associated message string.

```
(4) -> check ( 2 < 3 )
PASS
(5) -> check ( 3 < 2 ), "A simple test"
"A simple test" : FAIL
(6) ->
```

Notes

`CHECK` reports failures but does not stop execution. The optional message string can be used to identify individual tests. The expression is considered successful if it evaluates to a non-zero or true value.

DM - Display Memory

Syntax `dm <adr> [ , <len> [ , <fmt> ]]`

Description The `DM` command displays data from physical memory, starting at address `adr` and continuing for `len` bytes. If `len` is omitted, a default length of 8 bytes is used.

For numeric data display, the starting address is rounded down to an 8-byte boundary. For code display, the starting address is rounded down to a 4-byte boundary.

The optional `fmt` parameter selects the output format. Supported formats include hexadecimal (default), decimal, and disassembled instruction words.

Example The following example displays 16 bytes starting at address `0x1000`.

```
(3) ->da 0x1000, 10
00_0000_1000: 0000_0000_0000_0000  0000_0000_0000_0000
(4) ->
```

The `fmt` option can be used to display the data as disassembled instruction words.

```
(4) ->dm 0x1000, 10, code
00_0000_4000: NOP
00_0000_4010: NOP
00_0000_4020: NOP
(5) ->
```

Notes None.

DMOD - List Module Info

Syntax `dmod [ <modNum> ]`

Description The `DM` command displays the configured modules of the Town64 system. Each module is listed with its module number, type, hard physical address (`HPA`), and the optional soft physical address range (`SPA`) and size.

If no module number is specified, all modules are displayed. If a module number is provided, only the selected module is shown.

Example ...

```
(1) ->dmod
Mod  Type   HPA             SPA             Size
00   TLB    0x00_ffe0_0000
01   MEM    0x00_ffe0_1000  0x00_0000_0000  0004_0000
02   MEM    0x00_ffe0_2000  0x00_0004_0000  0004_0000
03   PROC   0x00_ffe0_3000

(2) ->dmod 2
 02   MEM    0x00_ffe0_2000  0x00_0004_0000  0004_0000

(3) ->
```

Notes None.

DO - perform command from history stack

Syntax

```
do [ <cmdNum> ]
```

Description

The `DO` command re-executes a command from the command history stack. The optional command number `cmdNum` selects the entry to be executed.

If no command number is specified, the most recently entered command is re-executed. Command numbers correspond to the indices shown by the `HIST` command.

The `DO` command itself is not aded to the command history stack.

Example

The following example first displays the command history and then re-executes the command with history index `1`.

```
(4) ->hist
[0]: help
[1]: da 0x1000, 16
[2]: help
[3]: da 0x2000, 16
(4) ->do 1
00_0000_1000: 0000_0000_0000_0000  0000_0000_0000_0000
(5) ->
```

Notes

See also the `HIST` command for further information on the command history mechanism.

DWIN - List Window Info

Syntax

```
dwin [ <sNum> ]
```

Description

The DWIN command displays a list of all currently created windows. Each window is shown with its name and type, window ID, assigned stack number, and—if applicable—the associated module number and module type.

Windows are listed regardless of whether they are enabled or disabled. If a window number is specified, only the selected window is displayed.

Specifying an optional stack number restricts the output to windows belonging to that stack.

Example

The following example shows windows created using the WN command. In this example, the module number for the CPU and TLB is 3, and the memory window starts at address 0x1000.

```
(5)  -> wn cpu, 3
(6)  -> wn ITLB, 3
(7)  -> wn MEM, 0x1000
(8)  -> dwin
Name      Stack     Id        WType     Mod       MType
CPU       1         1         CPU       3         PROC
TLB       1         2         TLB       3         PROC
MEM       2         3         Memory    1         MEM
(9)  ->
(9)  -> dw 2
MEM       2         3         Memory    1         MEM
(10) ->
```

Notes

See also the WN command.

ENV - Manage Environment Variables

Syntax `env [ <var> [ , <val> ]]`

Description The `ENV` command manages environment variables. An environment variable is a simple name–value pair. Variable names must be unique and are case-insensitive. Names may contain underscores, but the first character must be alphabetic. ENV variables are stored in upper case, but can entered either way.

Invoking `ENV` with no arguments lists all defined variables. Providing only a variable name displays its value, if the variable exists. Providing both a variable name and a value sets the variable. If the variable does not exist, it is created. Providing a variable name followed by the `-` character removes the variable.

Example

```
(13) -> env
 TRUE                          BOOL:    TRUE
 FALSE                         BOOL:    FALSE
 PROG_VERSION                  STR:     "A.00.01"
 GIT_BRANCH                    STR:     "main"
 PATCH_LEVEL                   NUM:     1
 SHOW_CMD_CNT                  BOOL:    TRUE
 CMD_CNT                       NUM:     14
 ECHO_CMD_INPUT                BOOL:    FALSE
 EXIT_CODE                     NUM:     0
 RDX_DEFAULT                   NUM:     16
 WORDS_PER_LINE                NUM:     8
 WIN_MIN_ROWS                  NUM:     24
 WIN_TEXT_WIDTH                NUM:     90

(14) ->
(14) -> env A_VAR, 100
(15) -> env A_VAR
 A_VAR                         NUM:     100
(15) ->
(16) -> env A_VAR, -
(17) ->
(17) -> env A_VAR
 ENV variable not found
(18) ->
```

Notes See the environment variable section for redefined variables and their initial setting at simulator start.

EXIT - Leave Simulator

Syntax `exit [ <val> ]`

Description The `EXIT` command terminates the simulator. An optional return value may be provided and is passed to the calling environment.

Both `EXIT` and its short form `E` are accepted as command aliases.

Example

```
(52) -> exit 100
```

Notes None.

HELP - list help info

Syntax

```
HELP
HELP <cmdId>
HELP "commands"
HELP "wcommands"
HELP "predefined"
```

Description

The `HELP` command displays help information for the simulator. Help topics are grouped into several categories:

- The `COMMANDS` keyword lists a summary of all available line-mode commands. Entering `HELP` with no arguments also displays this list.
- The `WCOMMANDS` keyword lists a summary of all available window-mode commands.
- The `PREDEFINED` keyword lists all predefined functions that can be used in command-line expressions.

To view detailed syntax for a specific command or predefined function, type `HELP <cmdId>` where *cmdId* is the name of the command or function.

Example

```
(7) ->help help
 help ( cmdId | ‘‘commands | ''wcommands | ‘‘predefined ) - list help
      info
(8) ->

(8) ->help da
da <adr> [ , <len> ] [ , <fmt> ] - display absolute memory
(9) ->
```

Notes

None.

HIST - List Command History

Syntax

`hist [ <depth> ]`

Description

The simulator command interpreter maintains a command history stack. Each command, except for `DO` and `REDO`, is added to this stack.

The `HIST` command lists the contents of the history stack, starting with the oldest entry and ending with the most recent one.

If an optional depth is specified, only the most recent `<depth>` entries are displayed.

Example

The following example shows the command history stack. The `HIST` command itself is not added to the stack.

```
(11) -> hist
[0]: env A_VAR, 100
[1]: env
[2]: env A_VAR
[3]: help ptlb
[4]: wn cpu, 3
[5]: wn tlb, 3
[6]: wf
(7) ->
```

Notes

None.

ITLB - Insert TLB Entry

Syntax `itlb <vAdr> , <info>`

Description The `ITLB` command inserts a translation entry into the unified Translation Lookaside Buffer (TLB). The `vAdr` parameter specifies the virtual address of the mapping, while `info` specifies the corresponding physical memory address and attributes. The `info` parameter directly corresponds to the `ITLB` instruction info fields.

Example

```
# insert a translation for virtual address 0x1000_FFFF_0000
# physical address 0x1_0000 with a
# page size of 4Kb
# access rights of read-write
# user level privilege.

(8)->itlb 0x1000_FFFF_0000, 0x0030_0000_0001_0000
```

Notes None.

LOADELF - load ELF file

Syntax `loadelf "<filePath>"`

Description The `LOADELF` command opens an ELF format file and places the ELF segments at the in the ELF file specified address in memory.

This command is currently under construction and will be finalized when the actual ELF file segments and sections are defined for Twin-64. Currently, it just loads the ELF segments at the specified physical address.

Example

```
Twin-64 Simulator, Version: A.01.00, Patch Level: 126
 Git Branch: main
 Load Config File: "~/config/T64ConfigFile"
 Open Log File: "~/log/T64LogFile"
 (9) ->loadelf "~/Loop-Test.out"
 Loading ~/Loop-Test.out
 Loading: Seg:  0, adr: 0x00005000, mSize: 0x00000004, align: 0
    x00001000, RX
 Loading: Seg:  1, adr: 0x00001000, mSize: 0x0000000c, align: 0
    x00000010, RWX
 Loading: Seg:  2, adr: 0x00002000, mSize: 0x00000020, align: 0
    x00000010, RWX
 Loading: Seg:  3, adr: 0x00003000, mSize: 0x00000024, align: 0
    x00000010, RWX
 Set entry: 0x00000000
 Done
 (10) ->
```

Notes Mid term, this command will be replaced by the initial program load facility IPL.

LOG - Write a log entry to the log file

Syntax

```
log "<logMsg>" [ , <expr> ]
```

Description

The `LOG` command writes an entry to the log file. A log entry consists of a message string `logMsg` and an optional expression value. The expression is evaluated and its resulting value is included in the log output. If no log file is available, the command report an error.

The `LOG` command is intended for recording diagnostic information, execution traces, and test-related output during script execution. It can be used independently, or in combination with the `CHECK` and `ASSERT` commands.

Both `CHECK` and `ASSERT` automatically write their results to the log file when logging is enabled. This includes the evaluation status (`PASS`/`FAIL`) and any associated message string.

Example

The following examples demonstrate the usage of the `LOG` command. The first example logs a simple message, while the second logs a message together with the result of a boolean expression.

```
(4) -> log "a log entry message"
(5) -> log "test a boolean expression", 2 < 3
(6) ->
```

Notes

Log files can be used to record execution traces and test results for debugging and validation purposes. The `CHECK` and `ASSERT` commands automatically extend this log by recording their evaluation results when logging is enabled.

`LOG` does not affect program flow and has no side effects other than producing log output.

Mx - modify memory

Syntax

```
mb <adr> , <expr>
ms <adr> , <expr>
mw <adr> , <expr>
md <adr> , <expr>
```

Description

The `Mx` command modifies a data item in memory. The expression must be a numeric value. There are four commands. The `MB` command modifies a byte at a byte aligned address, the `MS`, `MW` and `MD` command handle a short, word and double word data item at their natural alignment.

Example

```
(100)-> md 0x1000, 0x100      # set memory double at location 0x1000
                              # to the value of 0x100.

(101) -> mb 0x1009, 0xff      # set memory byte at location 0x1000
                              # to the value 0f 0xfF
```

Notes

None.

MR - modify register

Syntax `mr <reg> , <expr>`

Description The `MR` command modifies a register of the CPU in the current CPU window. The command reports an error if the current window is not a CPU window.

The `expr` parameter represents a simple expression. Numeric values can be added, predefined functions used.

Register can also be used in expressions. Their name refers to the current CPU window. If a register from another CPU is to be used, the register name is appended the module number of the processor.

Example

```
(4)->mr R5 100                  # set R5 of the CPU to 100
(5)->

(5)->mr R10 (0x100 + 0x10) * 2  # set R10 to 0x220
(6)->

(7)->mr R15 R7:3                # set R15 of the current CPU
                                # to the value of R7 in CPU 3.
```

Notes None.

NMOD - create a new T64 module

Syntax `NMOD <mType> , <modNum> { , <key>=<val> }*`

Description A Twin-64 system is a set of modules with at least one processor, one memory and one I7O module. The `NM` command adds a module to the Twin-64 simulator module table. The command is intended to be used by a configuration file. A configuration file will simply contains a list of commands to setup a system.

The `module type` parameter defines the module to be added. There are three types, `PROC`, `TLB`, `MEM` and `IO`. The module number parameter assigns the module number. The Twin-64 system supports up to 64 modules at the system level.

The remaining optional parameters pass configuration values for the module. They all are of the form "`name=val`".

Keyword	Value	Usage
SPA_ADR	32-bit address	Soft physical address start. The address must be aligned to the length parameter.
SPA_LEN	32-bit size	Soft physical address length. The size is the pageSize multiplied by a power of 2.
TLB	TLB_FA_xxS	TLB set size. The defined values are 16, 32, 64 and 128.
MEM	RAM | ROM	Memory access type. There are read-/write and read only memories.

Example

```
# add a processor module - 3 - with defaults.
NMOD proc, 3

# add a processor module - 4 - with a larger TLB
NM proc, 4, tlb=TLB_FA_128S

# add a memory module covering the first 256 pages
NM mem, 7, spa_adr=0x0, spa_len=256*0x1000
```

Notes None.

PTLB - Purge TLB entry

Syntax

```
ptlb <vAdr>
ptlb ALL
```

Description

The `PDTLB` command invalidates entries in the unified Translation Lookaside Buffer (TLB). If a virtual address `vAdr` is specified, only the TLB entry matching that virtual address range is purged. The `ALL` parameter will remove all entries from the TLB. The virtual address parameter will remove all TLB entries covering this address.

Example

```
(3)->purge the entire instruction TLB
(3)->ptlb

(4)->purge the instruction TLB entry covering 0x200_1000_0000
(4)->ptlb 0x200_1000_0000
```

Notes

If the simulator environment consists of serval processors, all TLBs will invalidate the specified virtual address range.

REDO - Redo command from history stack

Syntax `REDO [ <cmdId> ]`

Description The `REDO` command recalls a previously executed command from the history stack using its command Id and opens it for editing in the interactive command line.

Editing is performed directly in the command line buffer. Characters can be inserted or removed at the cursor position. Pressing `Enter` submits the modified command for execution.

The `REDO` command is only available in interactive console sessions (TTY). It is not available when input is read from a script file or redirected from a non-interactive source.

Example

```
(11) ->hist
 [0]: rmod all
 [1]: nmod tlb,  0, TLB_FA_16S
 [2]: nmod proc, 4
 [3]: nmod mem,  1, ROM, SPA_ADR=0xff_0000_0000, SPA_LEN=0x4000
 [4]: nmod mem,  2, RAM, SPA_ADR=0x0000_0000, SPA_LEN=0x40_0000
 [5]: reset all
 [6]: env halt_on_traps, true
 [7]: loadelf "~/Loop-Test.out"
 (11) ->redo 2
 nmod proc, 4
```

Notes The command depends on interactive terminal input and is therefore disabled in non-interactive execution modes such as script files or piped input.

RMOD - remove a T64 module

Syntax

```
RMOD <modNum>
RMOD ALL
```

Description

The `RMOD` command removes a T64 module from the Twin-64 system. If there are any windows associated with the module, they will be removed as well. The ALL option removes all modules.

Removing a module should be done with care. There could be interdependencies between modules, and removing one can cause invalid addresses and so on. It is however good practice to issue an remove all command in a module configuration file to ensure a clean state, especially when module configurations are loaded into a simulator that has already loaded a set of modules.

Example

```
RMOD 3              # remove T64 module 3
```

Notes

None.

RESET - Reset T64 modules

Syntax

```
RESET <modNum>
RESET ALL
```

Description

The `RESET` command resets a module or the entire system. The module will execute its defined reset tasks. The result is module specific. A memory module will clear its memory range, an I/O module will reset its internal state, and a processor will clear its internal registers and start at the architected instruction address.

Like all commands dealing with module management, this command should be taken with care. A common use could be to have this command as the last command in a configuration file to start the Twin-64 system.

Example

```
RESET 3              # reset module number three.
```

Notes

None.

RUN - Run the simulation

Syntax

```
RUN [ ]
```

Description

The `RUN` command starts or resumes continuous execution of the simulation. Once invoked, control is transferred to the TWIN-64 system and all configured and enabled processor and I/O modules are allowed to execute.

Unlike the `STEP` command, `RUN` does not specify a finite number of execution units. Execution continues until a system-wide stop condition occurs, such as a breakpoint, a simulator trap, or another condition that returns control to the simulator command interface.

A processor module encountering a breakpoint or a simulator trap causes the entire simulation to enter the HALT state. All modules are then stopped and control returns to the simulator command interface.

Because the modules execute concurrently in separate threads, they do not necessarily execute the same number of instructions or execution units before a system-wide halt occurs. The module may therefore have executed fewer or more units than the module that encounters the stopping condition.

Example

```
(80)->run            # start continuous simulation execution
```

Notes

The `RUN` command honors the enabled code and data breakpoints. When a breakpoint is encountered, the entire simulation is halted and control returns to the simulator command interface.

Trace settings, when enabled, may likewise cause execution to stop according to their configured behavior.

??? under construction: When the `RUN` command is executed, the command window will be connected to the system console and switched back on encountering a breakpoint.

STEP - Step a T64 module

Syntax `STEP [ [ , ] ]`

Description

The `STEP` command advances one or more halted T64 modules by executing one or more execution units. On a processor module, an execution unit is an instruction; on an I/O module, it is one module-specific execution unit.

When the `STEP` command is invoked without an argument, exactly one execution unit is performed on all modules and control then returns to the simulator command interface.

A step count greater than one causes the specified number of execution units to be performed on each selected module. Execution continues until the requested number of steps has been completed or a breakpoint or other trap causes the module to stop.

If the module number is provided and the module is halted, only the specified module executes the requested number of steps.

Unlike the `RUN` command, `STEP` executes a finite and precisely defined number of execution units. For a multi-step operation, the requested count is therefore completed independently for each selected module, unless execution is interrupted by a breakpoint or trap.

After the requested number of steps has completed, or execution has been stopped by a breakpoint or trap, control returns to the simulator command interface. The current state, registers, memory, and other simulator information can then be inspected before execution is resumed with another `STEP` or `RUN` command.

A single-step operation has special breakpoint semantics. When exactly one step is requested, code breakpoints are temporarily disabled for that operation. This allows the instruction at a code breakpoint to be stepped over without requiring an additional step merely to pass the breakpoint. Data breakpoints remain active and may still stop the instruction when it performs a matching data access.

When more than one step is requested, code and data breakpoints are honored normally.

Example

```
(5)->s              # execute one instruction on all modules.
(6)->

(6)->s 10           # execute ten instructions on all modules.
(7)->

(8)->s 10, 5        # execute ten instructions on module 5.
(9)->
```

Notes

A single-step operation ignores code breakpoints, but data breakpoints remain active. A multi-step operation honors both code and data breakpoints.

The `STEP` command always represents a finite execution request. The requested number of execution units is executed on each selected module, unless a breakpoint or other trap interrupts execution.

W - Display an expression value

Syntax `W <expr>`

Description The `W` command evaluates a simulator expression and displays its current value. Expressions can include:

- Registers and control/status registers
- Memory locations (by virtual or physical address)
- Arithmetic operations, logical operations, and constants
- Predefined functions provided by the simulator

The command is primarily used for inspecting system state during simulation or debugging. The evaluated value is printed in the current radix or display format of the active window. If no expression is provided, the command may display a prompt or an error, depending on the simulator configuration.

Example

```
# display the value of general-purpose register R1
(12)-> w R1
0x0000_0000_1234_5678

(13)-> w 0x100, dec
 256

# display the value at virtual memory address 0x1000
(14)-> w [0x1000]
0xDEAD_BEEF

# evaluate an arithmetic expression
(15)-> w R1 + R2 * 4
0x0000_0000_2468_ACF0
```

Notes None.

XF - Execute commands from a file

Syntax

```
XF "<filePath>"
```

Description

The `XF` command opens a text file and executes the simulator commands contained in that file as if they were entered interactively at the simulator command prompt.

Commands are read sequentially from the specified file and processed in the order in which they appear. Each line in the file is interpreted using the normal command parsing rules, including support for comments, expressions, and multi-line commands.

Execution of the command file continues until the end of the file is reached. Errors encountered while executing individual commands are reported, but do not terminate command file execution. Control returns to the simulator only after all commands in the file have been processed.

Command files may invoke additional `XF` commands, allowing command files to be structured hierarchically. Care should be taken to avoid unintended recursive execution.

The `XF` command is commonly used to automate simulator setup, initialize system state, configure windows, or execute repeatable test sequences.

Example

```
# initialize simulator state
xf "init.sim"

# load a test program and start execution
xf "boot_sequence.sim"
```

Notes

At present, command execution continues after errors encountered within a command file. Future versions of the simulator may provide options to alter this behavior, such as aborting execution on the first error.

Simulator Window Commands

Window commands apply to windows only. They feature window creation, arrangement and navigation.

Command Summary

Table 10: Simulator Window Commands

Command	Purpose
CWC	clear command window
CWL	set command window lines
WB	scroll window backward
WC	set window current
WD	disable window
WDEF	set window defaults
WE	enable window
WF	scroll window forward
WH	jump to window home index
WJ	jump to window index
WK	delete window
WL	set window lines
WN	create window
WOFF	disable all windows
WON	enable all windows
WR	set window radix
WS	assign window to stack
WSD	disable window stack
WSE	enable window stack
WT	toggle window display
WX	exchange window

Command Syntax

Command syntax is described using the following conventions. Reserved words are written in uppercase letters. Parameters are written in lowercase letters.

Optional elements are enclosed in square brackets `[ ]`. Elements that may be repeated are enclosed in curly braces `{ }`.

Numeric values are written in decimal notation by default. When expressed in hexadecimal notation, numeric values are prefixed with `0x`. For improved readability, underscore characters may be used as digit separators within hexadecimal values.

String literals are enclosed in double quotation marks (`" "`). Longer strings may be constructed by placing multiple string literals adjacent to each other.

A command line may extend across multiple lines. To continue a command on the next line, place a backslash (\) at the end of the line before the carriage return.

The following sections describe each command in detail, including its purpose, syntax, parameters, and usage examples.

CWC - Clear command window

Syntax `CWC`

Description The `CWC` command clears the command window. All other widows are not affected.

Example

```
cwc                # clears the command window.
```

Notes None.

CWL - Set command window lines

Syntax `cwl [ <lines> ]`

Description The `CWL` command sets the number of content lines displayed in the command window.

Changing this value controls how many lines of command output are visible at once. The command window is resized accordingly to accommodate the specified number of lines. If the parameter is omitted, the simulator default value for the window type will be used.

The cursor up and down keys scroll inside the command window. If the lines argument is omitted, the window default value is used.

Example

```
cwl 10            # Set the command window to display 10 lines.
```

Notes None.

WB - Window backward

Syntax `wb [ <amt> [ , <winNum> ]`

Description The `WB` command scrolls a window backward. The optional `<amt>` parameter specifies the number of items to move backward. The meaning of an item is window-type specific. For example, a physical memory window moves in units of 8 bytes, while a cache window moves in units of cache lines.

If the parameter is omitted, the number of items to move backward is derived from the number of items per line and the number of lines currently displayed in the window. For example, a TLB window displaying three lines moves backward by three TLB entries.

Each scrollable window has an item address limit determined by its window type. When scrolling past this limit, the address is set to the limit minus the number of lines to show.

If the window number parameter is omitted, the command applies to the current window. Otherwise, it applies to the specified window.

Example

```
wn tlb, 0          # Create a tlb window ( TLB is at slot 0 ).
wj 10               # set the window content start to entry 10.
wb 5                # Move backward five items.

wn mem, 0x1000      # Create a memory window.
wb                  # Window moves backward to address 0x0F80.
```

Notes None.

WC - Set current window

Syntax

```
wc <winNum>
```

Description

The `WC` command sets the current window to the specified window number.

Many window-related commands accept an optional window number parameter. When that parameter is omitted, these commands operate on the current window. The `WC` command therefore provides a convenient way to change the target window for subsequent commands without explicitly specifying a window number each time.

If the specified window number does not exist, the command fails and the current window remains unchanged.

Example

```
wn mem, 0x1000          # Create a window (window number 1).
wn mem, 0x2000          # Create another window (window number 2).
                        # Window 2 is the current window.
wc 1                    # Set the current window to window 1.
```

Notes

None.

WD - Window disable

Syntax

```
wd [ <start> [ , <end> ]]
wd ALL
```

Description

The `WD` command disables one or more previously enabled windows.

Disabling a window does not alter any of its attributes. Layout, size, position, and content are preserved from the time it was disabled.

If the window number parameter is omitted, the command applies to the current window. If a range of windows is specified, all windows in the range are enabled. The `ALL` keyword enables all currently disabled windows.

When a window is disabled, the current window selection remains unchanged unless no window is currently active, in which case the next enabled window becomes the current window.

Example

```
wd                  # Disable the current window.
wd 5                # Disable window number 5.

we 5                # Re-enable window number 5.

we 3,7              # enable windows 3 to 7.
```

Notes

None.

WDEF - Set window defaults

Syntax

```
wdef [ <start> [ , <end> ]]
wdef ALL
```

Description

Windows are created with default attributes, such as the number of lines to display or the numeric display radix. The `WDEF` command resets these attributes to their default values for a single window, a range of windows, or all windows.

If no parameter is given, the current window is set to its default. The option `ALL` is used to reset all windows to their default attributes.

Example

```
wdef                # Reset all windows to default attributes.
wdef 3, 7           # Reset windows 3 through 7 defaults.
wdef all                        # set all windows to their defaults.
```

Notes

None.

WE - Enable Window

Syntax

```
we [ <start> [ , <end> ]]
we ALL
```

Description

The `WE` command restores one or more previously disabled windows to the visible list of windows.

Enabling a window does not alter any of its attributes; layout, size, position, and content are preserved from the time it was disabled.

If the window number parameter is omitted, the command applies to the current window. If a range of windows is specified, all windows in the range are enabled. The `ALL` keyword enables all currently disabled windows.

When a window is re-enabled, the current window selection remains unchanged unless no window is currently active, in which case the first enabled window becomes the current window.

Example

```
wd                  # Disable the current window.
we                  # Re-enable the current window.

wd 4                # Disable window 4.
wd 5,10             # Disable windows 5 through 10.
we 5,7              # Re-enable windows 5 through 7.
we ALL              # Re-enable all disabled windows.
```

Notes

See also the `WD` command for disabling windows.

WF - Window forward

Syntax `wf [ <amt> [ , <winNum> ]`

Description The `WF` command scrolls a window forward. The optional `<amt>` parameter specifies the number of items to advance. The meaning of an item is window-type specific. For example, a physical memory window advances in units of 8 bytes, while a TLB window advances in units of TLB entries.

If the `<amt>` parameter is omitted, the number of items to advance is derived from the number of items per line and the number of lines currently displayed in the window. For example, a cache window displaying three lines advances by three cache-line entries.

Each scrollable window has an item address limit determined by its window type. When scrolling past this limit, the address is set to the limit minus the number of lines to show.

If the window number parameter is omitted, the command applies to the current window. Otherwise, it applies to the specified window.

Example

```
wn tlb, 3           # Create a TLB window for module 3.
wf                  # Advance the window by one page.
wf 5                # Advance five items in the current window.

wn mem, 0x1000      # Create a memory window.
wf                  # Window advances to address 0x1080.
```

Notes None.

WH - Window home

Syntax `WH [ <itemAdr> [ , <winNum> ]]`

Description A scrollable window has a home item address. The home item address is window-type specific and is shown in the window banner line.

The `WH` command returns the window's current item address to its home location. This is useful after the window has been scrolled using the `WF` or `WB` commands, or moved away from its home location using the `WJ` command.

The home item address can also be changed by supplying a new item address. The interpretation of the item address is window-type specific.

If the window number parameter is omitted, the command applies to the current window. Otherwise, it applies to the specified window.

Example

```
wn mem, 0x1000      # Create a memory window at address 0x1000.
wj 0x2000           # Move the window to address 0x2000.
wh                  # Return the window to its original location.

wh 0x4000           # Set a new home address.
```

Notes None.

WJ - Window jump

Syntax

```
wj <itemAdr> [ , <winNum> ]
```

Description

The `WJ` command sets the current item address of a scrollable window to a new location. If the window number is omitted, the command applies to the current window.

The interpretation of the item address is window-type specific. For example, in a physical memory window the item address is an 8-byte-aligned memory address, whereas in a TLB window the item address specifies the TLB entry number.

If the window number parameter is omitted, the command applies to the current window. Otherwise, it applies to the specified window.

Example

```
wn tlb, 3           # Create an TLB window for module 3.
wj 10               # Jump to TLB entry 10 in the current window.
```

Notes

None.

WK - Window kill

Syntax

```
wk [ <start> [ , <end> ]]
wk ALL
```

Description

The `WK` command removes one or more windows from the screen. The `<start>` and `<end>` parameters specify a range of window numbers to delete. If omitted, the current window is deleted. Alternatively, the `ALL` keyword removes all windows from the simulator.

Example

```
wk 4                # Deletes window 4.
wk 5, 10            # Deletes windows 5 through 10.
wk ALL              # Deletes all windows.
```

Notes

The command window itself cannot be created or deleted. Also, removing a window has no impact on the T64 modules and their state.

WL - Set window lines

Syntax `wl [ <lines> [ , <winNum> ]`

Description The `WL` command sets the number of lines displayed in a scrollable window. The total number of lines includes the banner line. If the line argument is omitted, the default line count of the window is used.

When the number of lines is increased, the terminal screen is enlarged to accommodate the additional content. Conversely, if the number of lines is reduced, the terminal window shrinks accordingly.

If the optional `<winNum>` parameter is omitted, the command applies to the current window; otherwise, it applies to the specified window.

Example

```
wn mem, 0x1000      # Create a memory window.
wl 20               # Set to 20 lines (19 content lines + banner).
wl 15, 2            # Set window number 2 to display 14 lines.
```

Notes Adjusting window lines may affect the layout of other windows if multiple windows share the terminal screen.

WN - Create a window

Syntax `wn <type> [ , <arg1> [ , <arg2> ]]`

Description

The `WN` command creates a new window. The mandatory `<type>` parameter specifies the type of window to create. Optional arguments provide additional data required for the window creation.

The newly created window becomes the current window and is initialized with default attributes, such as the number of lines, layout, and numeric radix.

Type	Arg 1	Arg 2	Action
`MEM`	`memAdr`	`mode`	Create a memory window with an initial view of type specified in mode. (PROC, TLB, MEM, TEXT)
`PROC`	`modNum`	`-`	Create all PROC-related windows (CPU, TLB) for the module.
`CPU`	`modNum`	`-`	Create a CPU window.
`TLB`	`modNum`	`-`	Create a TLB window.
`TEXT`	`filePath`	`-`	Create a text window displaying the content of a text file.

The window type `PROC` is a collection that includes CPU and TLB windows for the specified module.

The window type `CODE` accepts in addition to the memory address a processor module number. When the module is known, the module instruction address can be compared to the memory window address range and an indicator where the instruction is will be displayed.

Example

```
wn mem, 0x1000       # Create a memory window at address 0x1000.
wn mem, 0x3000, code # create a memory code window at address 0x3000.
wn cpu, 3            # Create a CPU window for module 3.
wn tlb, 4            # Create a TLB window for module 4.
```

Notes

All newly created windows start with default attributes, which can be modified later using commands like `WL` (set window lines) or `WR` (set window radix), if applicable for the window.

WOFF - Turn window display off

Syntax

```
woff
```

Description

The `WOFF` command turns off the display of all windows, leaving only the command window visible.

All windows are retained in their current state, including layout, size, position, and content. No windows are deleted or disabled. They can be restored to the screen using the `WON` command.

Example

```
woff              # Turn off all window displays.
won               # Restore to their previous display state.
```

Notes

See also the `WON` command for restoring windows.

WON - Turn window display on

Syntax

```
won
```

Description

The `WON` command restores the display of all windows that were previously hidden by the `WOFF` command.

All windows are restored to their previous state, including layout, size, position, and content. No windows are created or deleted; the command simply makes them visible again on the screen.

The current window remains the same as it was before `WOFF`, or the first window in the restored list becomes the current window if none was active.

Example

```
woff              # Turn off all window displays.
won               # Restore all windows.
```

Notes

See also the `WOFF` command for turning off window displays.

WR - Set window radix

Syntax

```
wr [ <radix> [ , <winNum> ]]
```

Description

The `WR` command sets the numeric display radix for a window. The optional `<radix>` parameter specifies the desired format:

- `HEX` — Display numbers in hexadecimal format.
- `DEC` — Display numbers in decimal format.

If the `winNum` parameter is omitted, the command applies to the current window. Otherwise, it applies to the specified window.

If the radix parameter is omitted, the command sets the radix to the window default radix.

Example

```
wn mem, 0x1000      # Create a memory window.
wr HEX              # Display numbers in hexadecimal.
wr DEC, 2           # Window number 2 display in decimal.
wr                  # Set current window radix to default.
```

Notes

Changing the radix affects only the display of numeric data. The underlying values remain unchanged. For some window types, certain content is always shown in a fixed radix. For example, addresses in a memory window are always displayed in hexadecimal, while the data values follow the selected radix.

WS - Move windows to stack

Syntax `ws <stackNum> [ , <start> [ , <end> ]`

Description The `WS` command moves one or more windows into a specific stack. A stack is a vertical column of windows. Multiple stacks are arranged horizontally on the screen and are ordered by their stack number.

Within a stack, windows are sorted by their window number. Windows can be enabled, disabled, or hidden independently of their stack placement.

If the window range is omitted, the current window is selected. If just the start window is specified, this window is selected for moving to the stack.

Example

```
wn mem, 0x1000       # Create a memory window, window 1.
wn proc, 3            # Create processor windows.
ws 2, 2, 3            # Move windows 2 and 3 to stack 2.
ws 1, 1               # Move window 1 to stack 1.
```

Notes Stacks allow grouping windows for easier organization on the screen. Windows maintain all their attributes when moved between stacks.

WSD - Disable Window Stack

Syntax

```
wsd <stackNum>
wsd ALL
```

Description

The `WSD` command disables one or more window stacks, removing all windows in the stack from the visible screen. Windows within a disabled stack are not deleted. All attributes, including layout, size, and position, are preserved.

The keyword ALL disables all window stacks. If the stack hosting the current window is disabled, the previous window is set as current window.

Notes

None.

WSE - Enable window stack

Syntax

```
wse <stackNum>
wse ALL
```

Description

The `WSE` command restores a previously disabled window stack to the visible screen. The ALL keyword enables all window stacks.

All windows in the stack are restored with their previous attributes, including layout, size, position, and content.

Example

```
wsd 2              # Disable stack 2.
wse 2              # Re-enable stack 2.
wsd ALL            # Disable all stacks.
wse ALL            # Re-enable all stacks.
```

Notes

See also the `WSD` command for disabling stacks.

WT - Window toggle

Syntax

```
wt [ <toggle> [ , <winNum> ]]
```

Description

The `WT` command changes the display view of a window by selecting one of its available toggle modes.

Each window type may define one or more toggle views. They are numbered from 1 to the defined toggle values for the window type. The current toggle value controls how the window content is interpreted and displayed.

When the `<toggle>` parameter is omitted, the command advances to the next available toggle view. When the maximum toggle value is reached, the toggle wraps around to zero.

When a toggle value is specified, the window jumps directly to that toggle view. Toggle values range from `0` to `maxToggle`, where `maxToggle` is window-type specific.

If the window number parameter is omitted, the command applies to the current window.

Example

```
wn mem, 0x1000      # Create a memory window.

wt                  # Advance to the next display mode.
wt                  # Advance again.

wt 0                # Show toggle view 0 (e.g. hex32).
wt 1                # Show toggle view 1 (e.g. hex64).

wt 2, 4             # Show toggle view 2 of window 4.
```

Notes

The meaning of toggle values is window-type specific. For example, memory windows may use toggles to switch between data formats such as 32-bit hexadecimal, 64-bit hexadecimal, decimal, or ASCII.

WX - Window exchange

Syntax `WX <winNum>`

Description Windows are displayed in order of their window ID. Often, however, the order needs to be changed to display windows in another order.

The `WX` command exchanges the window Id of the current window with the window specified in the command. After the exchange operation the window screen is redrawn.

Example

```
wn mem, 0x1000     # create mem window, window 1
wn mem, 0x2000     # create mem window, window 2 is current
wx 1               # exchange window 2 with 1
```

Notes None.

Simulator Breakpoints

The simulator provides a breakpoint facility that is independent of the breakpoint or trap instructions defined by the processor architecture. Its purpose is to observe the execution of the simulated machine and to stop the machine when a specified condition occurs. This facility is particularly useful when debugging the interaction between processor modules, I/O modules, and the system memory.

Motivation

A simulator is most useful when it provides more than simply the ability to execute a program. When something goes wrong during execution, it must also provide a way to stop the simulated machine at a useful point and inspect its state.

Simulator breakpoints are a facility of the simulated machine and are invisible to the program running on it. A breakpoint can be associated with a code address, causing the simulator to stop before an instruction at that address is executed. A breakpoint can also be associated with a data address or address range, causing the simulator to stop when a processor accesses that range.

System-Wide Breakpoints

The simulator normally runs several modules concurrently. In particular, a system may contain multiple processor modules, together with I/O modules and other system components. A breakpoint therefore cannot simply stop the processor that happens to encounter it.

When a processor encounters a simulator breakpoint, the simulator changes the state of the complete simulated system to `HALT`. All processor modules consequently stop executing. Control is then returned to the simulator command interpreter, where the user can inspect the system state and issue further commands.

The breakpoint facility is therefore associated with the system rather than with an individual processor. A breakpoint can specify which processor modules are capable of triggering it. This makes it possible, for example, to stop when a particular processor reaches an address while allowing the same address to be executed normally by another processor.

A breakpoint can also be defined for all processor modules. This is useful when debugging multiprocessor code where the processor reaching a particular address is not known in advance.

Instruction Breakpoints

An instruction breakpoint identifies an address at which execution is to stop. Processor modules check for an applicable simulator breakpoint before fetching the next instruction. If the current program counter matches an enabled instruction breakpoint, the simulator does not perform the instruction fetch. Instead, the system enters the `HALT` state and control is returned to the simulator.

No instruction in memory has to be modified. In particular, the simulator does not insert a breakpoint instruction into the program. The program image therefore remains unchanged. This is also important for debugging code that cannot easily be modified, such as read-only code, firmware, or code shared by several processors.

Data Breakpoints

The Twin64 simulator breakpoint can also monitor data accesses. Data breakpoints may be configured for read access, write access, or both.

Unlike an instruction breakpoint, a data breakpoint does not refer to the program counter. It refers to the address involved in a memory operation. When a processor executes an instruction that performs a memory access, the simulator checks whether the access matches an applicable data breakpoint. If it does, the instruction is not allowed to complete normally. The simulated system enters the `HALT` state and control is returned to the simulator.

Data breakpoints are particularly useful when the location of an unexpected change is known, but the instruction responsible for the change is not. For example, a breakpoint can be placed on a memory location containing a variable suspected of being overwritten. Execution then stops at the instruction that performs the write, allowing the processor state and the instruction responsible for the access to be examined.

Data breakpoints can also be configured for address ranges. This is useful for monitoring structures such as buffers, tables, device register areas, or other regions of memory without having to create a separate breakpoint for every address.

The address range is represented efficiently by an aligned address and an address mask. The breakpoint address identifies the beginning of the range and the mask identifies the address bits that participate in the comparison. Consequently, ranges whose size is a power of two can be tested with a single masked address comparison.

Breakpoint State

Each simulator breakpoint can be enabled or disabled independently. Disabling a breakpoint leaves its definition in the breakpoint table but prevents it from causing the simulator to halt. This is useful when a breakpoint is temporarily not required but may be needed again later.

A breakpoint that has been hit does not itself execute an architectural instruction or generate an operating-system exception. Instead, it is a simulator event. The simulator changes the system state to `HALT` and transfers control to the simulator command interpreter.

Once the machine is halted, individual processor modules can be single-stepped. This is useful for examining the machine's behavior instruction by instruction after a breakpoint has been encountered.

Breakpoint Representation

The simulator maintains a system-wide breakpoint table. Each entry describes the type of breakpoint, its enabled state, the address range, and the processor modules for which the breakpoint is active.

The address and address mask form the range definition. The module mask contains one bit for each module in the system. A set bit indicates that the corresponding module is capable of triggering the breakpoint. The command interface uses a module number when creating or removing a breakpoint. A special module number of -1 denotes all modules. This allows the user to create a system-wide breakpoint without having to construct a module mask explicitly.

If several modules request a breakpoint for the same type and address range, they share a single breakpoint table entry. The corresponding module bits are combined in the entry's module mask. When a breakpoint is removed for one module, only that module's bit is cleared. The breakpoint entry itself is removed when no module remains associated with it.

This representation separates the definition of the breakpoint condition from the modules that are interested in that condition. It is particularly appropriate for a multiprocessor simulator, where several processors may independently need to observe the same address.

Debugging Workflow

A typical debugging session therefore proceeds as follows:

- The simulator is running the simulated machine.
- The user creates one or more simulator breakpoints.
- Processor modules check for applicable instruction breakpoints before fetching instructions and data breakpoints when performing memory accesses.
- When a breakpoint condition is encountered, the complete simulated system enters the `HALT` state.
- Control returns to the simulator command interpreter.
- The user examines registers, memory, and other machine state.
- Individual processor modules can be single-stepped when required.
- The user resumes the system when the desired state has been examined.

The important property of this mechanism is that it does not require the software being debugged to cooperate with the simulator. The breakpoint is part of the simulator's observation of the machine and can therefore be used to debug both application software and the interaction between the simulated hardware components themselves.

Command Summary

The simulator breakpoints are accessed using the following five commands, which allow for creation, removal, enabling and disabling. Finally, a breakpoint list command gives an overview of the currently defined breakpoints.

Command	Purpose
BD	disable a simulator breakpoint
BE	enable a simulator breakpoint
BK	remove a simulator breakpoint
BL	list simulator breakpoints
BN	create a simulator breakpoint

The following sections describe each command in detail, including its purpose, syntax, parameters, and usage examples.

BD - Disable simulator breakpoint

Syntax `bd <bNum>`

Description The `BD` command disables a simulator breakpoint. The breakpoint remains in the breakpoint table and can be enabled again using the `BE` command.

The `bNum` parameter specifies the breakpoint number as shown by the `BL` command. Disabling a breakpoint does not change its address, type, or module association.

Example The following examples demonstrate the `BD` command.

```
(9) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
2    CODE   E       0x00_0000_2000       0000_0000_0000_0030
3    DATA   E       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   E       0x00_0000_3100:0100  0000_0000_0000_0020
(10) ->bd 2
(11) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
3    DATA   D       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   E       0x00_0000_3100:0100  0000_0000_0000_0020
(12) ->
```

Notes A disabled breakpoint remains available for later use. Use the `BE` command to enable it again.

BE - Enable simulator breakpoint

Syntax

```
be <bNum>
```

Description

The BE command enables a simulator breakpoint. The breakpoint must have previously been created with the BN command and remains in the breakpoint table after being disabled.

The bNum parameter specifies the breakpoint number as shown by the BL command. Enabling a breakpoint activates its configured address, type, and module association.

Example

The following examples demonstrate the BE command.

```
(9) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
3    DATA   E       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   D       0x00_0000_3100:0100  0000_0000_0000_0020
(10) ->be 4
(11) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
3    DATA   E       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   E       0x00_0000_3100:0100  0000_0000_0000_0020
(12) ->
```

Notes

Enabling a breakpoint does not change its configuration. Use the BD command to disable it again.

BK - Remove simulator breakpoint

Syntax `bk <bNum> [ , <modNum> ]`

Description The `BK` command removes a simulator breakpoint.

The `bNum` parameter specifies the breakpoint number as shown by the `BL` command. The optional `modNum` parameter specifies the module from which the breakpoint is to be removed. A module number of `-1` removes the breakpoint from all modules.

If the breakpoint is associated with more than one module, removing it from one module does not remove the breakpoint itself. The module is instead removed from the breakpoint's module mask. The breakpoint is removed from the breakpoint table only when no modules remain associated with it.

Example The following examples demonstrate the `BK` command.

```
(6) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
2    CODE   E       0x00_0000_2000       0000_0000_0000_0010
3    DATA   E       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   E       0x00_0000_3100:0100  0000_0000_0000_0020
(7) ->bk 1
(8) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
3    DATA   E       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   E       0x00_0000_3100:0100  0000_0000_0000_0020
(9) ->
```

Notes The breakpoint number of removed breakpoint is re-used for the next breakpoint create.

BL - List simulator breakpoints

Syntax `bl`

Description The `BL` command displays the currently defined breakpoints. A breakpoint has a number assigned at creation, the state, i.e. whether enabled or disabled, the address and for data breakpoints, the length. The length is appended to address separated by a colon.

The Modules column shows for which module the breakpoint is defined. It is a 64-bit mask, where each bit represents a module. Module 0 is the leftmost bit. For example, the value `0000_0000_0000_0010` represents module 4, a value of `0000_0000_0000_0018` represents module 4 and module 3.

Example The following examples demonstrate the `BE` command.

```
(6) ->bl
Idx  Type   State   Adr                  Modules
1    CODE   E       0x00_0000_1000       0000_0000_0000_0010
2    CODE   E       0x00_0000_2000       0000_0000_0000_0030
3    DATA   E       0x00_0000_3000:0004  0000_0000_0000_0010
4    DATA   E       0x00_0000_3100:0100  0000_0000_0000_0020
(7) ->
```

Notes None.

BN - Create a simulator breakpoint

Syntax `bn <modNum>, <type>, <adr> [ , <len> ]`

Description The `BN` command creates a simulator breakpoint.

The `modNum` parameter specifies the module for which the breakpoint is active. A module number of `-1` specifies all modules.

The `type` parameter specifies the type of breakpoint. The following types are supported:

- `CODE` - Break when execution reaches the specified address.
- `DATA` - Break on any data access to the specified address or range.
- `DATA_R` - Break on a read access to the specified address or range.
- `DATA_W` - Break on a write access to the specified address or range.

The `adr` parameter specifies the breakpoint address. For data breakpoints, the optional `len` parameter specifies the length of the address range. Data ranges of data breakpoint cannot overlap. Code breakpoints do not support address ranges.

If a breakpoint with the same type and address range already exists for another module, no new breakpoint entry is created. Instead, the specified module is added to the existing breakpoint's module mask. Newly created breakpoints are enabled automatically.

Example The following examples demonstrate the `BN` command.

```
(0) ->bn 4, code, 0x1000
(1) ->bn 4, code, 0x2000
(2) ->bn 4, data_r, 0x3000
(3) ->bl
Idx  Type   State   Adr                   Modules
1    CODE   E       0x00_0000_1000        0000_0000_0000_0010
2    CODE   E       0x00_0000_2000        0000_0000_0000_0010
3    DATA_R E       0x00_0000_3000:0004   0000_0000_0000_0010
(4) ->
```

Notes Data breakpoint ranges are defined by their starting address and length. The address is aligned to the corresponding range boundary.

Simulator Predefined Functions

The Twin64 simulator offers a set of predefined functions, which can be used in command expressions.

Table 12: Predefined Simulator Functions

Name	Type	Argument(s)	Purpose
asm	numeric	string	assembles an instruction
disasm	string	numeric	disassembles an instruction

ASM — Inline assembler

Syntax

```
asm ( argStr )
```

Description

The `ASM` predefined function assembles a single assembly instruction and returns its encoded machine instruction as a numeric value.

Parameters

Name	Type	Description
`argStr`	`string`	The assembly instruction to assemble, provided as a single line of text.

Return Value

Name	Type	Description
`instr`	`u32`	The assembled instruction encoded as a 32-bit unsigned integer.

Example

```
(4) -> w asm("LDO r1, 104(r5)")
0x2c42e00c
(5) ->
```

Notes

The instruction is assembled using the currently selected target architecture and assembler settings.

DISASM — Disassemble instruction

Syntax

```
disasm ( instr )
```

Description

The `DISASM` predefined function disassembles a single machine instruction and returns its assembly representation as a string.

Parameters

Name	Type	Description
`instr`	`u32`	The machine instruction to disassemble, provided as a 32-bit unsigned integer.

Return Value

Name	Type	Description
`asmStr`	`string`	The disassembled instruction as a single-line assembly string.

Example

```
(6) -> w disasm(0x2c42e00c)
"LDO r1, 100(r5)"
(7) ->
```

Notes

The instruction is disassembled using the currently selected target architecture and disassembler settings.

Simulator Variables

The simulator provides two categories of variables: built-in variables and environment variables.

Built-in variables represent objects that are part of the simulated system, such as CPU registers, status flags, or memory locations. Their values are determined directly by the execution of the simulated program and cannot be assigned by the user.

Environment variables are maintained by the simulator itself. They are used to provide information about the simulation environment rather than the simulated hardware. For example, the current command sequence number is stored in a predefined environment variable whose value is updated automatically by the simulator.

In addition to the predefined environment variables, users may define their own environment variables. User-defined environment variables behave in the same way as predefined ones, except that their values are assigned explicitly by the user.

All environment variables have global scope and are accessible from any command or script.

Predefined Variables

??? better word for these puppies...

Table 13: Predefined Simulator Variables

Name	Type	Access	Purpose
R0 .. R15	numeric	R/W	General register.
C0 .. C15	numeric	R/W	Control register.
IA	numeric	R/W	Instruction address register.

Environment Variables

Table 14: Environment Variables

Name	Type	Access	Purpose
R0 .. R15	numeric	R/W	General register.

Continued on next page

Name	Type	Access	Purpose
C0 .. C15	numeric	R/W	Control register.
IA	numeric	R/W	Instruction address register.

Simulator Configuration

The simulator can be configured using a set of global options that control its startup behavior and runtime environment. Configuration information can be supplied through multiple mechanisms:

- Program command-line parameters
- Environment variables
- Simulator commands executed during initialization

These mechanisms work together to define the initial simulator state before interactive use begins.

Program Parameters

Program parameters are specified on the simulator command line and control basic startup behavior, such as output verbosity, configuration file selection, and version or help display.

The following table lists the supported command-line parameters.

Name	Type	Usage
`help`	No argument	Displays a summary of available command-line options and exits.
`version`	No argument	Displays the simulator version information and exits.
`verbose`	No argument	Enables verbose output during simulator initialization and execution.
`config`	Required argument	Specifies the path to a configuration file containing simulator commands to be executed at startup.
`log`	Required argument	Specifies the path to the log file used for runtime logging.

Table 15: Simulator command-line options

Configuration Files

In addition to command-line parameters, the simulator can be configured using a configuration file specified with the `--configfile` program option.

A configuration file contains a sequence of simulator commands that are executed automatically after the initial program setup is complete. This mechanism allows complex simulator setups to be reproduced reliably and consistently.

Configuration files are typically used to:

- Set environment variables
- Create and initialize simulator windows
- Instantiate Twin-64 system modules
- Establish an initial debugging or inspection layout

After all commands in the configuration file have been executed, control is passed to the interactive command interface.

Log Files

// ??? to do ...

Configuration Example

// ??? to do ...

Part VIII

Appendix

Appendix - Instruction Commentary

This appendix presents extended commentary on selected instruction groups. It provides further explanation, design rationale, and hardware-level insights that complement the main instruction descriptions. Each subsection discusses the motivation, encoding, and implementation strategy for one or more related instructions, offering a deeper view into the processor's architectural design.

Arithmetic Operation Instructions

ADD, SUB, SHLxA, SHRxA

Boolean Operation Instructions

The Twin-64 architecture supports the standard Boolean instructions: `AND`, `OR`, and `XOR`. These instructions allow complementing the second operand as well as the output, enabling flexible masking and bit-field manipulation.

For example, one can apply a mask directly or use its complement to isolate or clear specific bits in a register efficiently.

When a Boolean instruction uses an immediate value as one operand, there are two possible interpretations for the immediate field:

- **Unsigned zero-extended:** the immediate is treated as a positive value and zero-extended to the machine word size. This is convenient for constructing masks affecting only low-order bits.
- **Signed sign-extended:** the immediate is interpreted as a signed value and extended with its sign bit. This allows the use of negative constants, which produce masks with high-order bits set.

Twin-64 adopts *sign-extension* for logical immediate values. The primary motivation is uniformity: the logical I-type instructions share the same immediate decoding path as arithmetic instructions such as `ADDI`, avoiding additional decode logic.

Sign-extension also enables several useful bit-manipulation idioms without requiring extra instructions. Examples include:

- `AND Rn, Rm, -2` produces the mask `0x…FFFE`, which clears the least significant bit.
- `AND Rn, Rm, -4096` yields `0x…F800`, aligning an address to a 4 KiB boundary.

- `XOR Rn, Rm, -1` results in `0x…FFFF`, effectively computing a bitwise NOT in a single instruction.
- `OR Rn, Rm, -256` creates `0x…FF00`, forcing the upper byte of a register to ones.

Using sign-extension for logical immediate values therefore provides both a straightforward hardware implementation and a flexible set of common masking and bit-twiddling operations.

Bit Operation Instructions

Twin-64 provides specialized bit-manipulation instructions such as `EXTR`, `DEP`, and `DSR` to support shifts, rotates, and other fine-grained operations on register fields.

These instructions are commonly used for:

- Extracting a subset of bits from a register (`EXTR`).
- Depositing a value into a specific bit field (`DEP`).
- Performing data shifts and rotations (`DSR`), which can emulate traditional shift/rotate instructions without dedicated opcodes.

For example, the `EXTR` instruction can be combined with `AND` masks to isolate or manipulate specific bits efficiently. This approach provides the functionality of classic shift and rotate instructions, while maintaining a compact instruction set.

Comparison and Branch Condition Encoding

This appendix explains the condition encoding and implementation philosophy used by the `CMP`, `CBR`, `ABR`, and `MBR` instructions. The design goal is to minimize hardware complexity by reusing a single subtraction/comparator path. Comparison operators are encoded in a compact and symmetric way that makes condition inversion trivial.

The following instructions use the same 3-bit comparison field (`cond`):

- **CMP** — Compares two operands using the 3-bit `cond` field and produces a boolean result (0/1) in a register.
- **CBR** — Compares two registers and branches based on the same 3-bit `cond` field. This performs a direct comparison without requiring a preceding `CMP`.
- **ABR** — Performs an add operation and branches using a 3-bit `cond` field that tests the *add result* against zero or other simple predicates. Intended for loop counters or pointer walking.
- **MBR** — Performs a move operation and branches using the same 3-bit `cond` field. The comparison is applied to the source operand, typically against zero.

All four instructions share a uniform 3-bit condition encoding, allowing common decoder and comparator logic. Branch target computation uses a sign-extended 15-bit offset shifted left by two, allowing word-aligned branch destinations within a ±64 KB range relative to the current instruction address.

Design Rationale The condition encoding is designed for symmetry and minimal logic cost. Conditions are arranged in complementary pairs so that flipping a single bit yields the inverse relation. This symmetry makes branch inversion trivial in hardware and simplifies assembler implementation.

Value	Binary	Mnemonic	Meaning / Branch if
0	000	EQ	Equal (==)
1	001	LT	Less than (<)
2	010	GT	Greater than (>)
3	011	EV	Even (bit 0 = 0)
4	100	NE	Not equal (!=)
5	101	GE	Greater than or equal (>=)
6	110	LE	Less than or equal (<=)
7	111	OD	Odd (bit 0 = 1)

The most significant bit (cond[2]) acts as an *invert bit*: flipping it inverts the sense of the comparison. The relationships between base and inverted conditions are as follows:

Base Condition	Inverted Condition	Encoding Pair
EQ	NE	000 / 100
LT	GE	001 / 101
GT	LE	010 / 110
EV	OD	011 / 111

Since Twin-64 consistently works on signed d64-but quantities, all comparisons are signed by default. The EV/OD conditions allow efficient branching on bit 0, which is useful for address alignment tests, distinguishing even/odd indices, and loop unrolling.

Control Flow Instructions

BB, B, BR, BV, BE

CBR, ABR, MBR

Immediate Instructions

ADDIL, LDIL, LDO

??? there are also computational instructions that allow for immediate values...

Pseudo-OpCode Load Immediate

Large immediate values do not fit into an instruction word. The approach is to emit a combination of `LDI` and `LDO` instructions. The `LDO` instruction offers a scaled offset, which can be used to optimize for the cases where the value is aligned to what the scale field offers. The following table depicts the instruction sequences to emit for immediate values.

Table 16: Load Immediate Pseudo OpCode

Reach	**Aligned**	**Non-Aligned**
13bit (+/- 4Kb)	same as non-aligned	LDO.B Rn, <val>(R0)
14bit (+/- 8Kb)	LDO.S Rn, <val>(R0)	LDI.L Rn, L%<val>(Rn) LDO.B Rn, R%<val>(Rn)
15bit (+/- 16Kb)	LDO.W Rn, <val>(R0)	LDI.L Rn, L%<val>(Rn) LDO.B Rn, R%<val>(Rn)
16bit (+/- 32Kb)	LDO.D Rn, <val>(R0)	LDI.L Rn, L%<val>(Rn) LDO.B Rn, R%<val>(Rn)
32bit	same as non-aligned	LDI.L Rn, L%<val>(Rn) LDO.B Rn, R%<val>(Rn)
52bit	same as non-aligned	LDI.L Rn, L%<val>(Rn) LDI.R Rn, M%<val>(Rn) LDO.B Rn, R%<val>(Rn)
64bit	same as non-aligned	LDI.L Rn, L%<val>(Rn) LDI.M Rn, M%<val>(Rn) LDI.U Rn, U%<val>(Rn) LDO.B Rn, R%<val>(Rn)

Memory Reference Instructions

Twin-64 provides conventional load and store instructions together with a small, carefully chosen set of addressing modes. In addition, the computational instructions `ADD`, `SUB`, `AND`, `OR`, `XOR`, and `CMP` may use these addressing modes to obtain their second operand. This reflects the register-memory character of the architecture.

Twin-64 supports two fundamental and orthogonal addressing modes for data access:

1. Base register plus immediate offset, and
2. Base register plus index register.

These modes form the basis of all load/store operations and all computational instructions that reference memory for their second operand.

Scaled Indexing and Implicit Alignment When the index-register mode is used, the index value is automatically scaled according to the size of the referenced data item. For example, a half-word access shifts the index left by one bit, while a word access shifts it left by two bits. This scaling increases the effective range of the index and ensures that the resulting address is aligned to the natural boundary of the accessed data. The effective address is computed as the sum of the base register and the scaled index.

Signed Data Semantics All data fetched by load instructions or by memory-operand computational instructions is interpreted as a signed quantity. Bytes, half-words, and words are sign-extended to the full 64-bit register width during access. This design aligns with the Twin-64 arithmetic model, simplifies the ALU and trap semantics, and avoids the complexity of maintaining separate signed and unsigned data paths.

Absence of Pre/Post-Increment Addressing Some historical architectures provided pre-increment and post-increment addressing modes, in which the effective address is both computed and written back to the base register. Twin-64 intentionally omits these modes.

Implicit updates require additional register-file write ports or multi-cycle update paths, increasing data path and hazard complexity. They also introduce subtle ordering and forwarding constraints that complicate pipeline timing. Since modern compilers seldom use such addressing forms, their practical value does not justify the associated hardware and verification cost. Their omission helps preserve the simplicity and implementation efficiency of Twin-64.

Pseudo-OpCode Load/Store with large offset

The load and store instructions offer a scaled offset field. This field is shifted by the data length for aligned access. This increases the reach for short, word and double word access. For larger offsets than what the scaled offset can encode, a combination of the `ADDIL` and the load and store instructions is required.

Table 17: Load Immediate Pseudo OpCode

Reach	In reach	Large than reach
Byte	LD.B Rn, <ofs>(Rb)	ADDIL Rb, L%<ofs> LD.B Rn, R%<ofs>(R1)
Short	LD.S Rn, <ofs>(Rb)	ADDIL Rb, L%<ofs> LD.S Rn, R%<ofs>(R1)
Word	LD.W Rn, <ofs>(Rb)	ADDIL Rb, L%<ofs> LD.W Rn, R%<ofs>(R1)
Double	LD.D Rn, <ofs>(Rb)	ADDIL Rb, L%<ofs> LD.D Rn, R%<ofs>(R1)

Load-Reserved and Store-Conditional Instructions

The `LDR` (Load Reserved) and `STC` (Store Conditional) instructions provide the architectural mechanism for implementing atomic read–-modify-–write sequences without requiring explicit locks. Together, they form the foundation for synchronization primitives such as semaphores, spinlocks, and atomic counters.

- **LDR** — loads a 64-bit value from memory and establishes a *reservation* on the cache line containing that address. The effective address is computed as the sum of the base register and the signed offset and must be 8-byte aligned.
- **STC** — attempts to store the value in `Rn` to the same memory location, succeeding only if the reservation remains valid. If the reservation is valid, `STC` completes normally and returns a success indication (typically a value of 1). If the reservation has been lost, the store is suppressed and a failure value (0) is returned to the destination register.

Design Rationale The `LDR`/`STC` pair allows atomic sequences to be expressed entirely in software, avoiding pipeline stalls and bus locks. The mechanism relies on a single internal reservation register that tracks the address (or cache line) of the most recent `LDR`. Only one reservation may be active per processor at any time. Using the cache system as the underlying reservation tracker reduces hardware cost and ensures natural invalidation under normal cache--coherency events. A reservation is cleared automatically under any of the following conditions:

- The cache line containing the reserved address is invalidated or evicted through the local or other processors.
- The local processor performs a write to the same cache line through any path other than `STC`.
- An interrupt, trap, or context switch occurs.
- A subsequent `LDR` instruction establishes a new reservation.

This model guarantees that at most one `STC` can succeed per `LDR`, ensuring forward progress while maintaining strong atomicity at the cache-line granularity.

Implementation Notes The reservation register may store the cache line tag rather than the full address, allowing fast comparison against cache operations. Integration with the cache controller ensures that any coherence or invalidation event automatically clears the reservation flag. Because interrupts also clear reservations, interrupt latency and atomicity properties remain predictable.

This design matches the behavior of similar primitives in other architectures (e.g., RISC-V `LR/SC` or PowerPC `lwarx/stwcx.`) but uses a cache-line–based validity rule rather than a fixed address match, simplifying coherence handling in multiprocessor systems.

Atomic Increment Example The following example implements an atomic increment function. The memory location is specified by base register GR5 and an offset.

```
atomic_inc:     LDR     R1, ofs( R5 )         # Load current value and set reservation
                ADD     R1, R1, 1             # Increment locally
                STC     R1, ofs( R5 )         # Attempt conditional store
                CBR.EQ  R1, R0, atomic_inc    # Retry if store failed (R1 = 0)
                BE (RL)                       # return
```

Each iteration performs a reserved load, modifies the value, and attempts a conditional store. If another processor writes to the same cache line between the `LDR` and `STC`, the reservation is cleared, causing `STC` to fail and the loop to retry. This ensures that the final store reflects an atomic update to memory.

Compare-and-Swap (CAS) Example The following example implements the compare and swap function.

```
#   Arguments:
#      Arg0   : base address of the value location
#      Arg1   : expected old value
#      Arg2   : desired new value
#
#   Returns:
#      Arg0  : 1 on success, 0 on failure

cas:          LDR    R1, 0(Arg0)           # load reserved value
              CBR.EQ R1, Arg1, cas_fail    # compare with expected value
                                           # If not equal, fail
              STC    Arg2, 0(Arg0)         # Attempt conditional store
              MBR    Arg2, Arg2, cas       # Retry if store failed
              OR     Arg0, R0, 1           # Indicate success
              BE ( RL )                    # return

cas_fail:     OR     Arg0, R0, 0           # Indicate failure
              BE ( RL )                    # return
```

This CAS sequence allows software-based atomic updates: the memory location is only updated if its current value matches the expected value. Failed stores due to lost reservations cause the loop to retry, guaranteeing atomicity even in multiprocessor environments.

Summary Together, these examples demonstrate how the `LDR`/`STC` mechanism enables fully software-based atomic read–modify–write operations. By combining reserved loads with conditional stores, the processor can implement synchronization primitives, spinlocks, and lock-free data structures without additional hardware beyond the reservation register and cache-line tracking.

Appendix - Instruction Notation Routines

describe the routines used in the instruction set operation part...

Table 18: Instruction Notation Routines

Function	Description
addOfs32(base, ofs)	the offset is added to the base register using a 32-but unsigned arithmetic. The upper portion of the base register remains unchanged.
signExtend(field)	the field in an instruction is sign extended to a 64-bit integer value.
immOperand(instr)	the singed 15-bit immediate value field in the instruction is selected and sign extended.
adrImmIndexed(instr)	the signed 13-bit immediate value field in the instruction is selected, shifted left by the dw field and sign extended. It is added to the base register RegB using 32-bit unsigned arithmetic. The upper 32-bits of RegB remain unchanged.
adrRegIndexed(instr)	the lower 32-bits of RegX are shifted left by the dw field and sign extended. The value is added to the base register RegB using 32-bit unsigned arithmetic. The data is fetched according to the data width.
memLoadImmIndexed(instr)	the signed 13-bit immediate value field in the instruction is selected, shifted left by the dw field and sign extended. It is added to the base register RegB using 32-bit unsigned arithmetic. The upper 32-bits of RegB remain unchanged.
memLoadRegIndexed(instr)	the lower 32-bits of RegX are shifted left by the dw field and sign extended. The value is added to the base register RegB using 32-bit unsigned arithmetic. The data is fetched according to the data width.

Continued on next page

Function	Description
memStoreImmIndexed(instr, data)	the signed 13-bit immediate value field in the instruction is selected, shifted left by the dw field and sign extended. It is added to the base register RegB using a 32-bit unsigned arithmetic. The upper 32 bits of RegB remain unchanged. The data is stored according to the data width.
memStoreImmIndexed(instr, data)	the lower 32-bits of RegX are shifted left by the dw field and sign extended. The value is added to the base register RegB using a 32-bit unsigned arithmetic. The upper 32 bits of RegB remain unchanged. The data is stored according to the data width.
testBit(reg, pos)	the bit in the passed register at the position is tested.
compare(cond, reg1, reg2)	Reg2 is subtracted from Reg1 and evaluated for the condition. If the condition is true, a 1 is returned.
deposit(reg1, reg2, pos, len)	The right most field of len in Reg 2 is extracted and deposited at the position in Reg 1.
extract(reg1, reg2, pos, len)	The field of len at the position in Reg2 is extracted and returned in Reg1.
doubleShiftR(reg1, reg2, len)	Reg1 and Reg2 are concatenated and shifted right by len. The rightmost 64 bits of the result are returned.
diagOperation(id, reg1, reg2)	Diagnostic function id is passed reg1 and reg2.
flushFromCache(adr)	Cache flush operation.
purgeFromCache(adr)	Cache purge operation.
insertIntoTlb(adr, info)	TLB insert.
purgeFromTlb(adr, info)	TLB purge.

Appendix -Debug and Trace Support

Note: The examples and mechanisms described in this appendix are fully consistent with the interruption architecture and trap table presented in Chapter **??**. In Twin-64, the `BREAK` instruction is implemented as a `TRAP` instruction with a specific breakpoint trap code. Recognition of breakpoint traps is controlled by the **B** bit, and single-instruction stepping is controlled by the **T** bit. This appendix illustrates how these bits are used together to provide precise, deterministic debugging and tracing, including correct handling of branch instructions, instruction boundaries, and breakpoint reinstallation. All sequences assume the debugger maintains breakpoint addresses and trap codes independently of the processor's saved instruction address.

Twin-64 architecture provides two independent control bits in the processor status word to support debugging and controlled execution:

- **B** --- Break Enable
- **T** --- Trace (Single-Step) Enable

These bits control distinct exception mechanisms and are designed to be used together to implement software breakpoints and single-instruction stepping.

B - Break Enable Bit

The behavior of the `BREAK` instruction is controlled by the breakpoint enable flag, which determines whether the instruction triggers an exception or executes normally. When $\mathbf{B = 1}$, executing a `BREAK` instruction triggers a *breakpoint exception*, allowing the debugger or runtime environment to halt execution and inspect the architectural state.

In contrast, when $\mathbf{B = 0}$, `BREAK` instructions are treated as no-ops and do not generate an exception, enabling programs that contain `BREAK` instructions to run uninterrupted. A breakpoint exception occurs *before* the `BREAK` instruction completes, so the architectural state reflects all execution up to, but not including, the instruction itself. This behavior ensures that the system state is consistent and predictable at the moment the exception is taken, which is essential for accurate debugging and reliable exception handling.

The B bit affects only the recognition of `BREAK` instructions. It has no effect on other traps, faults, or exceptions.

The B bit exists to give software explicit control over breakpoint recognition. It allows software to temporarily disable breakpoints while handling a breakpoint exception, prevent recursive breakpoint exceptions, permit `BREAK` instructions to safely exist in debugger or privileged code, and enable a breakpoint instruction to be replaced, executed past, and reinstalled without requiring instruction emulation.

T - Trace (Single-Step) Enable Bit

The behavior of the trace mechanism is controlled by the trace enable flag, which determines whether a trace exception is generated after instruction execution. When $\mathbf{T = 1}$, the processor generates a *trace exception* immediately after the next instruction completes, regardless of the instruction type. The instruction executes normally, allowing it to modify the architectural state, access memory, and change control flow through branches, calls, returns, or traps. Once the instruction retires, the trace exception is taken and the **T** bit is automatically cleared. This ensures that the trace exception reflects a precise and consistent architectural state *after* the instruction has fully completed.

The trace exception is taken after, rather than during, instruction execution, and occurs even if the instruction is not a branch. The exception is generated after exactly one instruction, independent of the number of cycles required for execution. The trace mechanism operates independently of fault recovery or assist mechanisms, and the **T** bit is not implemented using micro-architectural progress counters, recovery counters, or cycle-based mechanisms.

The T bit exists to support precise debugger functionality. It enables single-instruction stepping, allows tracing immediately after instruction execution, and ensures that execution can resume precisely after one instruction without requiring instruction emulation.

Interaction Between B and T

The **B** and **T** bits control independent mechanisms with distinct exception timing. The **B** bit enables breakpoint instruction traps, which are taken *before* instruction execution, whereas the **T** bit enables trace (single-step) traps, which are taken *after* instruction execution. Both bits may be set or cleared independently, allowing flexible combinations for debugging and tracing. The four possible combinations of the **B** and **T** bits and their resulting behavior are summarized below.

Bit	Value	Effect / Trap Timing
B	1	Breakpoints enabled; executing a `BREAK` instruction triggers a breakpoint exception *before the instruction executes.The BREAK instruction is implemented as a TRAP instruction with a breakpoint trap code, recognized only when B=1.*
B	0	Breakpoints disabled; executing a `BREAK` instruction does nothing (no exception)
T	1	Trace / single-step enabled; after the next instruction completes (any instruction type), a trace exception occurs; T is cleared automatically after the trace exception is taken, ensuring single-step behavior is deterministic.
T	0	Trace / single-step disabled; instructions execute normally without trace exceptions

Table 19: Behavior of the B and T Bits

Software Breakpoint Handling

A typical debugger sequence for handling software breakpoints proceeds as follows. First, the debugger replaces the target instruction with a `BREAK` instruction and executes with $\mathbf{B = 1}$ and $\mathbf{T = 0}$.

When a breakpoint exception occurs, the debugger restores the original instruction, temporarily disables breakpoints ($\mathbf{B = 0}$), enables tracing ($\mathbf{T = 1}$), and resumes execution.

After the next instruction completes, a trace exception is generated, allowing the debugger to regain control. Finally, the breakpoint is reinstalled, and normal program execution resumes. This sequence ensures precise control over program execution while maintaining consistent architectural state.

Single-Step Execution

Single-instruction execution without triggering breakpoints is accomplished by setting $\mathbf{B = 0}$ and $\mathbf{T = 1}$ before resuming execution. The processor executes exactly one instruction and then generates a trace exception, allowing the debugger to inspect the state immediately after the instruction.

If the stepped instruction is a branch, the trace exception occurs at the branch target, while the debugger retains the original breakpoint address for reinstallation.

Design Rationale

The **B** bit governs whether `BREAK` instructions generate exceptions, while the **T** bit controls the trace mechanism, causing execution to halt between instructions. By separating these two functions, the architecture avoids the need for instruction emulation, simplifies debugger implementation, and guarantees that trace exceptions are taken precisely at instruction boundaries, regardless of instruction complexity or execution latency.

This separation provides a clear and predictable model for debugging: breakpoint exceptions occur before the offending instruction executes, and trace exceptions occur after a single instruction has completed. Together, the **B** and **T** bits enable a complete, precise, and low-overhead debugging framework, supporting both single-instruction stepping and controlled breakpoint handling without interfering with normal program execution.

Appendix - Diagnostic Functions

Table 20: Diagnostic Functions

Id	Function	Arg0	Arg1
0	No operation	0	0
1	Write status display	value	0
2	Clear TLB	0	0

Appendix - Translation Lookaside Buffer

Twin-64 TLBs support variable page sizes, aligned to the page size. The hash function still hashes to the base page when traversing the page table. The page table is now a set of entries rather than a one-to-one mapping between physical and virtual pages. Consequently, we maintain a table with one entry per physical page for memory management.

On a TLB miss, we start with the base page index and walk the collision chain. If no entry is found, we then examine the next larger region size, hashing the start address of that region. If the region is not found in any chain, a page fault is raised.

The main challenge arises during write-back. We want to write back only the modified portions, not the entire region. To achieve this, a region is initially allocated as read-only. Any write attempt triggers a trap, at which point the region is split into two parts: one remains read-only, and the other is marked dirty. The split size is critical, as the size of the dirty portion cannot be changed later.

In practice, large regions are typically read-only and contain data such as libraries or program code. As a result, write-back situations are relatively rare, and the overhead of the write-back algorithm is justified.

Appendix - Processor Design

The design of a processor can be approached in many different ways, with choices in instruction set architecture, datapath organization, execution resources, and pipeline structure all influencing the final implementation. The instruction set design is closely connected to the processor design itself, as the operations supported by the architecture determine the required hardware resources and the way instructions are processed.

During the design of the instruction set, the processor model described in this chapter was the envisioned target architecture. The instruction formats, execution model, and available operations were selected with this implementation in mind. Although many different micro architectural implementations are possible, this chapter describes one possible realization of the architecture.

The following sections outline the processor pipeline and describe the purpose of each pipeline stage. Two implementations are presented: a single instruction issue processor, which focuses on simplicity and efficient resource usage, and a dual instruction issue processor, which increases instruction throughput by allowing compatible instructions to execute in parallel.we will sketch a single instruction issue and a dual instruction issue processor.

Pipeline Stages

The reference design implements a three-stage instruction pipeline, with each stage divided into two substages. The pipeline stages and their respective substages are described below. This design allows instruction fetch, decoding, memory access, and execution to overlap, improving overall instruction throughput.

1. Fetch/Decode Stage This stage is responsible for retrieving instructions from memory, decoding them, and accessing the register file. It combines the initial instruction fetch with the preparation of operands required for execution.

Instruction Memory Access Substage Fetch the next instruction from the instruction cache using the program state register. The program counter (PC) is updated to point to the next sequential instruction or prepared for a branch target if a branch was predicted in a previous cycle. This substage ensures that the instruction is available for decoding without delay.

Instruction Decode Substage Decode the instruction opcode to determine the operation type, source and destination operands, and any immediate values. This substage also prepares control signals and identifies register file accesses needed for the next stages.

2. Access Stage The access stage handles address computation and memory operations. It is responsible for calculating memory addresses for load/store instructions and branch target addresses for conditional or unconditional branches, as well as accessing the data cache when required.

Address Computation Substage Compute memory addresses for load/store instructions or the target address for branch instructions. This includes evaluating effective addresses, offsets, and index computations as required by the instruction.

Memory Access Substage Perform read or write operations to the data cache. This substage ensures that data is available for execution or written back to memory as needed.

3. Execution Stage Substage This stage performs arithmetic, logical, and branch resolution operations, committing results to the destination registers or updating the program state register. Both substages are capable of executing a single operation per cycle.

Execution / Conditional Branch Resolution Substage Execute arithmetic or logical instructions and commit the results to the destination registers. For branch instructions, evaluate the branch condition and update the program state register accordingly to reflect the correct next instruction address.

Single Issue Pipeline

Twin-64 has a three stage pipeline, with each stage have two subStages. The single issue pipeline processes one instruction at a time through the processor pipeline. Although multiple instructions may be present in different pipeline stages simultaneously, only one instruction is fetched, decoded, and issued during each clock cycle. This organization provides a simple control model while still allowing instruction throughput to approach one instruction per cycle once the pipeline is filled.

The pipeline is divided into three main stages: Fetch/Decode, Access, and Execute. Each stage is further divided into substages to balance the amount of work performed during a clock cycle. The division allows instruction fetch, operand preparation, memory access, and execution of different instructions to overlap.

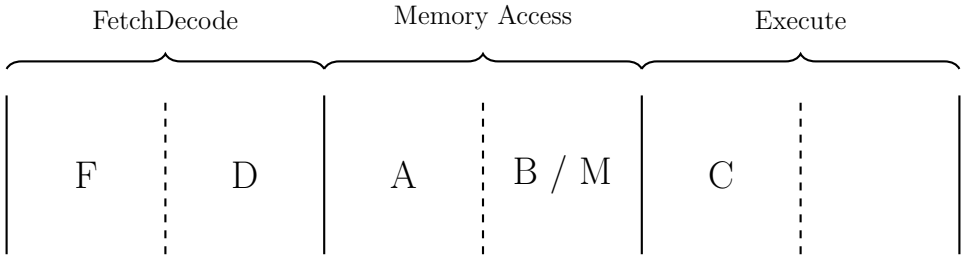


Figure 32: Single Instruction Pipeline

Fetch/Decode Stage The Fetch/Decode stage retrieves the next instruction from the instruction cache and decodes the instruction format. The decoder determines the operation type, identifies the source and destination registers, and prepares any immediate values required by the instruction. Register operands are read during this stage and passed to the following pipeline stages.

Access Stage The Access stage performs address calculation and memory operations. For load and store instructions, the effective memory address is calculated using a dedicated address generation unit. Address calculations use 32-bit adders, which are sufficient for the processor's memory addressing requirements. The data cache is accessed during this stage when required.

Execute Stage The Execute stage performs the actual instruction operation and commits the result. Arithmetic and logical instructions use the 64-bit ALU, while branch instructions are evaluated and resolved during this stage. System instructions perform control register operations or other processor management functions.

Since only one instruction is issued per cycle, all instructions share the same execution resources. This removes the need for complex instruction pairing logic and simplifies hazard detection and pipeline control. The single issue design therefore provides a compact and efficient processor implementation, while the dual issue pipeline extends this foundation by allowing compatible instructions to make use of additional pipeline parallelism.

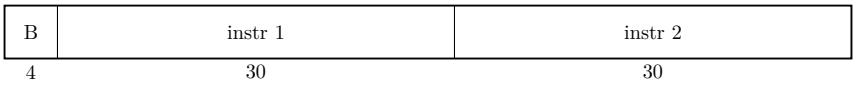
Dual Issue Pipeline

The dual issue pipeline extends the single issue design by allowing two independent instructions to be processed simultaneously. The pipeline remains divided into the Fetch/Decode, Access, and Execute stages, while the Access and Execute stages provide the additional resources required for two compatible instructions to proceed in parallel.

Instruction Bundle

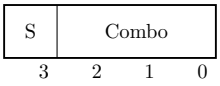
The instruction decoder interprets the two instructions contained in an instruction bundle. The processor supports the parallel execution of different instruction classes, including Compute, Branch, Memory, and System instructions.

Unlike the single issue architecture, where each instruction occupies a separate instruction word, the dual issue architecture fetches and decodes instruction bundles. Each bundle contains two instructions together with a bundle identifier that specifies the permitted instruction combination.



The two instructions are encoded using the standard Twin-64 instruction format. However, the instruction group field present in a single-issue instruction is omitted, as the instruction class is implied by the bundle identifier. Consequently, the bundle identifier determines the permitted combination of instruction types.

In addition to specifying the instruction combination, the bundle identifier contains a serialization flag. When this flag is set, the left instruction is guaranteed to complete before execution of the right instruction begins. This allows dependent instructions to be bundled while preserving their execution order.



The *Combo* field encodes the supported instruction bundle types. The bundle identifier is represented by a three-bit field. Of the eight possible encodings, five are currently assigned to instruction bundle types, while the remaining values are reserved for future use. This allows additional bundle types to be introduced without modifying the instruction format.

Value	**Bundle**	**Description**
0	CC	Two Compute instructions executed in parallel
1	CM	Compute instruction paired with a Memory instruction
2	CB	Compute instruction paired with a Branch instruction
3	CS	Compute instruction paired with a System instruction
4	BM	Branch instruction paired with a Memory instruction
5 .. 7		reserved

Instruction combinations not represented by one of the defined bundle types cannot be encoded within a single instruction bundle and must instead be issued as separate instructions. Restricting the available bundle types simplifies instruction decoding, hazard detection, resource allocation, and pipeline control while still covering the instruction pairings encountered most frequently in practice.

Instruction Pipeline

The dual issue pipeline closely resembles the single issue pipeline. The primary difference is that the Execute stage utilizes both execution substages, allowing two compatible computational operations to complete during a single pipeline cycle.

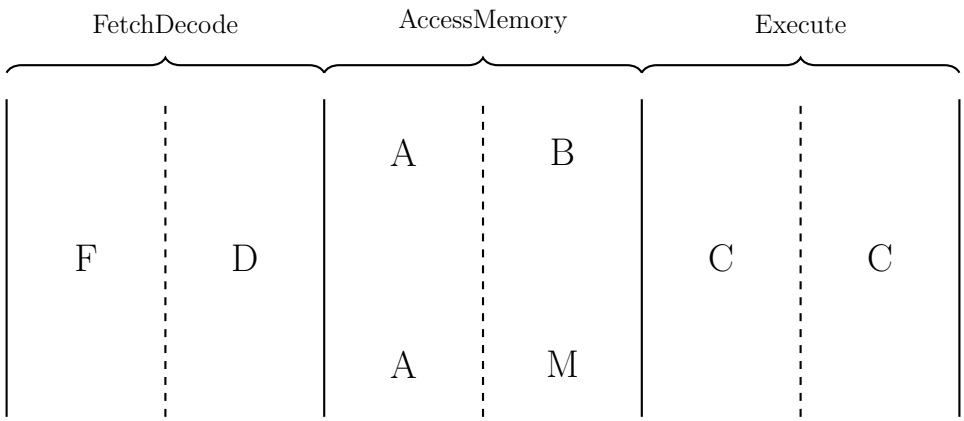


Figure 33: Dual Instruction Pipeline

The processor contains a single 64-bit ALU. Although two Compute instructions may be present simultaneously in the Execute stage, they occupy different execution substages. The ALU is therefore reused by successive substages rather than duplicated, allowing one computational result to be produced in each execution substage every clock cycle.

Fetch/Decode Stage The Fetch/Decode stage retrieves up to two instructions from the instruction cache and decodes their operation type and operand requirements. The decoder determines whether the instructions form a valid pair and generates the required control signals for the following stages. Register operands are read during this stage, allowing both instructions to enter the pipeline with their operands already available.

Access Stage The Access stage performs address generation and memory operations. Memory address calculation uses dedicated 32-bit adders, as the address space requirements do not require the full 64-bit arithmetic datapath. A Memory instruction can therefore calculate its effective address while another instruction performs an independent operation.

For load and store instructions, the memory access substage communicates with the data cache. The pipeline supports pairing a Memory instruction with a Compute or Branch instruction, provided that no data dependency or resource conflict exists.

Execute Stage The Execute stage performs arithmetic, logical, branch resolution, and system operations. The processor contains a single 64-bit ALU. This is sufficient because the two execution substages are separated in the pipeline and each substage can produce one result per cycle. A dual issue cycle therefore does not require two independent ALUs; instead, the existing execution resource is reused across the pipeline stages.

Compute instructions use the 64-bit ALU for arithmetic and logical operations, while Branch instructions use the execution stage for condition evaluation and program state updates. System instructions use the execution resources for control register access and privileged processor operations.

The dual issue pipeline improves instruction throughput by allowing independent instructions to overlap while avoiding unnecessary duplication of expensive execution resources. The supported instruction pairing rules provide a balance between increased performance and hardware complexity.

Design Rationale

The Twin-64 architecture is based on the principle that instruction-level parallelism should be exposed primarily by software rather than discovered dynamically by hardware. The instruction set therefore provides a small number of well-defined instruction bundle types that allow compatible operations to execute in parallel while preserving a simple and predictable execution model.

By limiting the supported instruction combinations, the processor avoids the complex scheduling, dependency analysis, and execution control required by more aggressive superscalar designs. The compiler is responsible for arranging instructions into suitable bundles whenever opportunities for parallel execution exist, while the processor verifies that the selected bundle can be executed safely.

This division of responsibilities results in a processor that offers increased instruction throughput without requiring extensive hardware resources. The architecture deliberately balances execution efficiency with implementation simplicity, making the processor easier to understand, verify, and implement while still exploiting a significant amount of instruction-level parallelism.

The name *Twin-64* reflects this architectural philosophy. The processor combines a 64-bit execution model with the ability to process two compatible instructions together. Rather than maximizing the amount of parallelism that can be extracted in hardware, Twin-64 provides a well-defined dual-issue execution model that allows hardware and compiler to cooperate in achieving efficient execution.

Appendix - Glossary

Absolute Address An address in the physical memory range from zero to the maximum physical memory size. This range is directly mapped to an equivalent virtual address range.

Address Space A logical domain of addresses. The CPU may define multiple address spaces, such as *main memory space* and *I/O space*. Each space defines an independent address range and access semantics.

Callee Save Registers Registers whose values must be preserved across a function call by the callee (the function being called). If the callee uses any of these registers, it must save their original values on entry (typically on the stack) and restore them before returning. This convention allows callers to assume the registers retain their values after the call.

Caller Save Registers Registers that may be overwritten by a called function. The caller must save these registers (if it needs their values later) before invoking another function. This convention allows the callee to use these registers freely without preserving their previous contents.

ELF Section A named subdivision in an ELF file used by the linker and compiler (e.g., `.text`, `.data`, `.bss`). Sections are grouped by the linker into ELF segments.

ELF Segment A loadable portion of an ELF file (defined by a program header). Each ELF segment is mapped into one or more regions of an address space during program loading.

Function Pointer A reference to a function descriptor.

Global Data Area All modules compiled for a load module allocate their global data in this area. The DP register points to the beginning of the area.

Global Data Pointer (DP) The address of the global data area. The address is normally kept in the DP register. Each module has a DP area in the global data area. Load modules also use this register to access the linkage table.

IA-relative Addressing Code that uses the current instruction address (IA) as the base for branches and for accessing constant data.

Linkage Table A table of address descriptors for functions external to the load module. One type of entry stores the address of an external function. Another type stores a reference to the external load module's global data area.

Load Module An executable unit of code produced by the assembler or compiler from one or several modules.

Mapped Region A runtime mapping that associates an ELF segment or memory source with a region in an address space. The emulator or loader tracks mapped regions to manage memory access and protection.

Object A unit of storage within a region. Objects correspond to program-visible entities or runtime allocations, such as global variables, heap blocks, or stack frames. Examples: *global object*, *heap object*, *stack object*.

Offset An unsigned 32-bit value that, together with the region, forms a virtual address. Examples: *code offset*, *data offset*.

Region A contiguous range of addresses within the virtual address space, reserved for a particular purpose. A region represents the upper 20 bits of the 52-bit virtual address range. Examples: *code region*, *data region*, *stack region*.

Virtual Address A virtual address consists of the region ID and the region offset. Together they form the architected virtual address space of 52 bits.

Appendix - Notes